

Computer Vision

2026. 1. 20

3C AI Campus
박 준 오

II. Deep Learning(CNN의 이해)

Machine Learning

(CNN의 이해)

II. Deep Learning(convolution의 이해)

1. CNN 주요 등장 배경

기존 fully connected 신경망의 한계

고차원 이미지 데이터를 그대로 처리할 경우, **파라미터 수가 폭증**하여 학습이 어렵고 오버피팅이 심해짐.

공간 구조 인식의 필요성

이미지는 **지역적 특징**(예: 모서리, 윤곽선)을 갖는데, 기존 MLP는 이를 인식하지 못함.

생물학적 영감

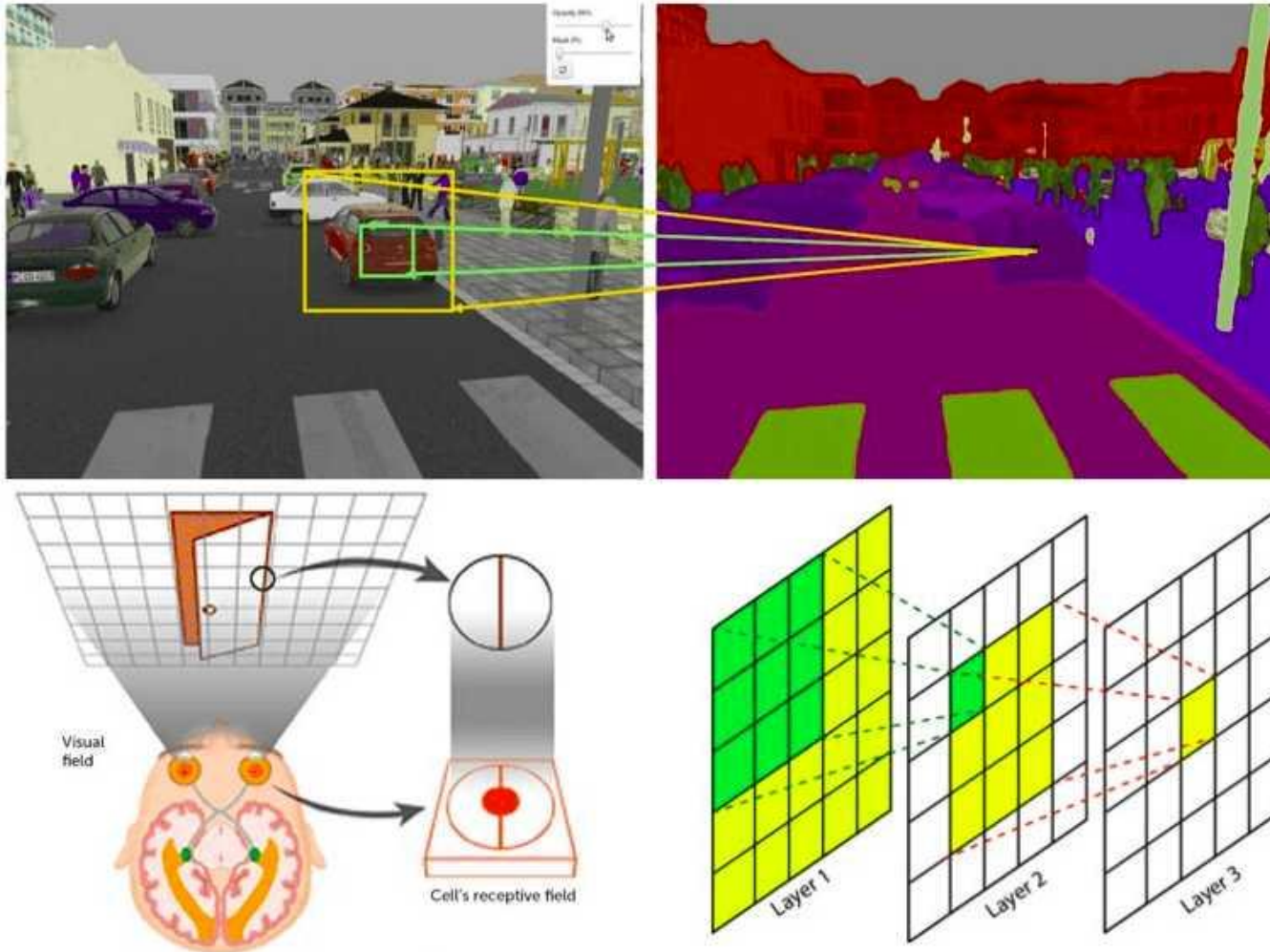
시각피질에서 **Receptive Field**가 존재한다는 사실에서 착안.

시각 피질의 뉴런이 우리 눈앞의 "작은 조각"들만 먼저 감지하고, 이를 조합하듯 마치 퍼즐을 맞추듯이 작은 정보들을 조합하여 전체적인 의미를 학습

대표적 시초

LeNet-5 (Yann LeCun, 1998): 숫자 인식(MNIST) 문제 해결을 위해 CNN 구조 등장.

II. Deep Learning(**convolution의 이해**)



II. Deep Learning(convolution의 이해)

2. CNN의 컨셉 진화 과정

1). 초기 문제: 기존 신경망의 한계

픽셀 간 관계를 무시하고, 픽셀단위의 학습은 모든 뉴런이 모든 픽셀과 연결되어 파라미터 수가 기하급수적으로 증가하는 문제가 발생

2). 해결의 실마리: 인간의 시각 시스템을 모방

뇌의 시각 피질(Visual Cortex)에서는 개별 뉴런이 작은 영역(Receptive Field)을 담당하고, 이 정보들을 결합하여 전체 이미지를 인식
→ 작은 영역을 먼저 분석하는 구조로 만들면 어떨까?

3. CNN 핵심 개념의 발전

Convolution Layer (합성곱 연산), ReLU (비선형성 도입) 정보 손실 방지

Pooling Layer (특징 요약) → 중요한 정보만 유지하고, 축소하여 연산량 감소

Fully Connected Layer (최종 분류) → 앞 단계에서 추출한 특징을 기반으로 클래스를 예측

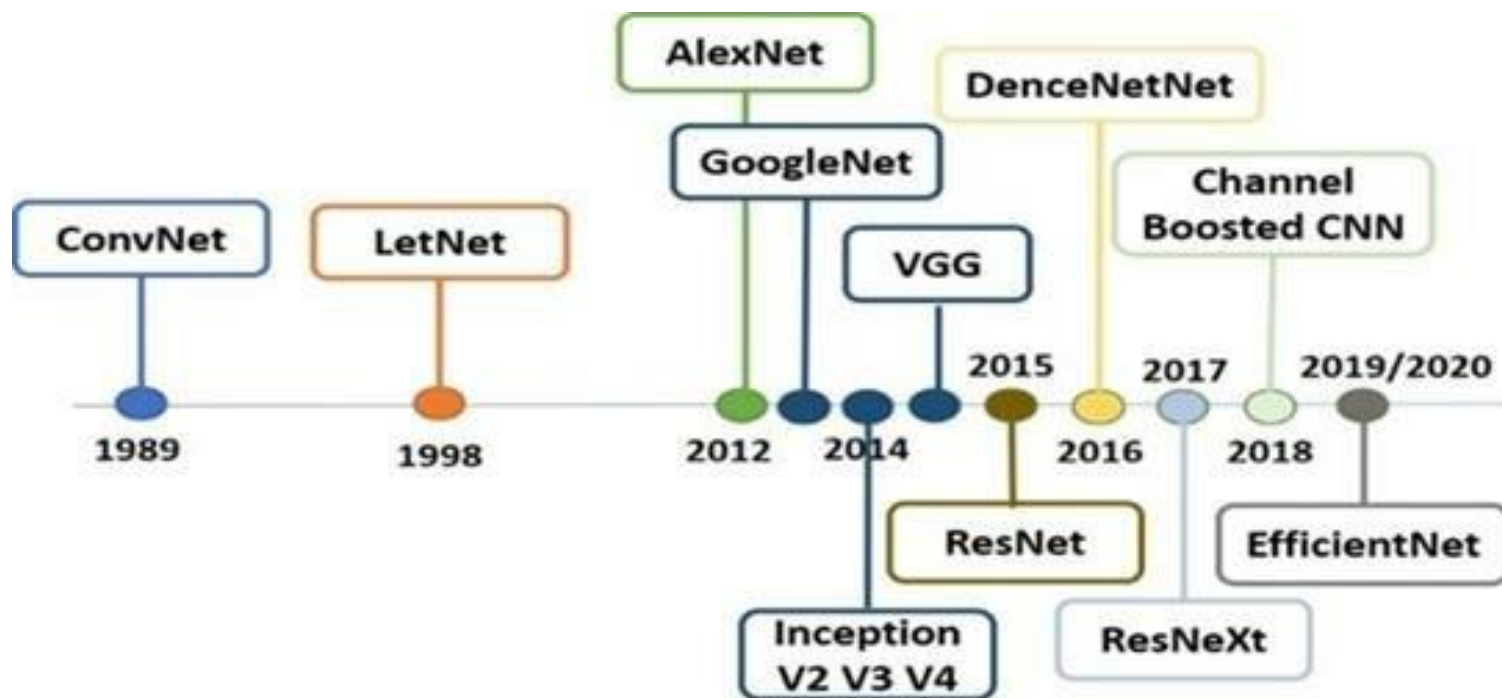
II. Deep Learning(convolution의 이해)

3. CNN의 응용 분야

분야	예시
이미지 분류	손글씨(MNIST), 사물 인식(ImageNet), 얼굴 인식
객체 탐지	YOLO, Faster R-CNN
세분화 (Segmentation)	자율주행, 의료 영상
영상 처리	슈퍼 해상도, 노이즈 제거
음성 처리	스펙트로그램 기반 음성 인식
자연어 처리 (제한적)	문장 내 패턴 추출 (CNN for text classification)

II. Deep Learning(convolution의 이해)

4. CNN 진화 과정



II. Deep Learning(convolution의 이해)

4. CNN 진화 과정

1) 네트워크 깊이 증가 (LeNet → AlexNet → VGG → ResNet)

초기에는 단순한 구조에서 점점 더 깊고 복잡한 구조로 발전.
하지만 기울기 소실 문제 발생 → ResNet에서 해결.

2) 연산량 최적화 (GoogLeNet, DenseNet, EfficientNet)

단순히 깊이를 늘리는 것이 아니라, 연산 효율과 성능을 모두 고려하는 방향으로 발전.

3) 잔차 학습과 피쳐 재사용 (ResNet, DenseNet)

깊은 네트워크에서도 학습이 가능하도록 Residual Connection(ResNet), Dense Connection(DenseNet) 개념 등장.

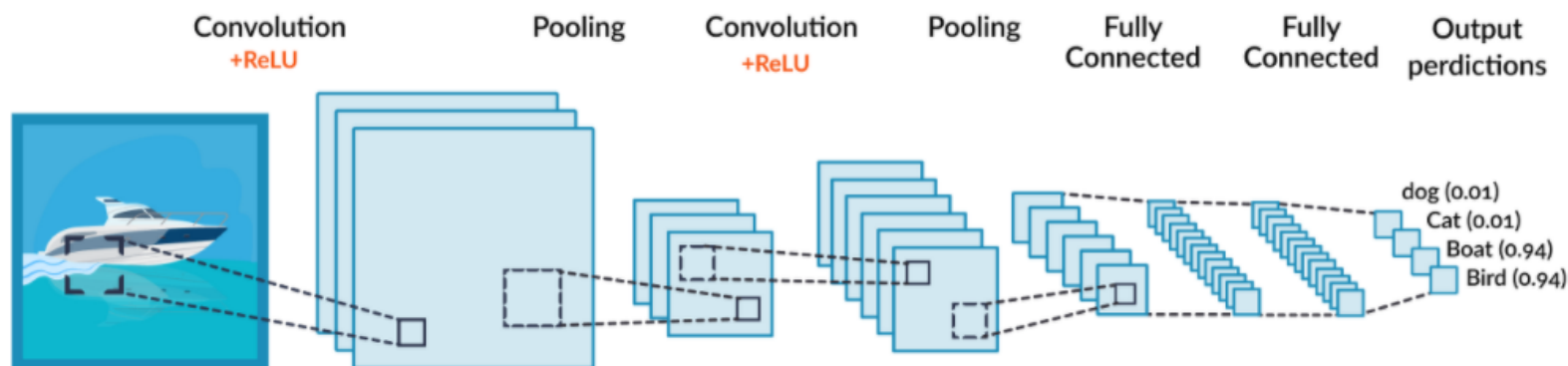
4) 최적의 네트워크 확장 (EfficientNet)

너비, 깊이, 해상도를 동시에 조절하여 최적의 성능을 찾는 기법 도입.

II. Deep Learning(convolution의 이해)

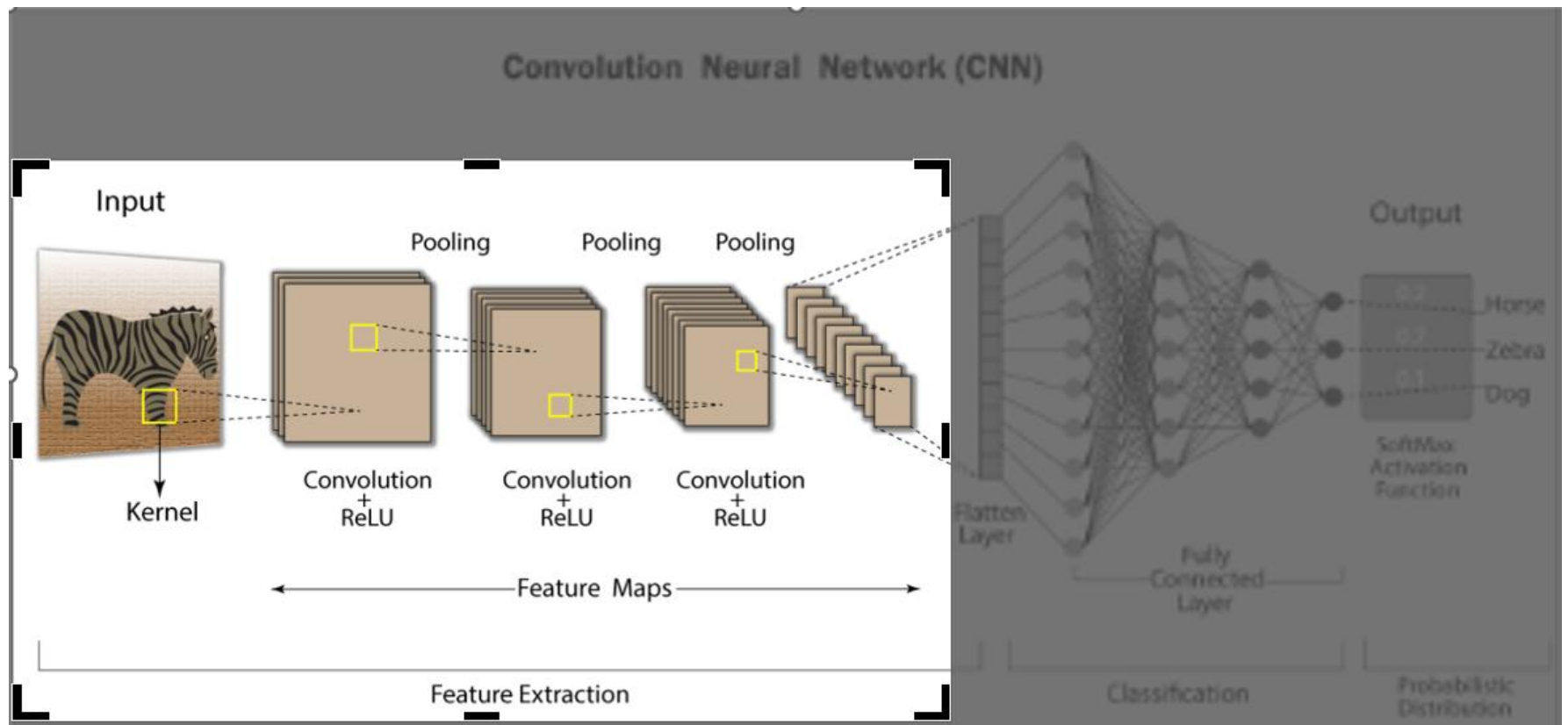
5.CNN(Convolution Neural Network) – convolution NN, 합성곱신경망

- 특히 이미지 및 비디오 처리에 활용
- 핵심 요소는 커널(**convolution** 연산시 입력 관련 기능 추출)이라는 필터
- 입력 데이터의 특징을 추출하여 특징들의 패턴 파악하는 구조
- Convolution과정과 Pooling 과정으로 진행



II. Deep Learning (convolution의 이해)

CNN(Convolution Neural Network) : Convolution + ReLU + Pooling ...



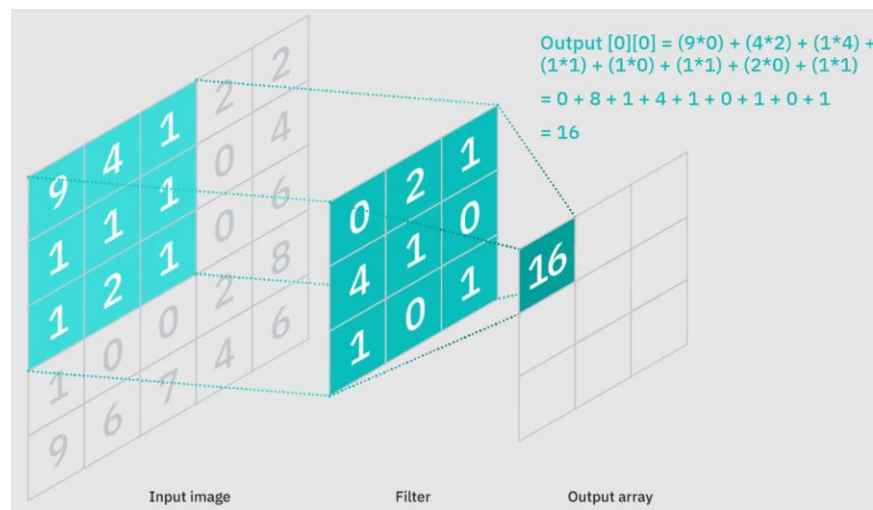
II. Deep Learning

Convolution(합성곱)

이미지나 다차원 데이터에 대해 **작은 필터를 이용하여 합성곱 연산을 수행**하는 것.

필터는 입력 데이터의 일부 영역과 가중치를 곱한 값을 모두 더하여 출력을 생성.
필터를 이동하며 입력 데이터에 대해 **합성곱 연산을 수행하면 특징 맵(Feature Map)이 생성.**

Convolution은 **입력 데이터의 패턴을 감지하고, 이미지에서 특징을 추출하는** 역할을 수행.
합성곱 연산을 통해 필터의 가중치를 학습하여 이미지의 특징을 감지하는 더 나은 필터를 생성할 수도 있다.

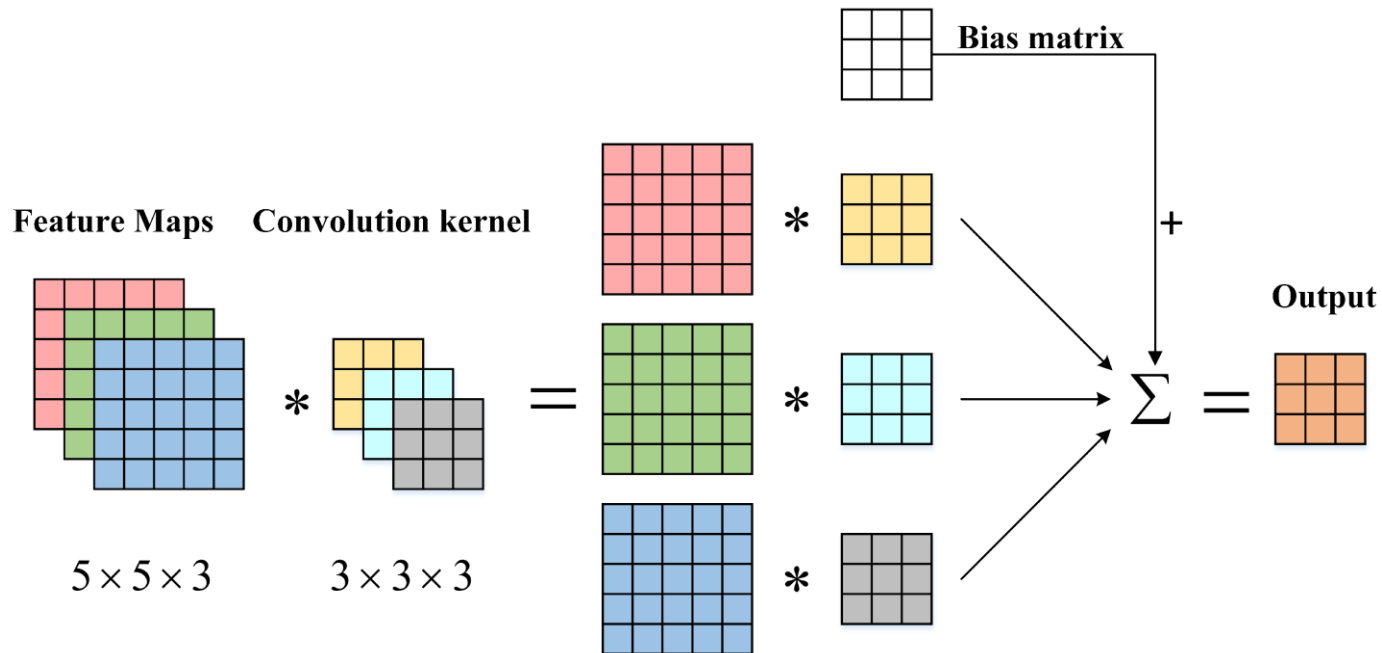


II. Deep Learning

Convolution

이미지나 다차원 데이터에 대해 **작은 필터를 이용하여 합성곱 연산을 수행하는 것.**

합성곱 연산의 결과에 **bias**를 추가하여 결과를 출력한다.



II. Deep Learning 활성화 함수 Relu

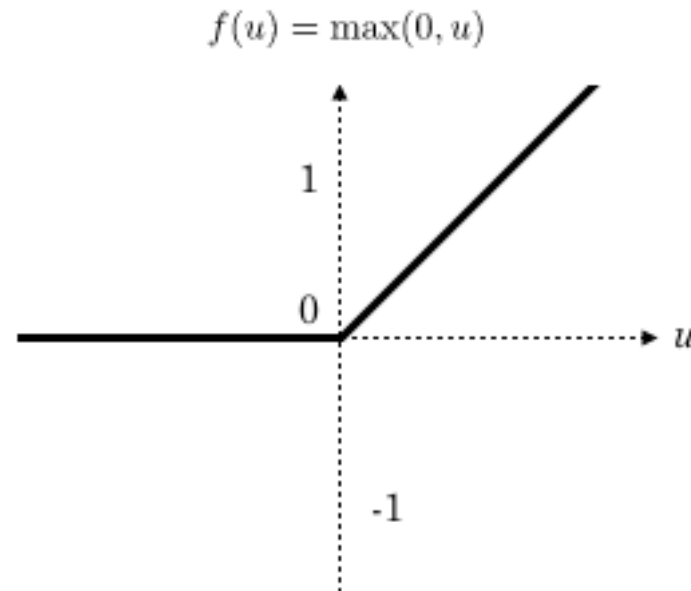
6. Relu 함수 : 활성화 함수

Convolution의 출력값에 적용되는 활성화 함수.

입력값이 0보다 작으면 0으로 출력하고, 0보다 크면 그 값을 그대로 출력.

음수 값을 제거하고 양수 값을 유지하는 역할.

$$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$$

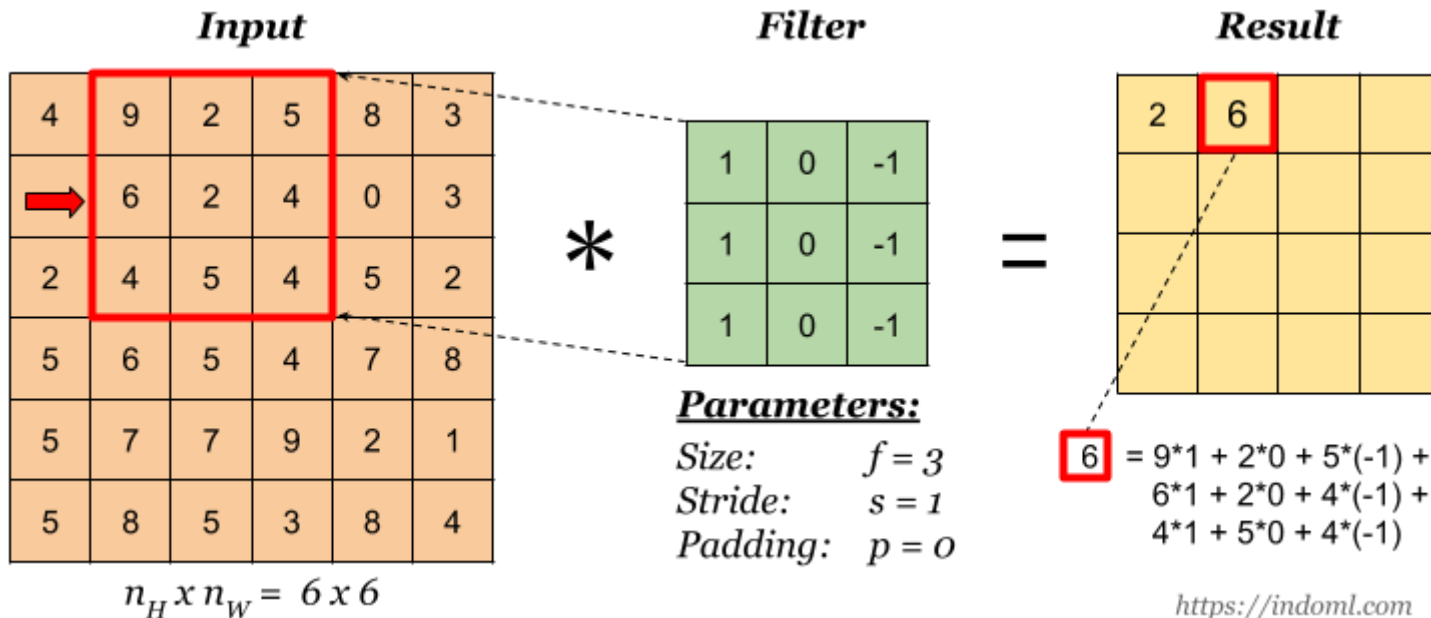


II. Deep Learning Stride

7.Stride : 필터의 이동 (단위)

Convolution 필터(kernel)가 입력 데이터를 얼마나 이동하는지를 결정하는 값.
일반적으로 Stride 값은 1로 설정.

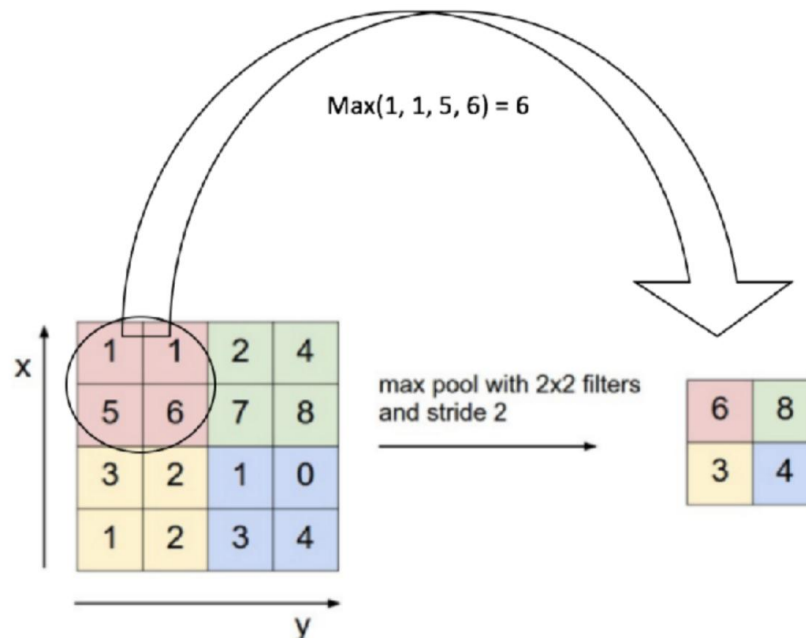
Stride 값을 늘리면 필터가 더 큰 간격으로 이동하게 되어 **출력 데이터의 공간적인 크기를 줄일 수 있고** Convolution 연산을 수행하는 영역의 크기를 조절할 수 있다.



II. Deep Learning Max Pooling

8. Max Pooling

Max Pooling은 입력 데이터에서 **최댓값을 선택**하여 출력하는 방식.
이를 통해 입력 데이터의 **공간적인 크기를 줄이고, 불필요한 정보를 감소**.
Pooling은 공간적인 불변성을 강화하고, 특징의 위치에 상대적인 정보를 제공합니다.
많은 파라미터를 줄여 연산량을 감소시키고, 모델의 **복잡도를 줄이는** 효과도 있다.

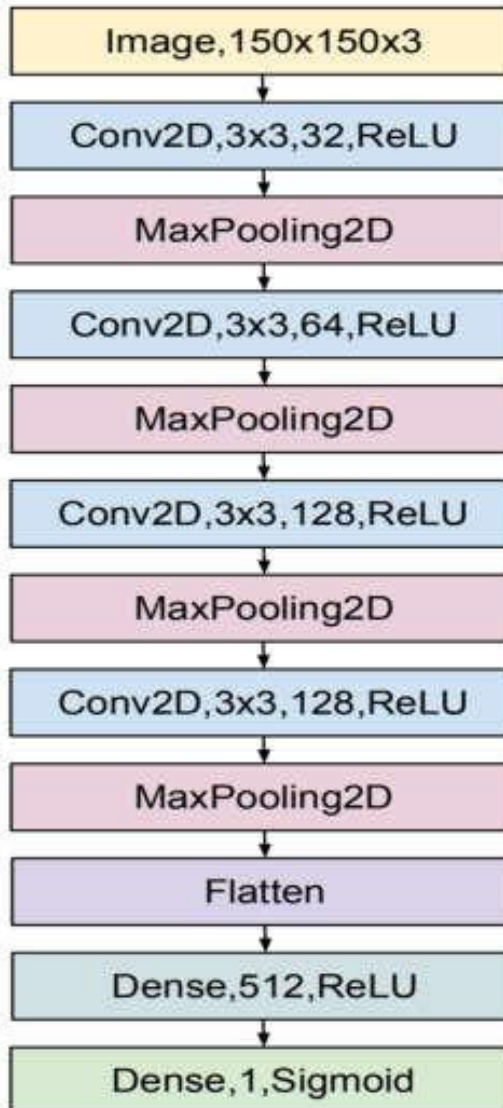


II. Deep Learning

입력된 내용에
3*3의 필터를 사용하여
32개의 맵을 생성

입력된 내용에
3*3의 필터를 사용하여
128개의 맵을 생성

입력된 내용에
512개의 노드를 가진
FC에 Relu 연결



입력된 내용에
3*3의 필터를 사용하여
64개의 맵을 생성

입력된 내용에
3*3의 필터를 사용하여
128개의 맵을 생성

입력된 내용에
1개의 노드를 가진
FC에 Sigmoid 연결

실습 예제 : 합성곱의 원리를 엑셀에서 구현하여 봅시다.

The figure illustrates the steps of a 3D convolution operation on a 6x6x3 input volume.

Input Volume (Grid 1): A 6x6x3 volume with values ranging from 1 to 9. A 3x3x3 kernel is highlighted in yellow, covering the top-left corner of the volume.

Kernel (Grid 2): A 3x3x3 kernel with values 1, 0, and -1, repeated across the depth planes. The kernel is highlighted with a red dashed border.

Output Volume (Grid 3): The resulting 6x6x3 output volume after the convolution operation. The output values range from -11 to 9. The value 6 is highlighted in red in the top-middle position of the output volume.

II. Deep Learning Padding

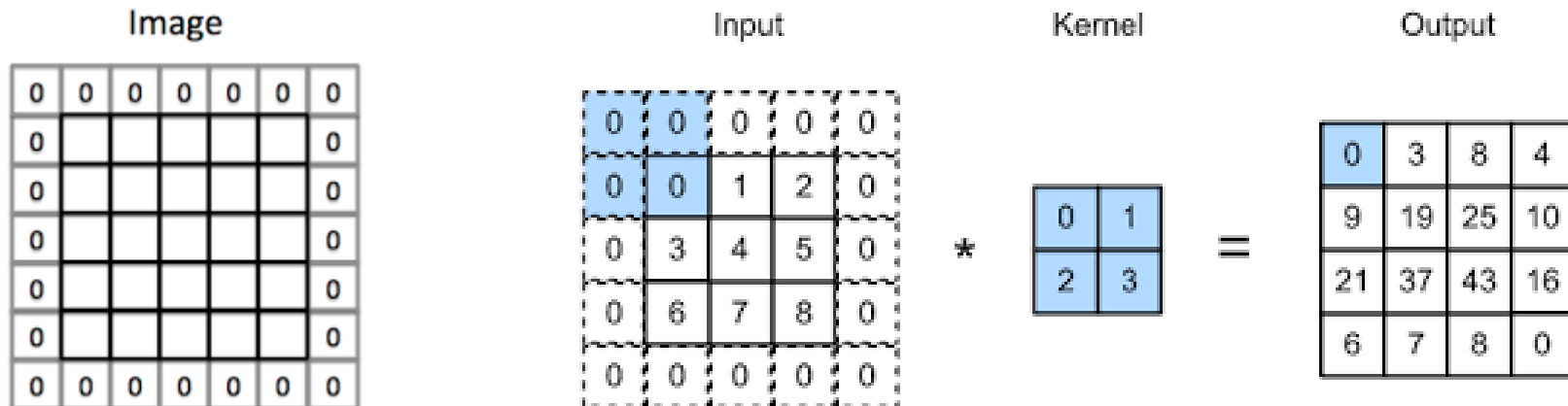
9.Padding : 입력 데이터의 주변 감싸기

패딩은 입력 데이터 주위에 적절한 개수의 0을 추가.

Convolution 연산 후 출력 데이터 크기를 조절하고, 경계에 있는 정보의 손실을 방지.

패딩을 적용하지 않은 경우에는 Convolution 연산을 반복하면서 출력 데이터의 크기가 점점 작아지게 된다.

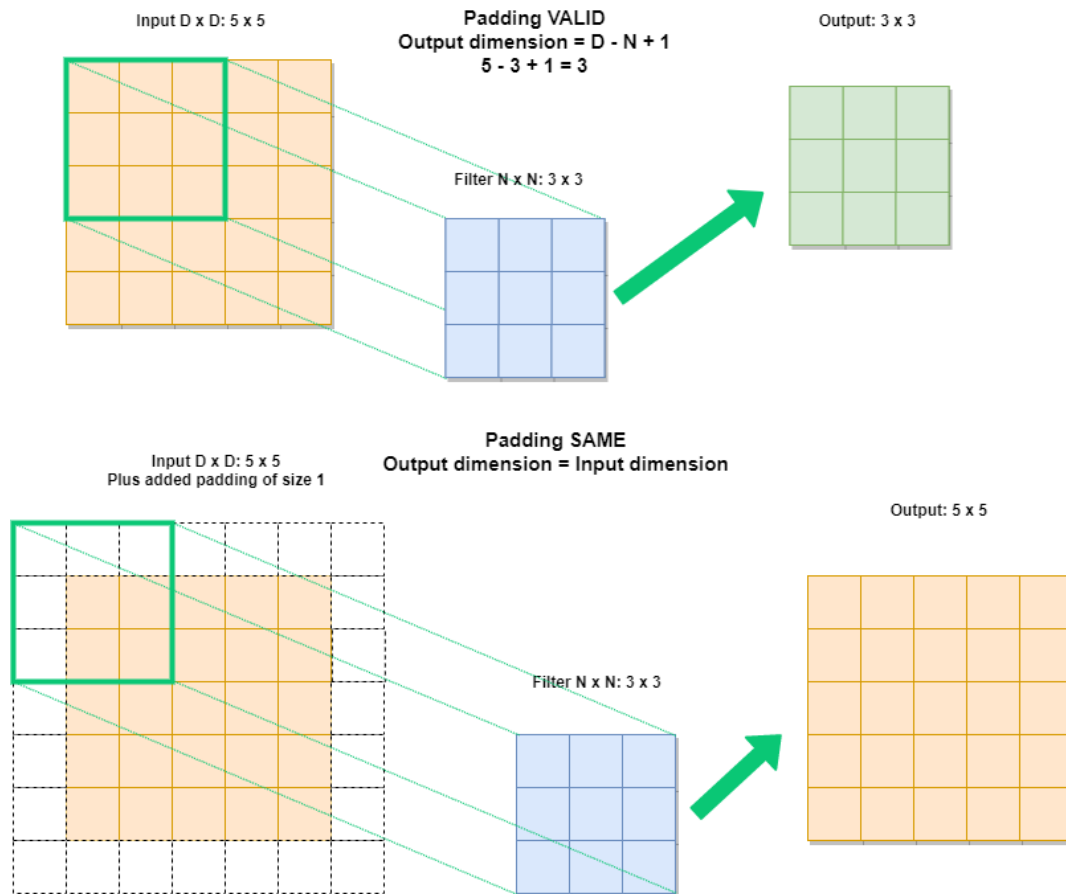
패딩을 적용한 경우에는 Convolution 연산을 하더라도 출력 데이터의 크기를 보존할 수 있다.



II. Deep Learning

Padding

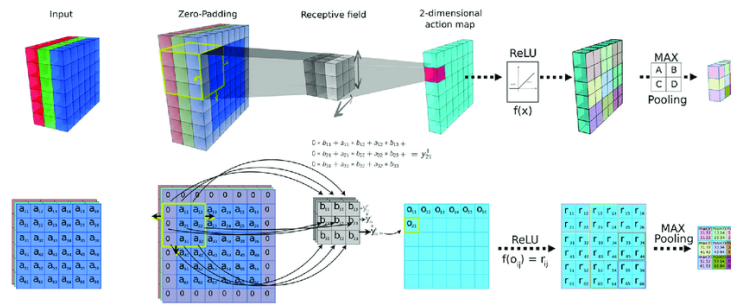
패딩을 적용한 경우에는 Convolution 연산을 반복하더라도 출력 데이터의 크기를 보존할 수 있다.



II. Deep Learning

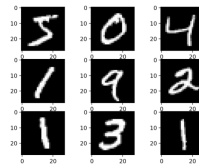
Convolution과 pooling은 feature를 뽑아내는 과정

<https://docs.google.com/spreadsheets/d/1wBiMTvbzDQH2r21mLMXbifTJtlqWUOIRUOnPcGHSsls/edit#gid=87581659>



II. Deep Learning CNN 모델링과 주요 용어

CNN의 모델링 사례 분석

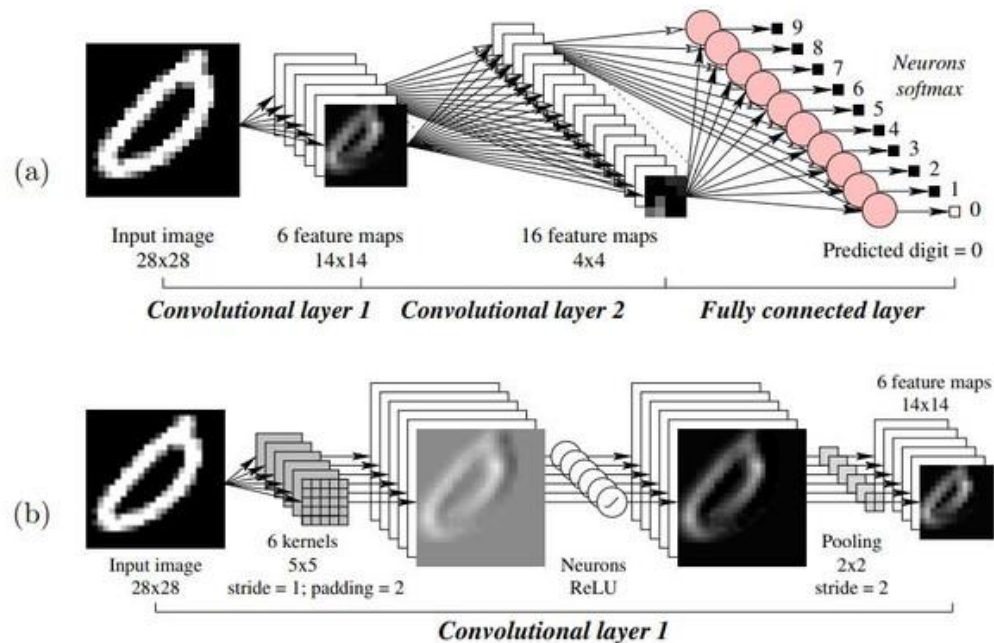


II. Deep Learning

CNN : 이미지의 특징을 추출한 합성곱 과정(convolution layer)
입력 이미지와의 합성곱(filter/stride) > 활성화 함수 > 특징 추출(맥스 풀링)

Input image > **Convolution Layers** > Fully connected layer > Output(classification)

Convolution Layer : Convolution by kernel > Activation func(ReLu) > Max pooling
(필터에 의한 합성곱 이미지는 필터의 개수만큼 생성된다 !!)

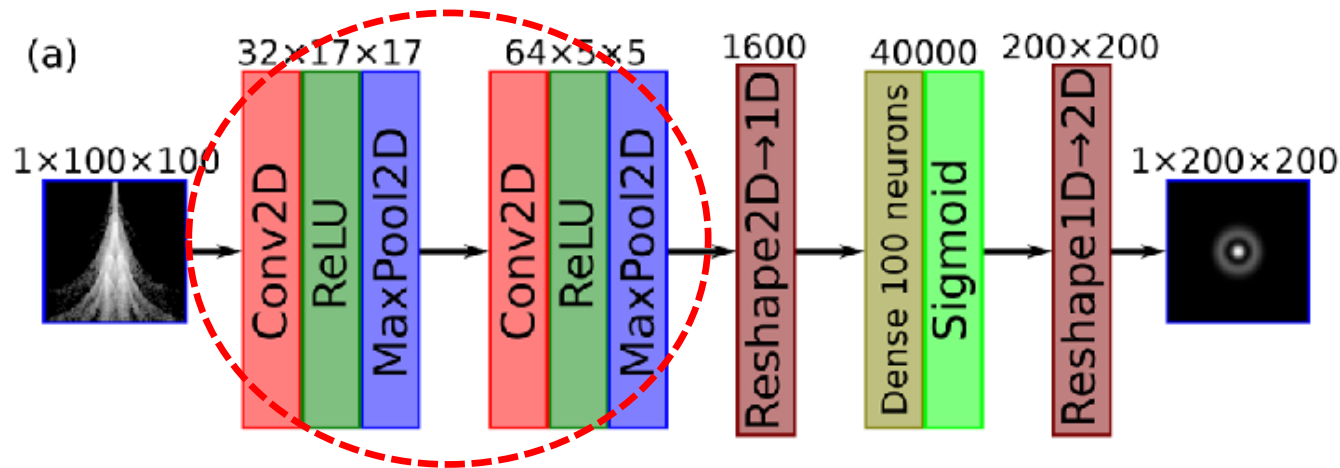


II. Deep Learning

CNN : 이미지의 특징을 추출한 합성곱 과정(convolution layer)

입력 이미지와의 합성곱(filter/stride) > 활성화 함수 > 특징 추출(맥스 풀링)

Convolution Layer : Convolution by kernel (**Conv2D**) > Activation(**Relu**) > **Max pooling**



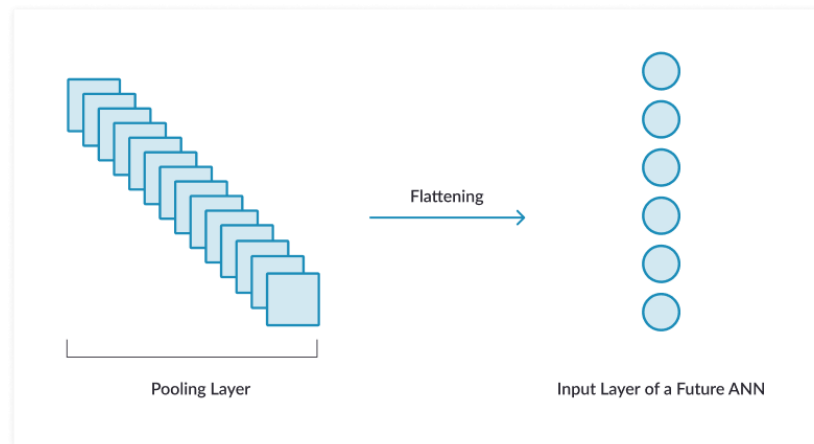
II. Deep Learning **flatten layer**

10. Flatten Layer :

Feature를 추출한 후 **입력 데이터의 차원을 평탄화하여 1차원 벡터로 변환**하는 역할.
FC는 입력을 1차원으로 받아들이기 때문에,
합성곱 레이어의 출력을 1차원으로 변환해야 한다.

이미지나 다차원 데이터를 특징 벡터로 변환(Flatten 레이어를 통해)

입력 데이터의 **공간적인 구조를 잃지만**, 대신 일련의 특징을 나타내는 벡터로 변환하여 다음 레이어에서 패턴을 인식하고 분류할 수 있다.

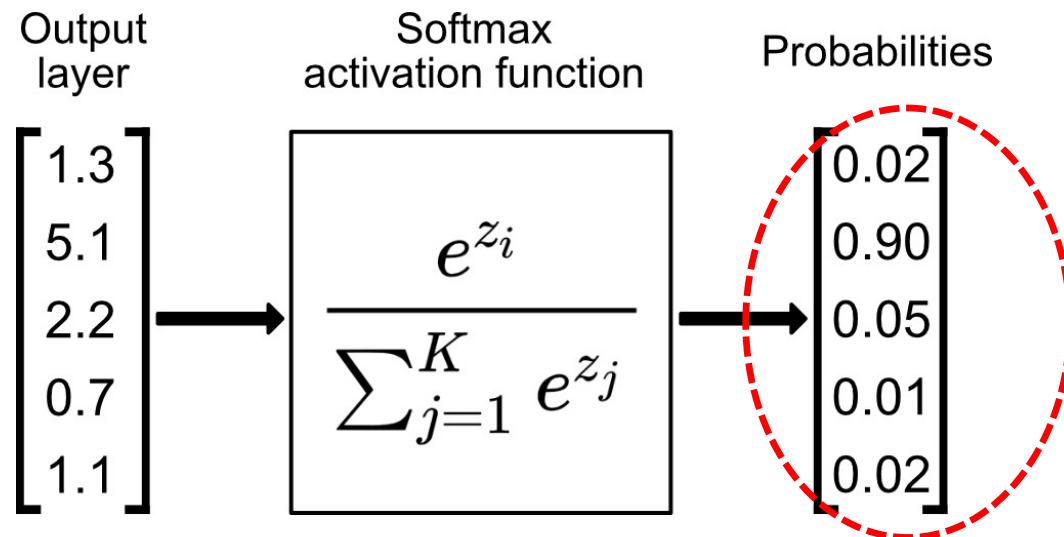


II. Deep Learning **Softmax(출력층 활성화 함수)**

11. Output Layer(Softmax)

CNN에서 **출력층을 구성하는 활성화 함수**로, 일반적으로 **Softmax 함수**가 사용된다.

Softmax 함수는 다중 클래스 분류 문제에서 각 클래스에 속할 확률을 출력
입력값을 클래스에 대한 확률로 변환,
출력값은 0과 1 사이의 값을 가지며, 모든 클래스에 대한 확률의 합은 1이 된다.

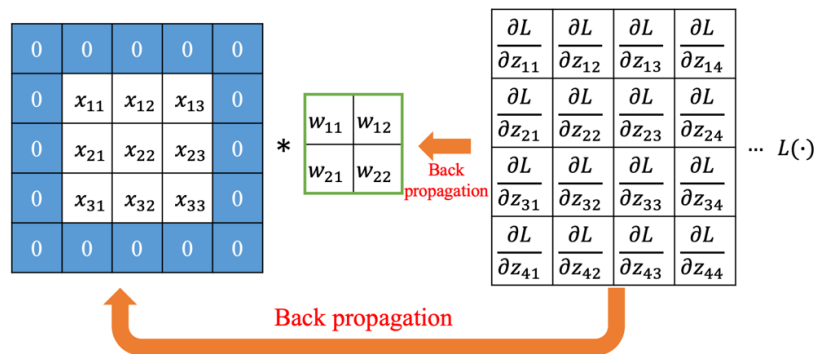


II. Deep Learning **Back propagation**

12. Back propagation

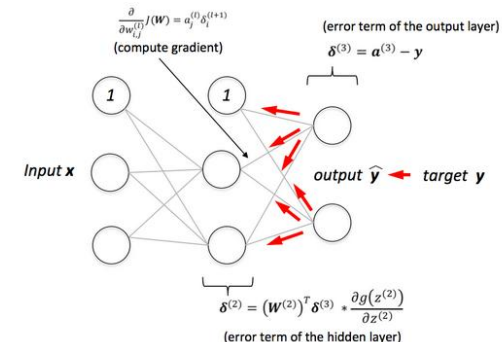
예측값의 정확도를 높이기 위해 **출력값과 실제 예측값을 비교 (Loss = 오차함수)하여 역방향으로 진행하며 가중치를 변경하는 작업**

역전파 과정은 출력층에서 시작하여 입력층(또는 최초의 히든 레이어)까지 거슬러 올라가며 각 레이어의 가중치를 업데이트.
업데이트가 끝나면 순전파 과정을 다시 시작하여 새롭게 업데이트된 가중치를 사용해 신경망을 통과한 다음, 다시 역전파를 진행하여 가중치를 더욱 최적화



Padding (p)= 1 $L(\cdot)$ = Loss function.
Stride (s)= 1

Created by brilliantcode.net



II. Deep Learning

필터 종류	주 역할
수직 에지 필터	세로 경계 탐지
수평 에지 필터	가로 경계 탐지
대각선 에지	사선 구조
코너 필터	모서리
블러/평균 필터	저주파 정보
고주파 필터	텍스처

II. Deep Learning

Deep Learning(CNN)의 구현
(convolution neural network)

II. Deep Learning **conv2D()**

13. **conv2D (input, filter, strides, padding...)**

합성곱의 구현(convolution by kernel)

이미지의 특징을 추출하기 위해 입력 데이터에 필터(커널)를 적용하여 출력을 계산. 컨볼루션 연산은 입력 데이터와 필터 간의 곱셈을 수행하고, 합산하여 출력을 생성

Image Input: 합성곱 연산을 위한 입력 데이터 **[batch, in_height, in_width, in_channels]**
예시로 50개의 배치로 30*30크기의 칼라 이미지 입력 **[50,30,30,3]**

* TensorFlow(Keras)에서는 학습(model.fit) 단계에서 배치 크기를 따로 지정하지 않으면 자동으로 **batch_size=32**가 사용

파라미터

- **filters:** 필터의 개수 지정. 출력 채널의 개수
예시로 **필터 4*4, 입력채널 3개, 필터 개수 32개 [4,4,3,32]**
- **kernel_size:** 보통 정사각형 형태로 지정하며, (height, width) 형태로 튜플로 지정.
- **strides:** 필터를 적용하는 간격 지정. 기본값은 (1, 1)
- **padding:** 'valid'는 패딩을 사용하지 않음을 의미하고,
'same'은 출력 크기를 입력과 동일하게 유지하기 위해 패딩을 추가.
- **activation:** 활성화 함수를 지정. 일반적으로는 ReLU

II. Deep Learning



► [Keras 3 API documentation](#) / [Layers API](#) / [Convolution layers](#) / [Conv2D layer](#)

Conv2D layer

Conv2D class

[\[source\]](#)

```
keras.layers.Conv2D(
    filters,
    kernel_size,
    strides=(1, 1),
    padding="valid",
    data_format=None,
    dilation_rate=(1, 1),
    groups=1,
    activation=None,
    use_bias=True,
    kernel_initializer="glorot_uniform",
    bias_initializer="zeros",
    kernel_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    bias_constraint=None,
    **kwargs
)
```

Conv2D layer

Conv2D class

About Keras

Getting started

Developer guides

Code examples

Keras 3 API documentation

Models API

Layers API

The base Layer class

Layer activations

Layer weight initializers

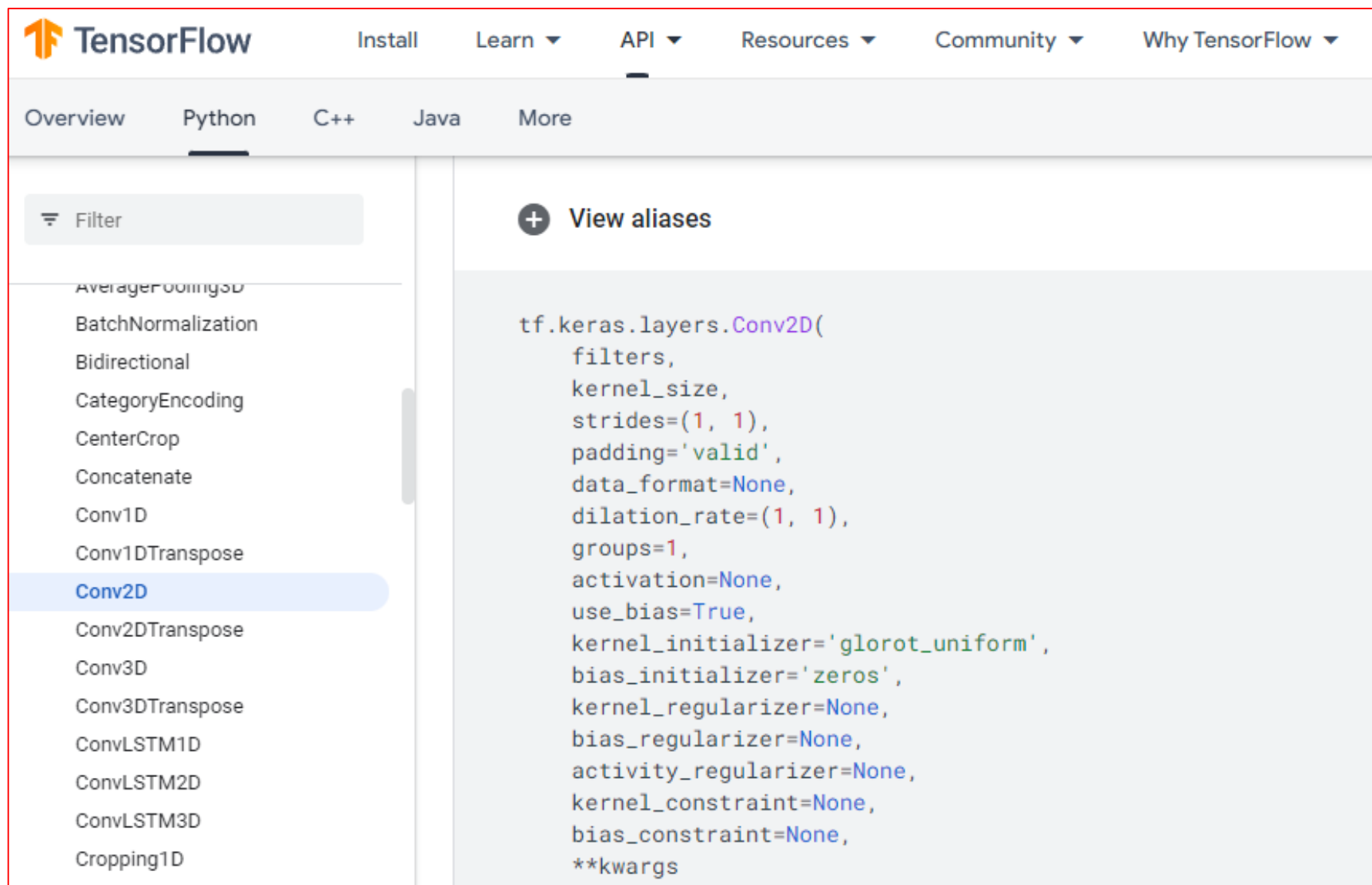
Layer weight regularizers

Layer weight constraints

Core layers

Convolution layers

II. Deep Learning



The screenshot shows the TensorFlow API documentation page for `tf.keras.layers.Conv2D`. The page has a top navigation bar with links for **TensorFlow**, **Install**, **Learn**, **API**, **Resources**, **Community**, and **Why TensorFlow**. Below this is a sub-navigation bar with **Overview**, **Python**, **C++**, **Java**, and **More**. The **Python** tab is selected. On the left side, there is a sidebar with a **Filter** button and a list of layers. The **Conv2D** layer is highlighted in blue. The main content area on the right shows the **View aliases** section, which contains the following code snippet:

```
tf.keras.layers.Conv2D(  
    filters,  
    kernel_size,  
    strides=(1, 1),  
    padding='valid',  
    data_format=None,  
    dilation_rate=(1, 1),  
    groups=1,  
    activation=None,  
    use_bias=True,  
    kernel_initializer='glorot_uniform',  
    bias_initializer='zeros',  
    kernel_regularizer=None,  
    bias_regularizer=None,  
    activity_regularizer=None,  
    kernel_constraint=None,  
    bias_constraint=None,  
    **kwargs
```

II. Deep Learning

```
# Conv2D 레이어
conv = tf.keras.layers.Conv2D(
    filters=4,      # 출력 채널 수
    kernel_size=(3,3), # 필터 크기
    strides=(1,1),
    padding='same',  # 출력 크기 동일
    activation=None
)

# Conv 연산 수행
outputs = conv(inputs)
print("출력 shape:", outputs.shape) # (2, 5, 5, 4)
```

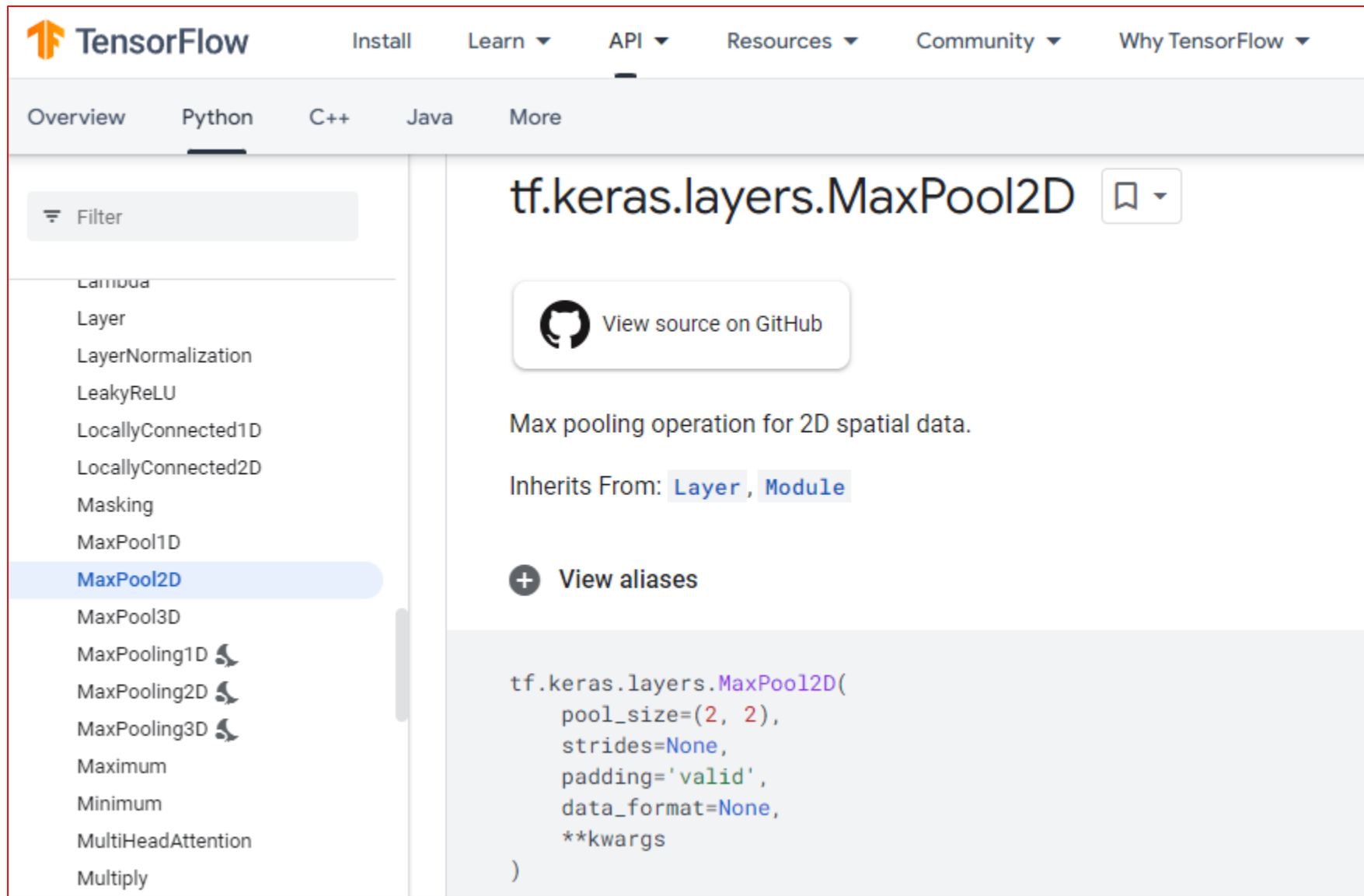
padding='same': 입력 이미지의 공간 크기를 출력에서도 유지
입력 채널(RGB) 3개를 하나의 필터가 모두 계산한 뒤,
출력 채널별로 다른 필터로 4개의 결과를 뽑는 과정

☞ batch=2는 모든 과정을 2장에 병렬로 돌리기 때문에 shape 그대로 유지!

II. Deep Learning

 Keras About Keras Getting started Developer guides Code examples Keras 3 API documentation Models API Layers API The base Layer class Layer activations Layer weight initializers Layer weight regularizers Layer weight constraints Core layers	Search Keras documentation... Keras 3 API documentation / Layers API / Pooling layers / MaxPooling2D layer MaxPooling2D layer MaxPooling2D class [source] <pre>keras.layers.MaxPooling2D(pool_size=(2, 2), strides=None, padding="valid", data_format=None, name=None, **kwargs)</pre> Max pooling operation for 2D spatial data. Downsamples the input along its spatial dimensions (height and width) by taking the maximum value over an input window (of size defined by <code>pool_size</code>) for each channel of the input. The window is shifted by <code>strides</code> along each dimension. The resulting output when using the "valid" padding option has a spatial shape (number of rows or columns) of: <code>output_shape = math.floor((input_shape - pool_size) / strides) + 1</code> (when <code>input_shape >= pool_size</code>) The resulting output shape when using the "same" padding option is: <code>output_shape = math.floor((input_shape - 1) / strides) + 1</code>	MaxPooling2D layer MaxPooling2D class
---	---	---

II. Deep Learning



The screenshot shows the TensorFlow API documentation for the `tf.keras.layers.MaxPool2D` class. The page is part of the TensorFlow documentation site, with navigation links for Install, Learn, API, Resources, Community, and Why TensorFlow. The left sidebar lists various layers, with `MaxPool2D` highlighted. The main content area shows the class name, a link to view the source on GitHub, a description of the max pooling operation, the classes it inherits from (`Layer` and `Module`), a link to view aliases, and a code snippet for creating a `MaxPool2D` layer.

TensorFlow Install Learn API Resources Community Why TensorFlow

Overview **Python** C++ Java More

Filter

- Camunda
- Layer
- LayerNormalization
- LeakyReLU
- LocallyConnected1D
- LocallyConnected2D
- Masking
- MaxPool1D
- MaxPool2D**
- MaxPool3D
- MaxPooling1D
- MaxPooling2D
- MaxPooling3D
- Maximum
- Minimum
- MultiHeadAttention
- Multiply

tf.keras.layers.MaxPool2D

[View source on GitHub](#)

Max pooling operation for 2D spatial data.

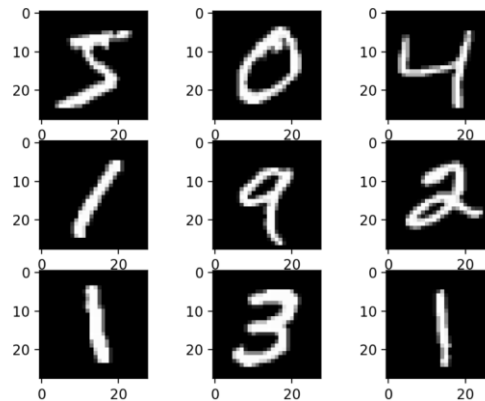
Inherits From: [Layer](#), [Module](#)

[+ View aliases](#)

```
tf.keras.layers.MaxPool2D(  
    pool_size=(2, 2),  
    strides=None,  
    padding='valid',  
    data_format=None,  
    **kwargs  
)
```

II. Deep Learning (ANN/CNN with MNIST)

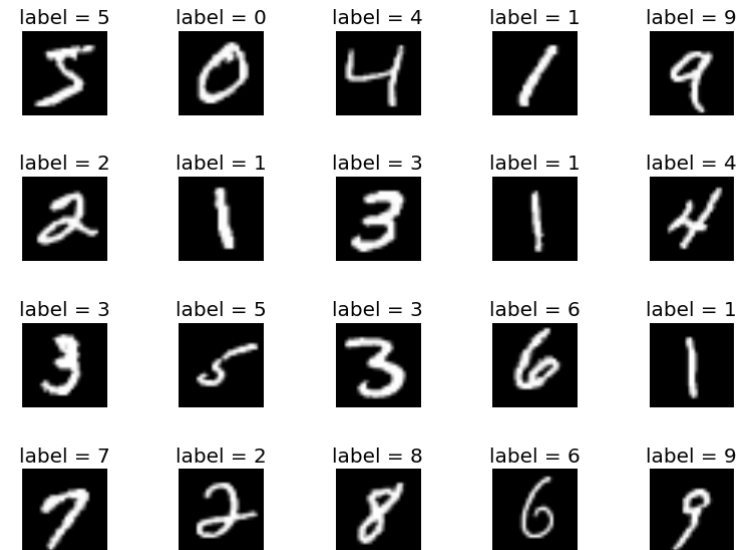
ANN/CNN with MNIST



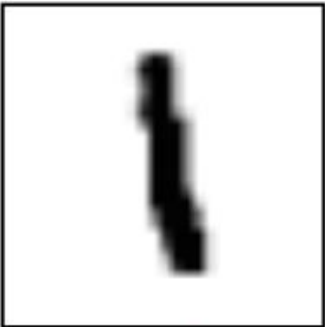
II. Deep Learning

MNIST Dataset

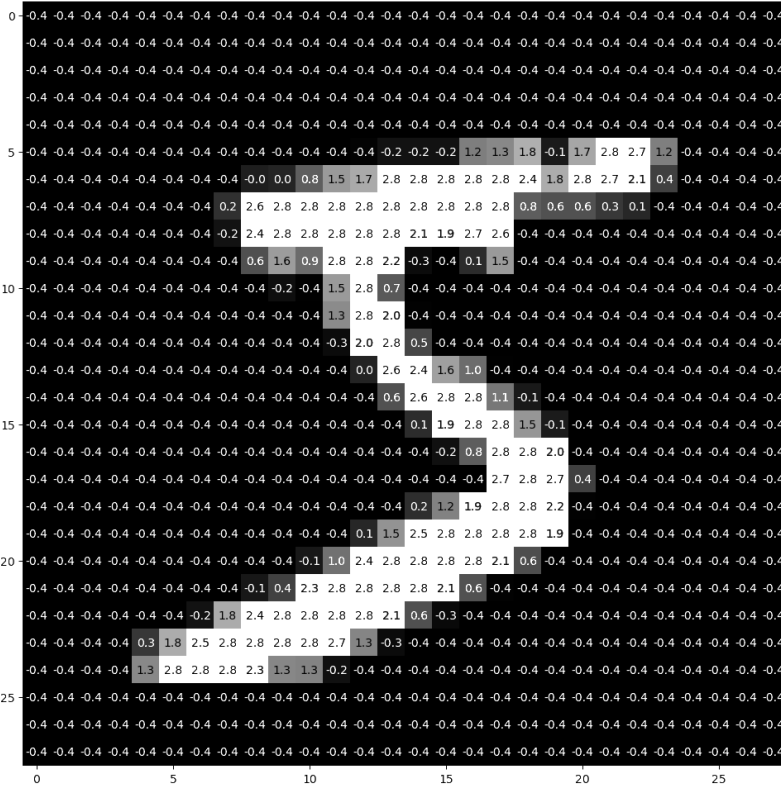
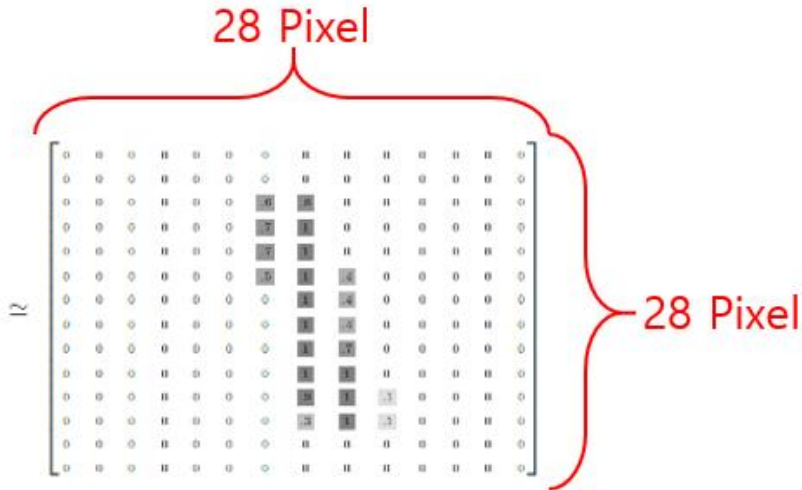
- 6만 개의 training 이미지와 1만개의 test 이미지.
- 각 이미지는 손으로 쓴 0~9의 숫자들을 스캔한 것
- 28 x 28 픽셀 영역에서 각 픽셀은 0~255 사이의 값, 회색조 칼라.
- 각 이미지에는 0~9까지의 해당 숫자 값이 분류명(label)으로 부여되어 있다.



II. Deep Learning



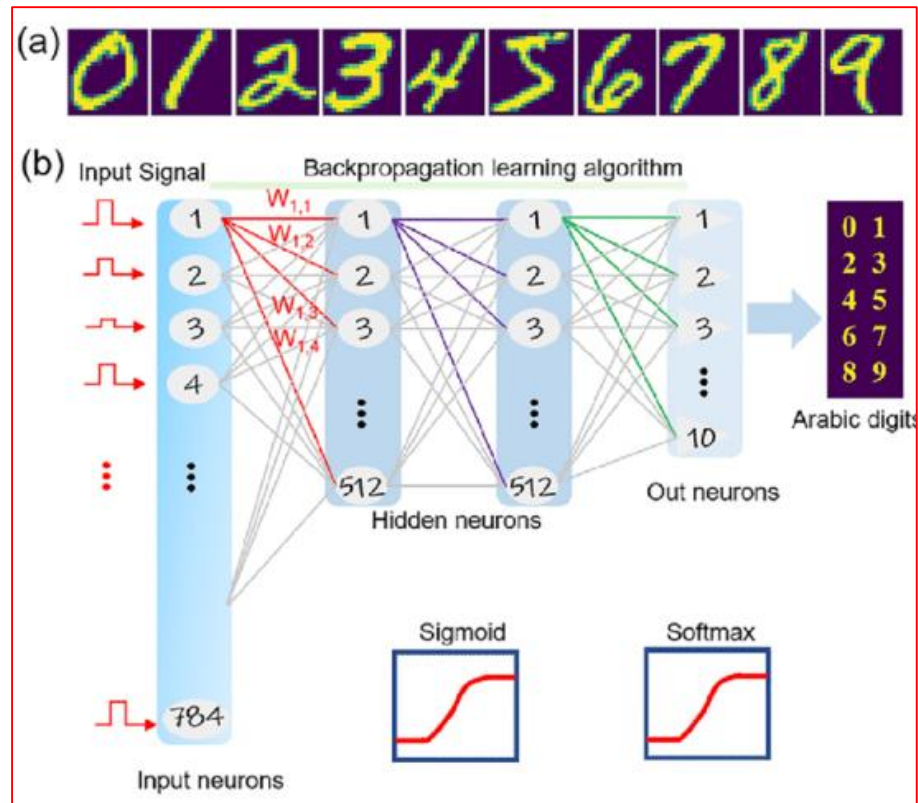
숫자 1



II. Deep Learning

Train Data > 입력층(28*28) > 은닉층 > 출력층(0~9) > Optimizer > Training closing

(Feed Forward + Back Propagation(weight&bias update with optimizer))



II. Deep Learning

Deep Learning 순서도(ANN/CNN)

1. Data definition : import **mnist.load_data()**
2. Data preprocessing : **normalization(Min-Max)**, **to_categorical()**
3. Modeling : **Sequential(), model.add(),**
Dense() for ANN / Conv2D(), MaxPool2D(), Flatten() for CNN
4. Modeling_Compile : **model.compile(), optimizer, loss = categorical_crossentropy**
(one hot encoding이 아니라면 **sparse_categorical_crossentropy**)
5. Model train : **model.fit()**
6. Model evaluation : **model.evaluate(), confusion matrix**

II. Deep Learning

ANN with MNIST
(Full code)

II. Deep Learning (ANN with MNIST)

```
1 import tensorflow as tf
2 import numpy as np
3 from tensorflow.keras.datasets import mnist
4
5 (x_train, L_train), (x_test, L_test) = mnist.load_data()
6
7 print('\n train shape = ', x_train.shape,
8       ', Label shape = ', L_train.shape)
9 print(' test shape = ', x_test.shape,
10       ', Label shape = ', L_test.shape)
11
12 print('\n train Label = ', L_train) # 학습데이터 정답 출력
13 print(' test Label(L_test) = ', L_test) # 테스트 데이터 정답 출력
```



```
train shape = (60000, 28, 28) , Label shape = (60000,)
test shape = (10000, 28, 28) , Label shape = (10000,)
```



```
train Label = [5 0 4 ... 5 6 8]
test Label(L_test) = [7 2 1 ... 4 5 6]
```

II. Deep Learning (ANN with MNIST)

```
1 import tensorflow as tf
2 import numpy as np
3 from tensorflow.keras.datasets import mnist
4
5 (x_train, L_train), (x_test, L_test) = mnist.load_data()
6
7 print('#n train shape = ', x_train.shape,
8       ', Label shape = ', L_train.shape)
9 print(' test shape = ', x_test.shape,
10       ', Label shape = ', L_test.shape)
11
12 print('#n train Label = ', L_train) # 학습데이터 정답 출력
13 print(' test Label(L_test) = ', L_test) # 테스트 데이터 정답 출력
```

```
train shape = (60000, 28, 28) , Label shape = (60000,)
test shape = (10000, 28, 28) , Label shape = (10000,)
```

```
train Label = [5 0 4 ... 5 6 8]
test Label(L_test) = [7 2 1 ... 4 5 6]
```

**csv file로 받아 볼 경우는 정답데이터 1개와
0부터255까지의 숫자가 784(28*28)가 콤마로 분리되어 존재한다.
또한 케라스에 내장된 데이터는 정답과 훈련데이터가 분리되어 있다**

1 x_train[0]

ndarray (28, 28) [show data](#)



1 L_train[0]

np.uint8(5)

II. Deep Learning (ANN with MNIST)

```
1 first_data_train = x_train[0]
2
3 for row in first_data_train:
4     for pixel in row:
5         print(f'{pixel:3d}', end=' ')
6     print()
7
```

x_train 첫번째 데이터를 입력

첫번째 데이터의
행과 열의 입력자료를 출력

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 3 18 18 18 126 136 175 26 166 255 247 127 0 0 0 0
0 0 0 0 0 0 0 0 0 30 36 94 154 170 253 253 253 253 253 225 172 253 242 195 64 0 0 0 0
0 0 0 0 0 0 0 0 49 238 253 253 253 253 253 253 253 251 93 82 82 56 39 0 0 0 0 0
0 0 0 0 0 0 0 0 18 219 253 253 253 253 253 198 182 247 241 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 80 156 107 253 253 205 11 0 43 154 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 14 1 154 253 90 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 139 253 190 2 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 11 190 253 70 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 35 241 225 160 108 1 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 81 240 253 253 119 25 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 45 186 253 253 150 27 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 16 93 252 253 187 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 249 253 249 64 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 46 130 183 253 253 207 2 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 39 148 229 253 253 253 250 182 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 24 114 221 253 253 253 253 201 78 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 23 66 213 253 253 253 253 198 81 2 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 18 171 219 253 253 253 253 195 80 9 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 55 172 226 253 253 253 253 244 133 11 0 0 0 0 0 0 0 0 0 0
0 0 0 0 136 253 253 253 212 135 132 16 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```


II. Deep Learning (ANN with MNIST)

```
1 first_data_test = x_test[0]
2
3 for row in first_data_test:
4     for pixel in row:
5         print(f'{pixel:3d}', end=' ')
6     print()
7
```

x test 첫번째 데이터를 입력

첫번째 데이터의 행과 열의 입력자료를 출력

[illegible]

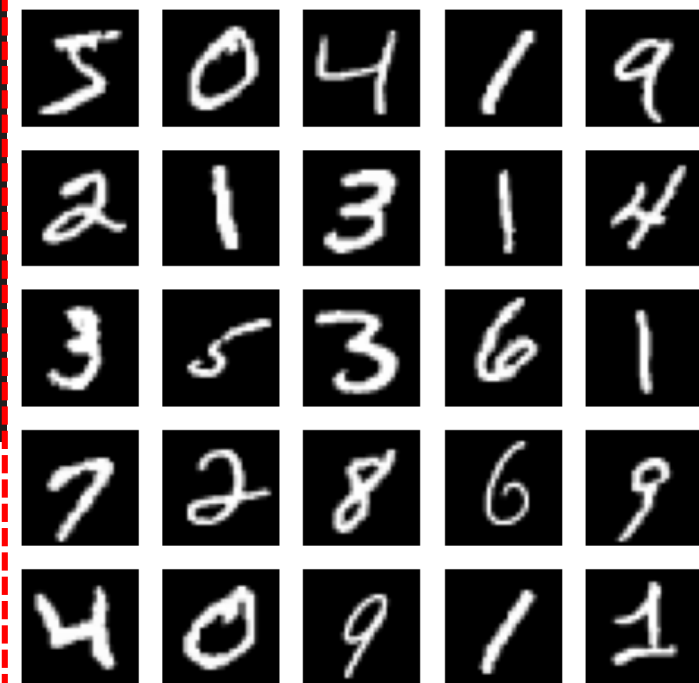
II. Deep Learning

```
1 import matplotlib.pyplot as plt
2
3 # 25개의 이미지 출력
4 plt.figure(figsize=(6, 6))
5
6 for index in range(25):    # 25 개 이미지 출력
7
8     plt.subplot(5, 5, index + 1) # 5행 5열
9     plt.imshow(x_train[index], cmap='gray')
10    plt.axis('off')
11    # plt.title(str(t_train[index]))
12
13 plt.show()
```

x_train 첫번째 데이터~ 25번째 데이터를 출력

첫번째 데이터의
행과 열의 입력자료를 출력

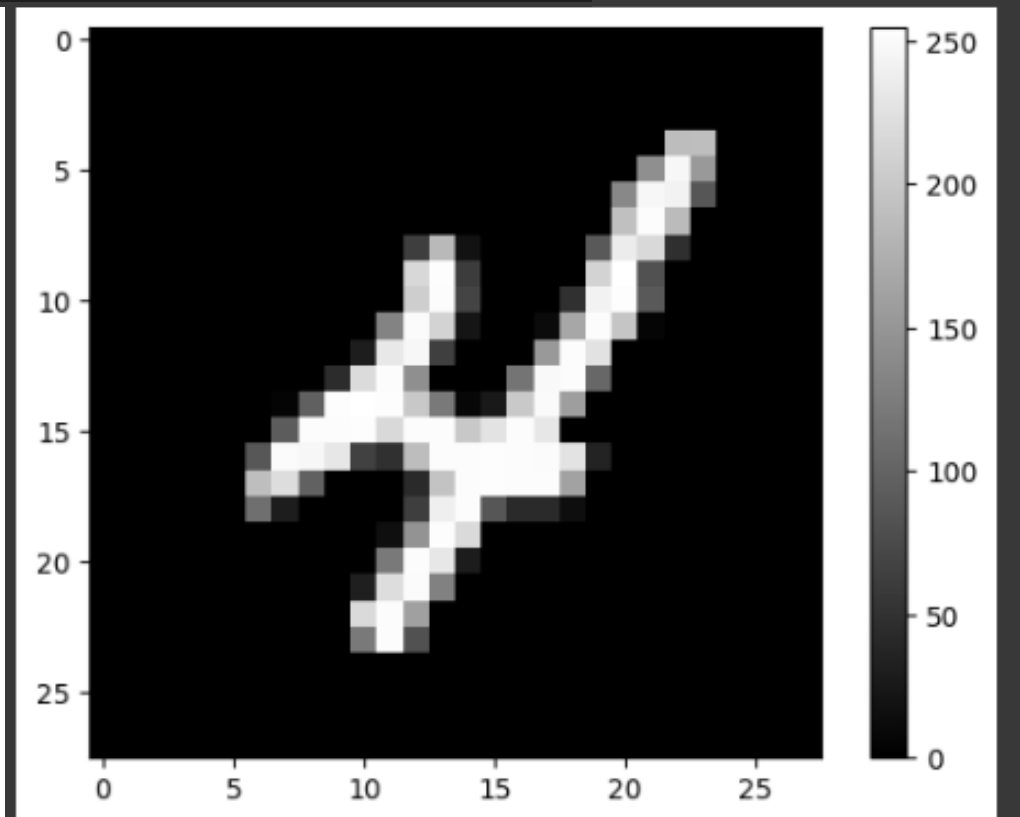
figsize=(6,6)은 그림의 가로,세로 길이 (인치)



II. Deep Learning

```
1 plt.imshow(x_train[9], cmap='gray')  
2 plt.colorbar()  
3 plt.show()
```

x_train 10번째 데이터를 출력



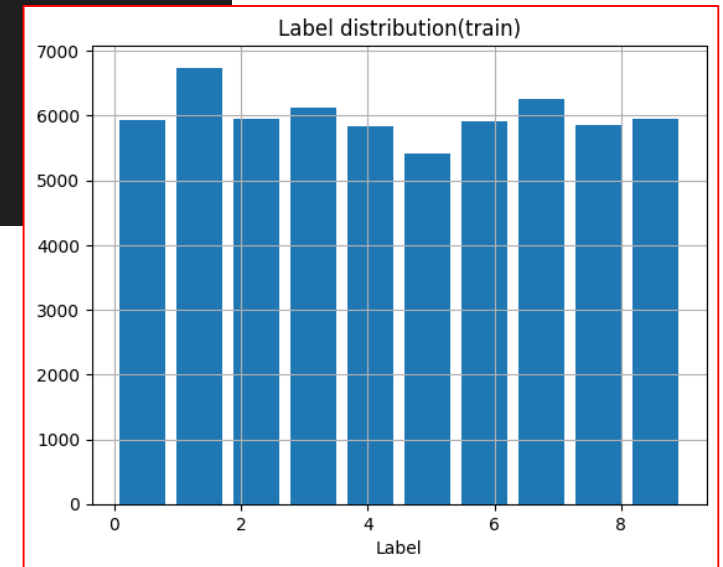
II. Deep Learning

```
1 plt.title('Label distribution(train)')
2 plt.grid()
3 plt.xlabel('Label')
4
5 plt.hist(L_train, bins=10, rwidth=0.8)
6
7 plt.show()
```

**라벨 데이터(타겟 데이터)별 빈도수를
히스토그램으로 시각화 출력한다**

MNIST는 완벽한 균등 분포가 아닌,
각 숫자별로 비슷한 수준의 표본 수를 가진
비균등한 분포

모델이 자주 등장하는 숫자에 편향됨
즉, “자주 나오는 클래스=중요한 클래스”라고 잘못 배울 위험
>> **샘플링 조정, 가중치 조정, 데이터 증강 등으로 해결 시도.**



II. Deep Learning

```
1 # 학습데이터 정답 분포 확인
2 label_distribution = np.zeros(10)
3
4 for i in range(len(L_train)):
5
6     label = int(L_train[i])
7
8     label_distribution[label] = label_distribution[label] + 1
9
10 print(label_distribution)
```

[5923. 6742. 5958. 6131. 5842. 5421. 5918. 6265. 5851. 5949.]

라벨 데이터(타겟 데이터)별 빈도수를 출력한다

II. Deep Learning

label_distribution = np.zeros(10): 크기가 10인 0으로 초기화된 배열 생성.

for idx in range(len(L_train)): L_train의 길이만큼 반복
훈련 데이터의 레이블을 하나씩 확인하기 위한 반복문.

label = int(L_train[idx]): L_train의 idx번째 요소를 정수로 변환하여 label 변수에 저장.
이는 현재 순회 중인 훈련 데이터의 레이블을 나타낸다.

label_distribution[label] = label_distribution[label] + 1: label 변수에 해당하는
인덱스 위치의 label_distribution 값을 1 증가. 현재 레이블의 개수를 1 증가.

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

II. Deep Learning

```
1 # Normalization of feature data
2
3 x_train = (x_train - 0.0) / (255.0 - 0.0)    # Min_Max
4
5 x_test = (x_test - 0.0) / (255.0 - 0.0)     # Min_Max
6
7
8 # One-Hot Encoding of label data
9
10 L_train = tf.keras.utils.to_categorical(L_train, num_classes=10) # 원핫인코딩 처리
11
12 L_test = tf.keras.utils.to_categorical(L_test, num_classes=10)   # 원핫인코딩 처리
```

입력데이터 : Min-max 로 정규화(normalization) 처리,

라벨데이터 : 원핫인코딩(to_categorical())

II. Deep Learning

`x_train = (x_train - 0.0) / (255.0 - 0.0):`

훈련 데이터인 `x_train`의 값을 0에서 255 사이로 정규화(normalization).

이를 위해 모든 값을 0(min value)으로 빼고, 255로 나누어서 각 값이 0과 1 사이의 범위로 맞춘다.

`x_test = (x_test - 0.0) / (255.0 - 0.0):`

테스트 데이터인 `x_test`에도 동일한 정규화 과정을 적용.

`t_train = tf.keras.utils.to_categorical(t_train, num_classes=10):`

훈련 데이터의 레이블인 `t_train`을 원-핫 인코딩(one-hot encoding) 형식으로 변환.

이를 위해 `to_categorical` 함수를 사용하고, 레이블의 클래스 개수인 `num_classes`를 10으로 지정.

이 과정을 통해 각 레이블은 10차원의 벡터로 변환되며,

해당하는 인덱스만 1이고 나머지는 0인 형태로 표현.

`t_test = tf.keras.utils.to_categorical(t_test, num_classes=10):`

테스트 데이터의 레이블에도 동일한 원-핫 인코딩 과정을 적용.

* 오류 주의 : 오타등으로 재실행시 원핫 인코딩이 두번 실행될 수 있다.

처음: 정수 레이블 (예: 3) → [0,0,0,1,0,0,0,0,0] (shape: (batch_size, 10))

다시 한 번 적용하면 → one-hot 인코딩된 것을 또 인코딩 → (batch_size, 10, 10)

II. Deep Learning

`t_train = tf.keras.utils.to_categorical(t_train, num_classes=10):`

훈련 데이터의 레이블인 `t_train`을 원-핫 인코딩(one-hot encoding) 형식으로 변환.
이를 위해 `to_categorical` 함수를 사용하고, 레이블의 클래스 개수인 `num_classes`를 10으로 지정.
이 과정을 통해 각 레이블은 10차원의 벡터로 변환되며,
해당하는 인덱스만 1이고 나머지는 0인 형태로 표현.

수학적으로 "차원"은 벡터의 요소 수를 의미
프로그래밍에서는 배열의 구조를 말하며, `shape`로 구분

관점	(10,)	(10,1)	(1,10)
수학적 차원	10차원 벡터	10개의 1차원 벡터	10차원 벡터
프로그래밍 차원 (<code>ndim</code>)	1차원	2차원	2차원
형태 (<code>shape</code>)	(10,)	(10,1)	(1,10)
사용 용도	신경망 출력 벡터	RNN 입력, 테이블 행	Dense 레이어 입력 등

II. Deep Learning

1 model.summary()

Model: "sequential"		
Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 100)	78,500
dense_1 (Dense)	(None, 10)	1,010

Total params: 79,510 (310.59 KB)
Trainable params: 79,510 (310.59 KB)
Non-trainable params: 0 (0.00 B)

```
1 # 1. Sequential 모델에 Input 레이어 추가
2 model = tf.keras.Sequential([
3     tf.keras.layers.Input(shape=(28, 28)), # 명시적인 Input 레이어
4     tf.keras.layers.Flatten(),
5     tf.keras.layers.Dense(100, activation='relu'),
6     tf.keras.layers.Dense(10, activation='softmax')
7 ])
```

입력(28*28=784) : 2차원의 이미지 데이터를 1차원으로 평탄화.

입력 이미지의 크기가 (28, 28)이므로 input_shape를 (28, 28)로 지정

입력 1개당 100개의 노드와 bias 100개가 임의의 초기값으로 추가

(fully connected, 파라미터 78,500 = 784(input)*100(node) + 100(bias))

입력 100에서 출력 10개와 bias 10개 추가 노드(1010 = 100(input) * 10(node) + 10(bias))

II. Deep Learning

model = tf.keras.Sequential():

Sequential 모델은 레이어를 선형으로 쌓아 구성하는 가장 간단한 형태의 모델.

tf.keras.Input(shape=(28, 28))

tf.keras.layers.Flatten()

Flatten 레이어 2차원의 이미지 데이터를 1차원으로 평탄화.

입력 이미지의 크기가 (28, 28)이므로 input_shape를 (28, 28)로 지정.

tf.keras.layers.Dense(100, activation='relu')

Dense 레이어를 추가. 이 레이어는 100개의 뉴런을 가지며, 활성화 함수로 ReLU를 사용.

이 레이어는 입력과 모든 뉴런 사이에 연결이 존재하는 완전 연결층(Fully Connected Layer).

tf.keras.layers.Dense(10, activation='softmax')

마지막으로 Dense 레이어를 추가. 이 레이어는 10개의 뉴런을 가지며, 활성화 함수로 softmax를 사용.

이 레이어는 최종 출력을 각 클래스(0부터 9까지의 숫자)에 대한 확률로 변환하는 역할.

II. Deep Learning

```
1 model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
2               loss='categorical_crossentropy',
3               metrics=['accuracy'])
4
5 model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 100)	78500
dense_1 (Dense)	(None, 10)	1010

```
=====
Total params: 79,510
Trainable params: 79,510
Non-trainable params: 0
```

II. Deep Learning

`model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3), loss='categorical_crossentropy', metrics=['accuracy']):`

모델을 컴파일. 이때 옵티마이저로는 Adam을 사용하며, 학습률은 $1e-3$ (0.001)로 설정. 손실 함수로는 `categorical_crossentropy`를 사용하며, 평가 지표로는 정확도(`accuracy`)를 사용.

`Model.summary():`

모델의 구조를 요약하여 출력.
각 레이어의 이름, 출력 크기, 파라미터 개수 등의 정보를 확인.
이를 통해 모델의 구조와 파라미터 개수를 쉽게 파악할 수 있다.

II. Deep Learning

```
1 ModelFit = model.fit(x_train, L_train, epochs=30, validation_split=0.3)
```

```
Epoch 2/30
1313/1313 [=====] - 6s 5ms/step - loss: 0.1457 - accuracy: 0.9568 - val_loss: 0.1408 - val_accuracy: 0.9581
Epoch 3/30
1313/1313 [=====] - 7s 5ms/step - loss: 0.1026 - accuracy: 0.9698 - val_loss: 0.1201 - val_accuracy: 0.9657
Epoch 4/30
1313/1313 [=====] - 6s 5ms/step - loss: 0.0769 - accuracy: 0.9771 - val_loss: 0.1116 - val_accuracy: 0.9664
Epoch 5/30
1313/1313 [=====] - 5s 4ms/step - loss: 0.0613 - accuracy: 0.9817 - val_loss: 0.1104 - val_accuracy: 0.9673
Epoch 6/30
1313/1313 [=====] - 7s 5ms/step - loss: 0.0498 - accuracy: 0.9850 - val_loss: 0.1032 - val_accuracy: 0.9709
Epoch 7/30
1313/1313 [=====] - 5s 4ms/step - loss: 0.0401 - accuracy: 0.9884 - val_loss: 0.1073 - val_accuracy: 0.9689
Epoch 8/30
1313/1313 [=====] - 7s 5ms/step - loss: 0.0320 - accuracy: 0.9900 - val_loss: 0.1050 - val_accuracy: 0.9706
Epoch 9/30
1313/1313 [=====] - 5s 4ms/step - loss: 0.0261 - accuracy: 0.9928 - val_loss: 0.1067 - val_accuracy: 0.9701
```

```
Epoch 27/30
1313/1313 [=====] - 5s 4ms/step - loss: 0.0034 - accuracy: 0.9990 - val_loss: 0.1564 - val_accuracy: 0.9726
Epoch 28/30
1313/1313 [=====] - 5s 4ms/step - loss: 0.0037 - accuracy: 0.9987 - val_loss: 0.1518 - val_accuracy: 0.9736
Epoch 29/30
1313/1313 [=====] - 5s 3ms/step - loss: 0.0027 - accuracy: 0.9994 - val_loss: 0.1664 - val_accuracy: 0.9715
Epoch 30/30
1313/1313 [=====] - 5s 4ms/step - loss: 0.0052 - accuracy: 0.9984 - val_loss: 0.1671 - val_accuracy: 0.9711
```

주어진 학습 데이터 x_train, L_train을 사용하여 7:3의 비율로 손실/정확도를 비교

II. Deep Learning

ModelFit = model.fit(x_train, L_train, epochs=30, validation_split=0.3):

주어진 학습 데이터 `x_train`과 정답 데이터 `t_train`을 사용하여 모델을 학습.
`epochs=30`은 전체 데이터셋을 30번 반복하여 학습한다.

`validation_split=0.3`은 학습 데이터 30%를 검증(validation) 데이터로 사용한다는 의미.
학습 결과는 저장.

주어진 데이터를 사용하여 모델을 학습하고,
각 에폭(epoch)마다 학습 손실(loss)과 정확도(accuracy),
검증 손실(validation loss)과 검증 정확도(validation accuracy) 출력.

학습 완료 후 객체에는 학습 이력(history) 정보가 저장되어 있어서
학습 과정을 시각화하거나 평가 등에 활용할 수 있다.

II. Deep Learning

```
1 test_loss, test_accuracy = model.evaluate(x_test, L_test) # loss, accuracy
2
3 print('test loss : ', test_loss)
4 print('test accuracy : ', test_accuracy)
```

```
313/313 [=====] - 1s 3ms/step - loss: 0.1256 - accuracy: 0.9788
test loss : 0.1255691599845886
test accuracy : 0.9787999987602234
```

model.evaluate(x_test, t_test)

주어진 테스트 데이터 **x_test**와 해당하는 정답 데이터 **L_test**를 사용하여 모델을 평가.

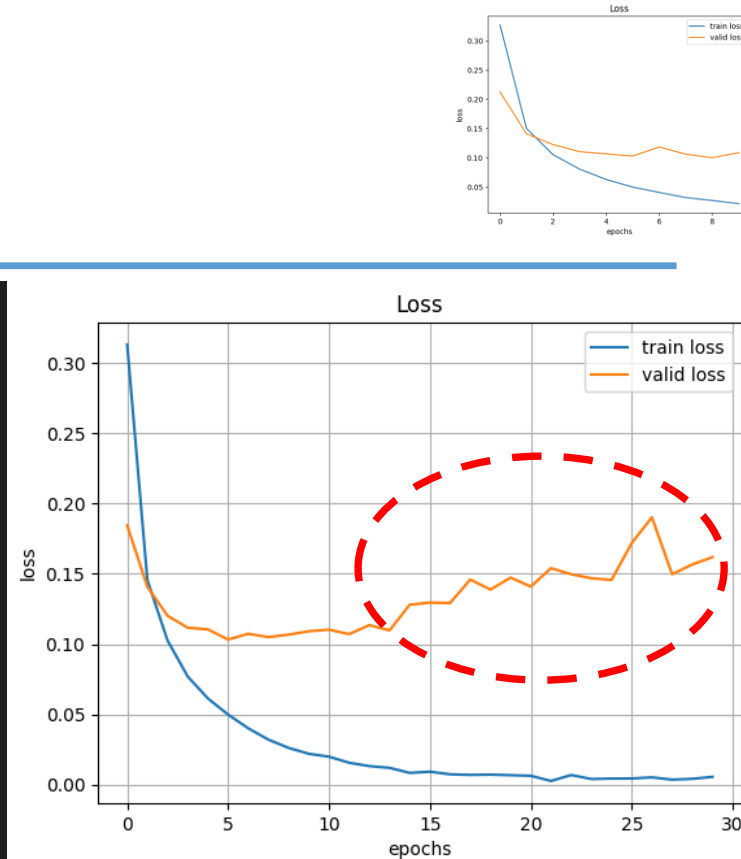
실행 결과로는 평가 지표들이 출력. **Loss 0.1428, accuracy : 0.9745**

accuracy는 정확도를 나타내며, 모델이 입력 데이터를 얼마나 정확하게 예측하는지를 나타낸다.
정확도는 예측 결과가 정답과 일치하는 샘플의 비율로 계산.

loss는 손실 값으로, 모델이 주어진 입력 데이터에 대해 얼마나 잘 예측하는지를 나타낸다.
주로 오차 함수(손실 함수)에 의해 계산

II. Deep Learning

```
1 plt.title('Loss')
2 plt.xlabel('epochs')
3 plt.ylabel('loss')
4 plt.grid()
5
6 plt.plot(ModelFit.history['loss'], label='train loss')
7 plt.plot(ModelFit.history['val_loss'], label='valid loss')
8
9 plt.legend(loc='best')
10
11 plt.show()
```



주어진 학습 데이터 x_{train} , L_{train} 을 사용하여 테스트와 검증을 7:3의 비율로 손실/정확도를 비교

검증 데이터의 손실이 줄지 않는다면 모델이 **훈련 데이터에 과적합 가능성**.

검증 데이터의 손실이 줄지 않는다는 것은 모델이 훈련 데이터에 대해서는 좋은 성능을 보이지만, 다른 데이터에 대해서는 성능이 좋지 않다는 것을 의미.

이는 모델이 훈련 데이터에 지나치게 적합되어 해당 데이터의 노이즈까지 학습하고 있을 수 있기 때문.

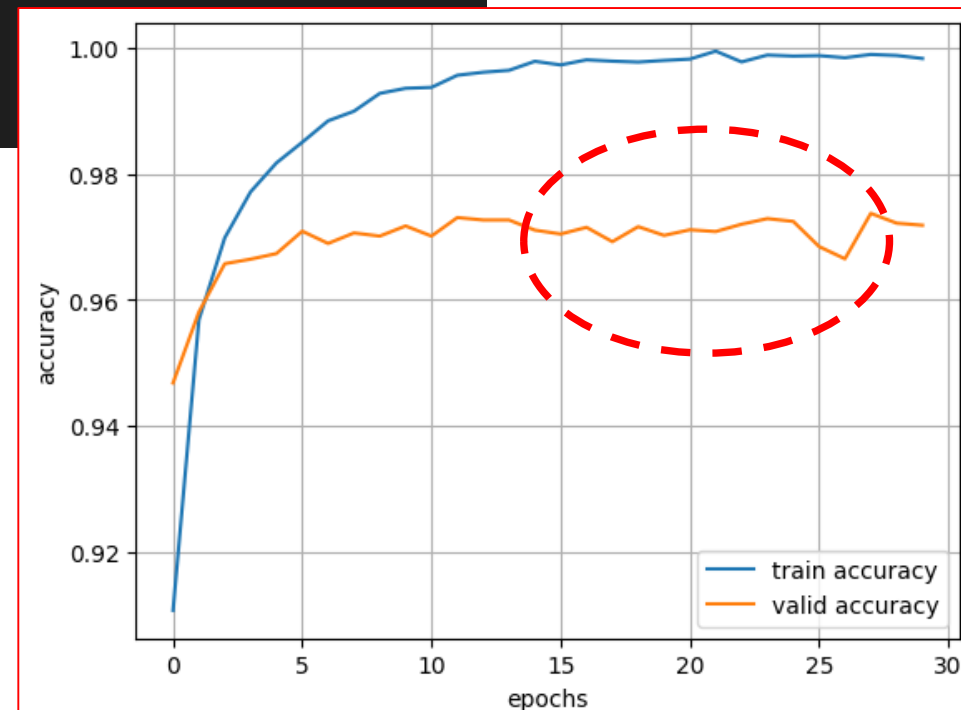
II. Deep Learning

```
1 plt.xlabel('epochs')
2 plt.ylabel('accuracy')
3 plt.grid()
4
5 plt.plot(ModelFit.history['accuracy'], label='train accuracy')
6 plt.plot(ModelFit.history['val_accuracy'], label='valid accuracy')
7
8 plt.legend(loc='best')
9
10 plt.show()
```

주어진 학습 데이터 x_{train} , L_{train} 을 사용하여 테스트와
검증을 7:3의 비율로 손실/정확도를 비교

검증 데이터의 정확도가 개선되지 않는다면
모델이 **훈련 데이터에 과적합 가능성**.

에폭 증가에도 0.97 정도의 정확도에서 개선되지 않고 있다



II. Deep Learning

상황	대응 방법
과적합이 시작됨	<code>early stopping</code> , <code>dropout</code> , <code>regularization</code> 사용 고려
학습 횟수 줄이기	현재로선 6~7 epoch 정도에서 early stop 하는 것이 좋을 수 있음
데이터 부족	data augmentation 또는 학습 데이터 확장 고려

II. Deep Learning

```
1 from sklearn.metrics import confusion_matrix
2 import seaborn as sns
3
4 plt.figure(figsize=(6, 6))
5
6 predicted_value = model.predict(x_test) # 784개의 입력 x_test가 10개의 출력값으로 나온다
7
8 Cmatrix = confusion_matrix(np.argmax(L_test, axis=-1), # 원핫인코딩 처리된 L_test에서 제일 큰값을 갖는
9                             np.argmax(predicted_value, axis=-1)) # 원핫인코딩 처리된 값 중 제일 큰값을 갖는 인
10
11 sns.heatmap(Cmatrix, annot=True, fmt='d')
12 plt.show()
```

히트맵을 통해 예측치와 실제치의 차이를 보며 **모델의 정확도가 떨어진 라벨값**을 파악

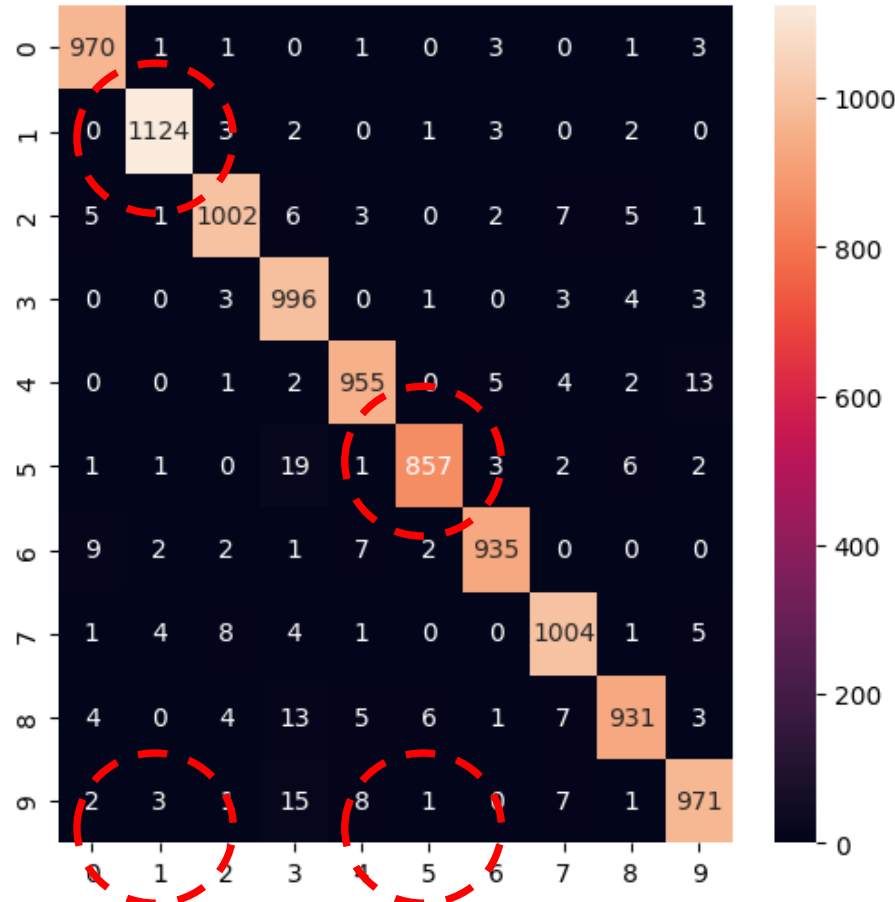
히트맵에서 y축은 실제 정답 클래스를, x축은 모델이 예측한 클래스

II. Deep Learning

현재 5에 대한 정답수가 857로 가장 낮고 1에 대한 정답수가 가장 높게 나타난다.

히트맵을 해석할 때는 **가로 방향으로 예측된 클래스를 확인하고**, **세로 방향으로 실제 클래스를** 확인하여 모델의 예측 정확도와 클래스 간의 일치 여부를 평가할 수 있다.

밝은 색상은 예측이 정확함을, 어두운 색상은 예측이 잘못됐거나 클래스 간의 불일치



II. Deep Learning

```
Cmatrix = confusion_matrix(np.argmax(t_test, axis=-1),  
                           np.argmax(predicted_value, axis=-1))
```

실제 레이블(`t_test`)과 예측된 값(`predicted_value`)을 사용하여 혼동 행렬(confusion matrix)을 생성

- **`np.argmax(t_test, axis=-1)`**: 실제 레이블(`t_test`)의 원-핫 인코딩된 형태에서 가장 큰 값을 갖는 인덱스를 추출. **실제 클래스를 나타내는 정수값**으로 변환하는 과정.
- **`np.argmax(predicted_value, axis=-1)`**: 예측된 값(`predicted_value`)의 원-핫 인코딩된 형태에서 가장 큰 값을 갖는 인덱스를 추출. **예측된 클래스를 나타내는 정수값**으로 변환하는 과정.

• 혼동 행렬은 각 클래스 별로 실제 클래스와 예측된 클래스의 조합에 따라 카운트된 값으로 구성. 예를 들어, `cm[i][j]`는 실제로는 클래스 `i`에 속하지만 모델이 클래스 `j`로 예측한 데이터 포인트의 개수를 나타낸다.

```
sns.heatmap(cm, annot=True, fmt='d')
```

`cm`을 히트맵으로 표시.

`annot=True`는 각 셀에 숫자 값을 표시하도록 지정,

`fmt='d'`는 숫자 형식을 정수로 지정.

히트맵은 셀의 색상을 사용하여 데이터의 상대적인 값을 시각화하는데 사용.

II. Deep Learning

axis=-1은 NumPy 배열에서 마지막 축(axis)을 나타낸다.

NumPy의 다차원 배열은 여러 개의 축을 가질 수 있으며, 각 축은 해당 축을 따라 배열의 차원이 결정. 예를 들어, 2차원 배열은 행과 열의 두 개의 축을 가지고 있다. 이는 배열의 차원에 상관없이 항상 마지막 축을 선택하는 것을 의미.

예를 들어, (3, 4, 5) 모양의 3차원 배열에서 **axis=-1**은 마지막 축인 크기 5의 축을 선택.

axis=-1 : 다차원 배열에서 마지막 축을 기준으로 연산을 수행하거나 인덱싱을 할 때 편리하기 때문.

특히, 신경망에서 일반적으로 **마지막 축은 클래스 레이블**이나 예측 확률과 같은 정보를 담고 있기 때문에, **axis=-1**을 사용하여 해당 정보에 접근하거나 연산을 수행할 수 있다.

II. Deep Learning

heatmap 함수

- data: 히트맵에 표시할 데이터 배열.
- annot: 히트맵 셀에 숫자를 표시할지 여부를 지정하는 불리언 값.
- fmt: 히트맵 셀에 표시될 숫자의 형식을 지정하는 문자열 형식.
- cmap: 색상 맵(Color map)으로, 히트맵의 색상을 결정.
- linewidths: 셀 간의 경계선 두께를 지정하는 값.
- linecolor: 셀 간의 경계선 색상을 지정하는 값.
- xticklabels: x축에 표시할 레이블 리스트.
- yticklabels: y축에 표시할 레이블 리스트.

이 매개변수들은 히트맵의 외관을 조정하는 데 사용.

data 매개변수에는 2차원 배열이 주로 사용되며, 해당 배열의 값에 따라 히트맵의 각 셀의 색상이 결정.

II. Deep Learning

```
1 predicted_value = np.round(predicted_value)
2 predicted_value = predicted_value.astype(int)
3 print(predicted_value)
```

```
[[0 0 0 ... 1 0 0]
 [0 0 1 ... 0 0 0]
 [0 1 0 ... 0 0 0]
 ...
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]]
```

Model.predict(x_test)로 예측된 값(predicted_value)는 소프트맥스의 확률 형태로 출력된다.

정수로 출력해 보면 원핫인코딩의 형태로 볼 수 있다.

Argmax를 통해 최대값(1)을 갖는 자리의 인덱스가 출력될 수 있다.

II. Deep Learning

```
1 print(Cmatrix)
2 print('#n')
3
4 for i in range(10):
5     print(('label = %d#t(%d/%d)#taccuracy = %.3f' %
6           (i, np.max(Cmatrix[i]), np.sum(Cmatrix[i]),
7             np.max(Cmatrix[i])/np.sum(Cmatrix[i]))))
```

```
[[ 970   1   1   0   1   0   3   0   1   3]
 [   0 1124   3   2   0   1   3   0   2   0]
 [   5   1 1002   6   3   0   2   7   5   1]
 [   0   0   3  996   0   1   0   3   4   3]
 [   0   0   1   2  955   0   5   4   2  13]
 [   1   1   0  19   1  857   3   2   6   2]
 [   9   2   2   1   7   2  935   0   0   0]
 [   1   4   8   4   1   0   0 1004   1   5]
 [   4   0   4  13   5   6   1   7  931   3]
 [   2   3   1  15   8   1   0   7   1  971]]
```

```
label = 0      (970/980)      accuracy = 0.990
label = 1      (1124/1135)    accuracy = 0.990
label = 2      (1002/1032)    accuracy = 0.971
label = 3      (996/1010)     accuracy = 0.986
label = 4      (955/982)      accuracy = 0.973
label = 5      (857/892)      accuracy = 0.961
label = 6      (935/958)      accuracy = 0.976
label = 7      (1004/1028)    accuracy = 0.977
label = 8      (931/974)      accuracy = 0.956
label = 9      (971/1009)     accuracy = 0.962
```

II. Deep Learning (Predict your letter with MNIST)

**Predict your letter with MNIST,
trained with ANN**



Sam.png



SA.png



ANNwithMNIST_
trained0716.h5

이미지는 MNIST 데이터셋과 같은 형태,
흑백 이미지이고 배경이 검은색이며 숫자가 흰색으로.
이미지의 크기가 너무 크거나 작으면 모델의 성능에 영향을 줄 수 있다.

II. Deep Learning (Predict your letter with MNIST)

Predict your letter with MNIST

```
1 # 모델 저장
2 model.save('my_model.keras', include_optimizer=False)

1 from PIL import Image
2 import numpy as n
3
4 # 모델 불러오기
5 model=tf.keras.models.load_model('/content/my_model.keras')
```



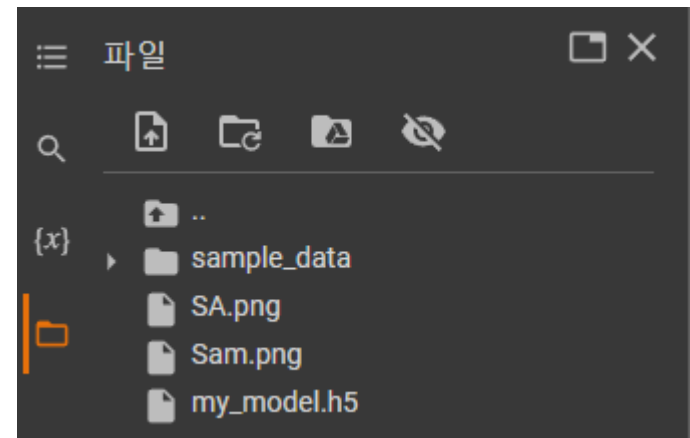
Sam.png



SA.png



ANNwithMNIST_
trained0716.h5



II. Deep Learning (Predict your letter with MNIST)

```
7 # 실제 이미지 파일 불러오기
8 img = Image.open('/content/Sam.png').convert('L') # 'image.png'는 실제 이미지 파일 경로
9 img = img.resize((28, 28)) # MNIST 데이터와 동일한 크기인 28x28로 이미지 크기 조정
10
11 # 이미지 데이터를 NumPy 배열로 변환
12 input_data = np.array(img)
13
14 # 데이터 전처리 (색상 반전 및 정규화)
15 input_data = (255 - input_data) / 255.0
16
17 # 배치 차원 추가
18 input_data = np.expand_dims(input_data, axis=0)
```

PIL(Python Imaging Library)에서

이미지를 처리할 때 'L'은 "luminance"(휘도), 이미지를 흑백(그레이스케일)으로 변환

II. Deep Learning (Predict your letter with MNIST)

PIL(Python Imaging Library)에서 이미지를 처리할 때 'L'은 "luminance"(휘도), 이미지를 흑백(그레이스케일)으로 변환

```
input_data = (255 - input_data)/255.0
```

비주얼 표현 변경: 원래의 흑백 이미지에서, 흰색은 높은 픽셀 값(255에 가까움)을 가지며, 검은색은 낮은 픽셀 값(0에 가까움)을 갖는다. (255 - input_data) 연산을 사용하면 순서가 뒤바뀌어, 흰색이 낮은 값(0에 가까움)을, 검은색이 높은 값(255에 가까움)을 가지게 됨.

```
input_data = np.expand_dims(input_data, axis=0):
```

배열에 추가적인 차원을 추가

원본 input_data 배열의 shape이 (28, 28)이었다면, 이 코드를 실행한 후에는 shape이 (1, 28, 28)로 변환
대부분의 딥 러닝 모델이 여러 이미지를 한 번에 처리하도록 설계되어 있어서, 단일 이미지를 입력하더라도 그 이미지가 3차원 배열로 제공되어야 한다. 이 경우 첫 번째 차원은 '배치' 차원으로, 처리할 이미지의 수를 나타냄.

II. Deep Learning (Predict your letter with MNIST)

```
1 # 모델을 사용하여 예측 수행
2 prediction = model.predict(input_data)
3
4 # 예측 결과 출력
5 print("Predicted digit:", np.argmax(prediction))
```

```
1/1 [=====] - 0s 53ms/step
Predicted digit: 3
```



Sam.png



SA.png



ANNwithMNIST_
trained0716.h5

II. Deep Learning (Predict your letter with MNIST : CNN)

CNN with MNIST, (full code)

이미지는 MNIST 데이터셋과 같은 형태,
흑백 이미지이고 배경이 검은색이며 숫자가 흰색으로.
이미지의 크기가 너무 크거나 작으면 모델의 성능에 영향을 줄 수 있다.

II. Deep Learning (Predict your letter with MNIST : CNN)

정규화

```
x_train = x_train / 255.0
```

```
x_test = x_test / 255.0
```

CNN 입력용 reshape

```
x_train = x_train.reshape(-1, 28, 28, 1) # CNN으로 변경
```

```
x_test = x_test.reshape(-1, 28, 28, 1) # CNN으로 변경
```

원-핫 인코딩

```
L_train = tf.keras.utils.to_categorical(L_train, num_classes=10)
```

```
L_test = tf.keras.utils.to_categorical(L_test, num_classes=10)
```

모델 구성 (CNN으로 변경)

```
model = tf.keras.models.Sequential()
```

```
model.add(tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28,28,1))) # CNN으로 변경
```

```
model.add(tf.keras.layers.MaxPooling2D((2, 2))) # CNN으로 변경
```

```
model.add(tf.keras.layers.Conv2D(64, (3, 3), activation='relu')) # CNN으로 변경
```

```
model.add(tf.keras.layers.MaxPooling2D((2, 2))) # CNN으로 변경
```

```
model.add(tf.keras.layers.Flatten()) # CNN으로 변경
```

```
model.add(tf.keras.layers.Dense(64, activation='relu')) # CNN으로 변경
```

```
model.add(tf.keras.layers.Dense(10, activation='softmax')) # CNN으로 변경
```

컴파일

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
```

```
loss='categorical_crossentropy',
```

```
metrics=['accuracy'])
```

II. Deep Learning (Predict your letter with MNIST : CNN)

```
1 import tensorflow as tf
2 from tensorflow.keras.datasets import mnist
3
4 # 1. 데이터 로드
5 (x_train, L_train), (x_test, L_test) = mnist.load_data()
6
7 # 2. 정규화
8 x_train = x_train / 255.0
9 x_test = x_test / 255.0
10
11 # 3. CNN 입력용 reshape (batch, height, width, channel)
12 x_train = x_train.reshape(-1, 28, 28, 1)
13 x_test = x_test.reshape(-1, 28, 28, 1)
14
15 # 4. 원-핫 인코딩
16 L_train = tf.keras.utils.to_categorical(L_train, num_classes=10)
17 L_test = tf.keras.utils.to_categorical(L_test, num_classes=10)
18
```

-1 : 해당 축의 크기를 NumPy가 자동 계산
주로 배치 크기(batch size)를 자동으로 맞출 때 사용

II. Deep Learning (Predict your letter with MNIST : CNN)

```
1 # 5. 모델 구성 (CNN)
2 model = tf.keras.models.Sequential()
3
4 model.add(tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
5 model.add(tf.keras.layers.MaxPooling2D((2, 2)))
6
7 model.add(tf.keras.layers.Conv2D(64, (3, 3), activation='relu'))
8 model.add(tf.keras.layers.MaxPooling2D((2, 2)))
9
10 model.add(tf.keras.layers.Flatten())
11 model.add(tf.keras.layers.Dense(64, activation='relu'))
12 model.add(tf.keras.layers.Dense(10, activation='softmax'))
13
```

II. Deep Learning (Predict your letter with MNIST : CNN)

```
1 # 5. 모델 구성 (CNN)
2 model = tf.keras.models.Sequential()
3
4 model.add(tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
5 model.add(tf.keras.layers.MaxPooling2D((2, 2)))
6
7 model.add(tf.keras.layers.Conv2D(64, (3, 3), activation='relu'))
8 model.add(tf.keras.layers.MaxPooling2D((2, 2)))
9
10 model.add(tf.keras.layers.Flatten())
11 model.add(tf.keras.layers.Dense(64, activation='relu'))
12 model.add(tf.keras.layers.Dense(10, activation='softmax'))
13
```

II. Deep Learning (Predict your letter with MNIST : CNN)

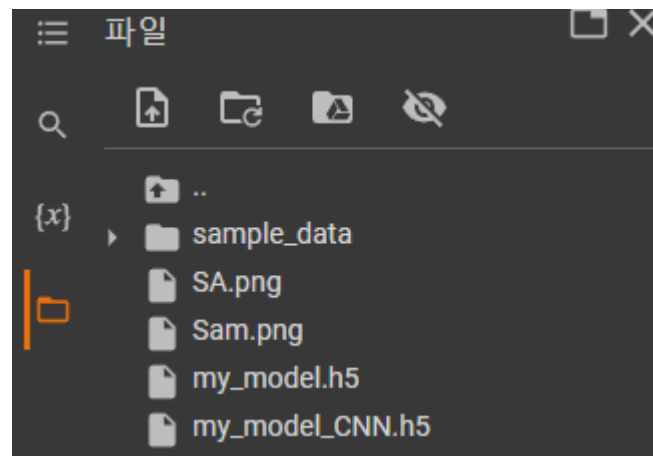
```
1 # 6. 컴파일
2 model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
3               loss='categorical_crossentropy',
4               metrics=['accuracy'])
5
6 # 7. 학습
7 model.fit(x_train, L_train, epochs=30, batch_size=32, validation_split=0.3)
8
9 # 8. 평가
10 test_loss, test_accuracy = model.evaluate(x_test, L_test)
11 print("Test Loss:", test_loss)
12 print("Test Accuracy:", test_accuracy)
```

II. Deep Learning (Predict your letter with MNIST : CNN)

Predict your letter with MNIST trained by CNN

```
[26] 1 # 모델 저장  
2 model.save('my_model_CNN.h5') # 'my_model_CNN.h5'는 저장하려는 파일명을 입력
```

```
[27] 1 from PIL import Image  
2 import numpy as np  
3  
4 # 모델 불러오기  
5 model = tf.keras.models.load_model('/content/my_model_CNN.h5') # 불러오려는 모델 파일명을 입력.
```



II. Deep Learning (Predict your letter with MNIST : CNN)

```
7 # 실제 이미지 파일 불러오기
8 img = Image.open('/content/Sam.png').convert('L') # 'image.png'는 실제 이미지 파일 경로
9 img = img.resize((28, 28)) # MNIST 데이터와 동일한 크기인 28x28로 이미지 크기 조정
10
11 # 이미지 데이터를 NumPy 배열로 변환
12 input_data = np.array(img)
13
14 # 데이터 전처리 (색상 반전 및 정규화)
15 input_data = (255 - input_data) / 255.0
16
17 # 배치 차원 추가
18 input_data = np.expand_dims(input_data, axis=0)
```



CNNwithMNIST_
trained0716.h5



Sam.png



SA.png

II. Deep Learning (Predict your letter with MNIST : CNN)

```
2 # 모델을 사용하여 예측 수행
3 prediction = model.predict(input_data)
4
5 # 예측 결과 출력
6 print("Predicted digit:", np.argmax(prediction))
7
```

```
1/1 [=====] - 0s 114ms/step
```

```
Predicted digit: 3
```



CNNwithMNIST_
trained0716.h5



Sam.png



SA.png

II. Deep Learning (Predict your letter with MNIST : CNN)

ANN

label = 0	(965/980)	accuracy=0.985
label = 1	(1123/1135)	accuracy=0.989
label = 2	(1008/1032)	accuracy=0.977
label = 3	(985/1010)	accuracy=0.975
label = 4	(956/982)	accuracy=0.974
label = 5	(862/892)	accuracy=0.966
label = 6	(942/958)	accuracy=0.983
label = 7	(986/1028)	accuracy=0.959
label = 8	(942/974)	accuracy=0.967
label = 9	(964/1009)	accuracy=0.955

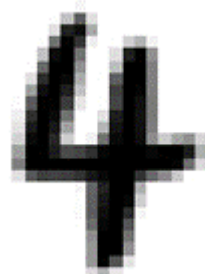
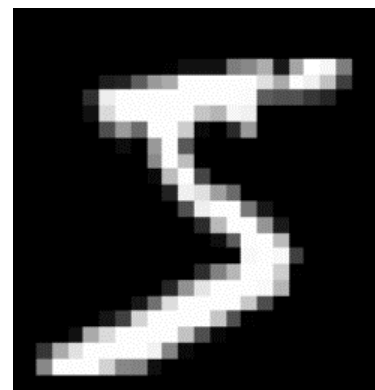
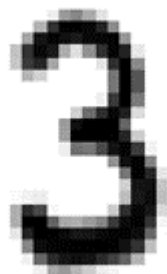
test loss : 0.12021242827177048
test accuracy : 0.9787999987602234

CNN

label = 0	(970/980)	accuracy=0.990
label = 1	(1130/1135)	accuracy=0.996
label = 2	(1025/1032)	accuracy=0.993
label = 3	(1002/1010)	accuracy=0.992
label = 4	(977/982)	accuracy=0.995
label = 5	(882/892)	accuracy=0.989
label = 6	(946/958)	accuracy=0.987
label = 7	(1018/1028)	accuracy=0.990
label = 8	(959/974)	accuracy=0.985
label = 9	(987/1009)	accuracy=0.978

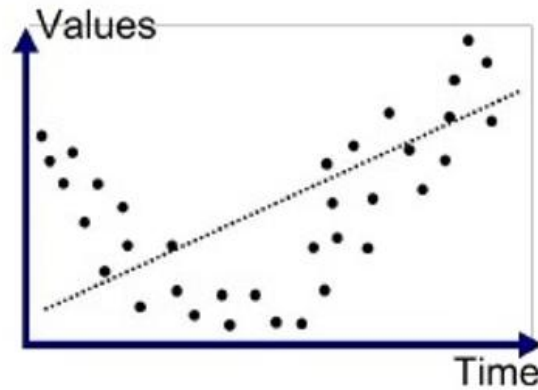
test loss : 0.06392188370227814
test accuracy : 0.9897000193595886

II. Deep Learning (Predict your letter with MNIST)

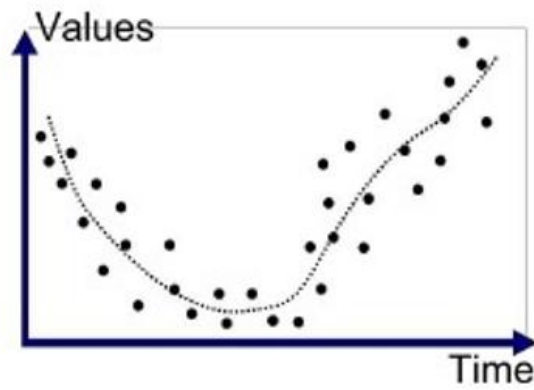


5	0	4	1	9
2	1	3	1	4
3	5	3	6	1
7	2	8	6	9
4	0	9	1	1

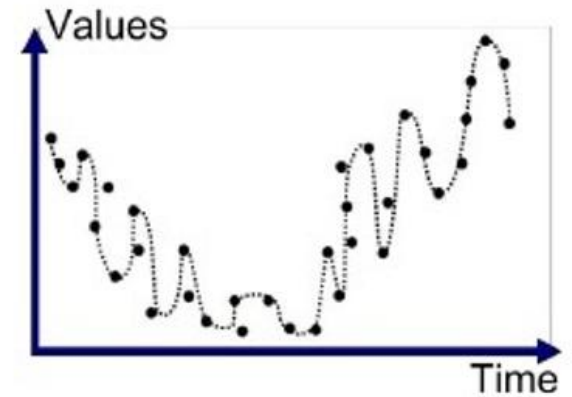
II. Deep Learning **(Predict your letter with MNIST)**



Underfitted



Good Fit/Robust



Overfitted

II. Deep Learning (Predict your letter with MNIST)

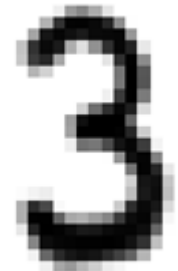
Grayscale Image



Resized Image



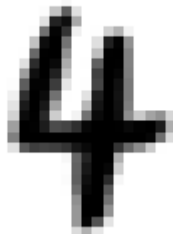
Scaled Image



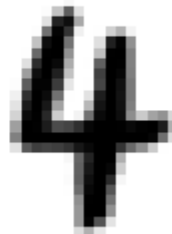
Grayscale Image



Resized Image



Scaled Image



Grayscale Image



Resized Image



Scaled Image



II. Deep Learning (Predict your letter with MNIST)

Batch Normalization 추가: 각 층 사이에 배치 정규화를 추가하여 학습 안정성을 높이고, 더 깊은 네트워크 구조가 가능하도록 합니다.

Dropout 추가: 과대적합을 방지하기 위해 드롭아웃을 추가합니다.

Dense Layer 개수 및 노드 수 조정: Dense 층의 개수 및 크기를 조정하여 모델의 복잡성을 줄이고, 더 적절한 피쳐 표현을 학습하도록 합니다.

데이터 증강(Data Augmentation): 이미지를 회전, 이동, 확대 등의 방식으로 데이터셋을 증강하여 모델이 다양한 패턴을 학습하도록 유도합니다.

Early Stopping: 과대적합을 방지하기 위해 early stopping을 사용하여 더 이상 성능이 개선되지 않으면 학습을 중단하도록 합니다.

II. Deep Learning (Predict your letter with MNIST)

아키텍처 조정: 다양한 모델 구조 실험: VGG, ResNet과 같은 보다 깊은 네트워크 아키텍처를 사용해 볼 수 있습니다.

전이 학습(Transfer Learning): 사전 훈련된 모델을 사용하고, 마지막 레이어만 조정하는 방법을 고려할 수 있습니다.

하이퍼파라미터 튜닝: 학습률 스케줄링: 학습 중에 학습률을 조정하여 더 나은 수렴을 유도할 수 있습니다.

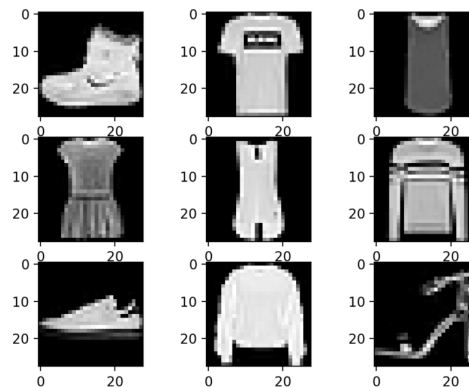
다양한 손실 함수 시도: MNIST는 분류 문제이므로, 다른 손실 함수(예: Focal Loss)를 시도해보는 것도 좋은 방법입니다. 이는 클래스 불균형 문제를 완화하는 데 도움이 될 수 있습니다.

정규화 기법: L2 정규화와 같은 정규화 기법을 도입하여 모델의 일반화 능력을 향상시킬 수 있습니다.

Ensemble 방법: 여러 모델을 훈련시켜 이들의 예측을 결합하여 최종 예측을 개선할 수 있습니다.

II. Deep Learning (CNN with Fashion MNIST)

CNN with Fashion MNIST

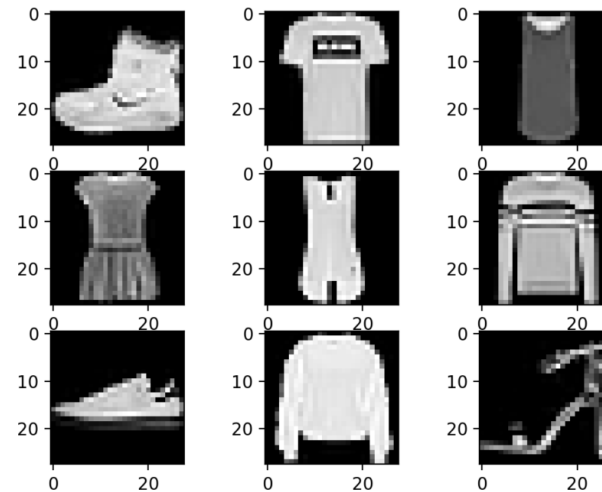


II. Deep Learning

Fashion MNIST

MNIST와 유사하게 **옷과 신발, 가방의 이미지들을** 모아놓은 데이터셋.
이미지 크기는 **28x28픽셀**이며
60,000개의 학습셋과 10,000개의 테스트. 흑백 이미지

Label	Description
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot



1. Deep Learning **CNN** with CIFAR10

CNN with CIFAR10

비행기

자동차

새

고양이

사슴

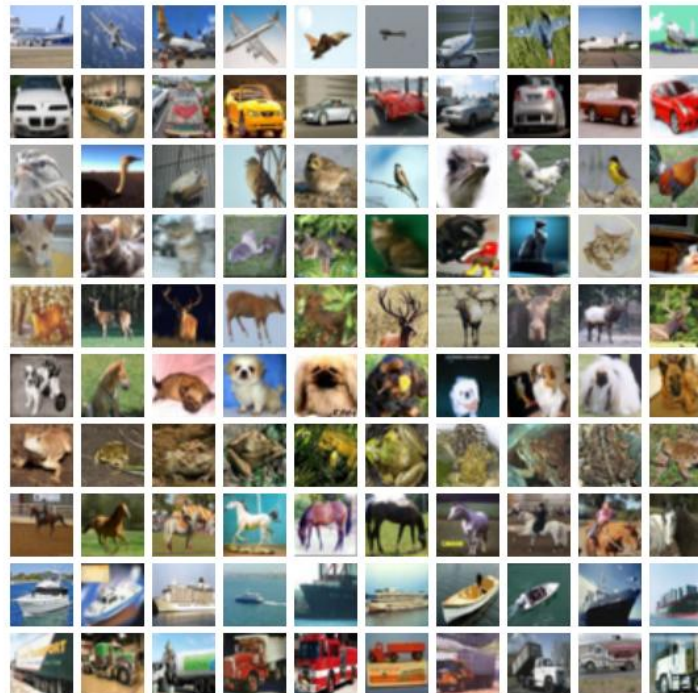
개

개구리

말

배

트럭



II. Deep Learning

CIFAR-10

10개의 다른 클래스로 구성된 60,000개의 32x32 컬러 이미지로 구성.

Airplane(비행기) Automobile(자동차) Bird(새) Cat(고양이)
Deer(사슴) Dog(개) Frog(개구리) Horse(말) Ship(배) Truck(트럭)

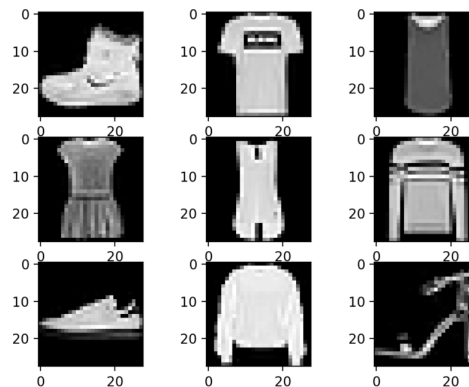
훈련 세트에는 클래스마다 5,000개의 이미지가 있고(50,000),
테스트 세트에는 클래스마다 1,000개의 이미지(10,000)

이미지는 다양한 크기, 각도, 조명 조건 및 배경으로 구성, 실제 세계에서의 다양한 상황을 모델에 적용/일반화를 향상.



II. Deep Learning (CNN with Fashion MNIST)

CNN with Fashion MNIST

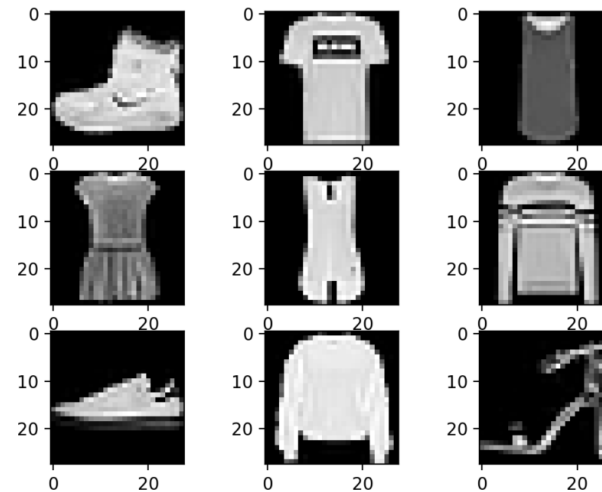


II. Deep Learning

Fashion MNIST

MNIST와 유사하게 **옷과 신발, 가방의 이미지들을** 모아놓은 데이터셋.
이미지 크기는 **28x28픽셀**이며
60,000개의 학습셋과 10,000개의 테스트. 흑백 이미지

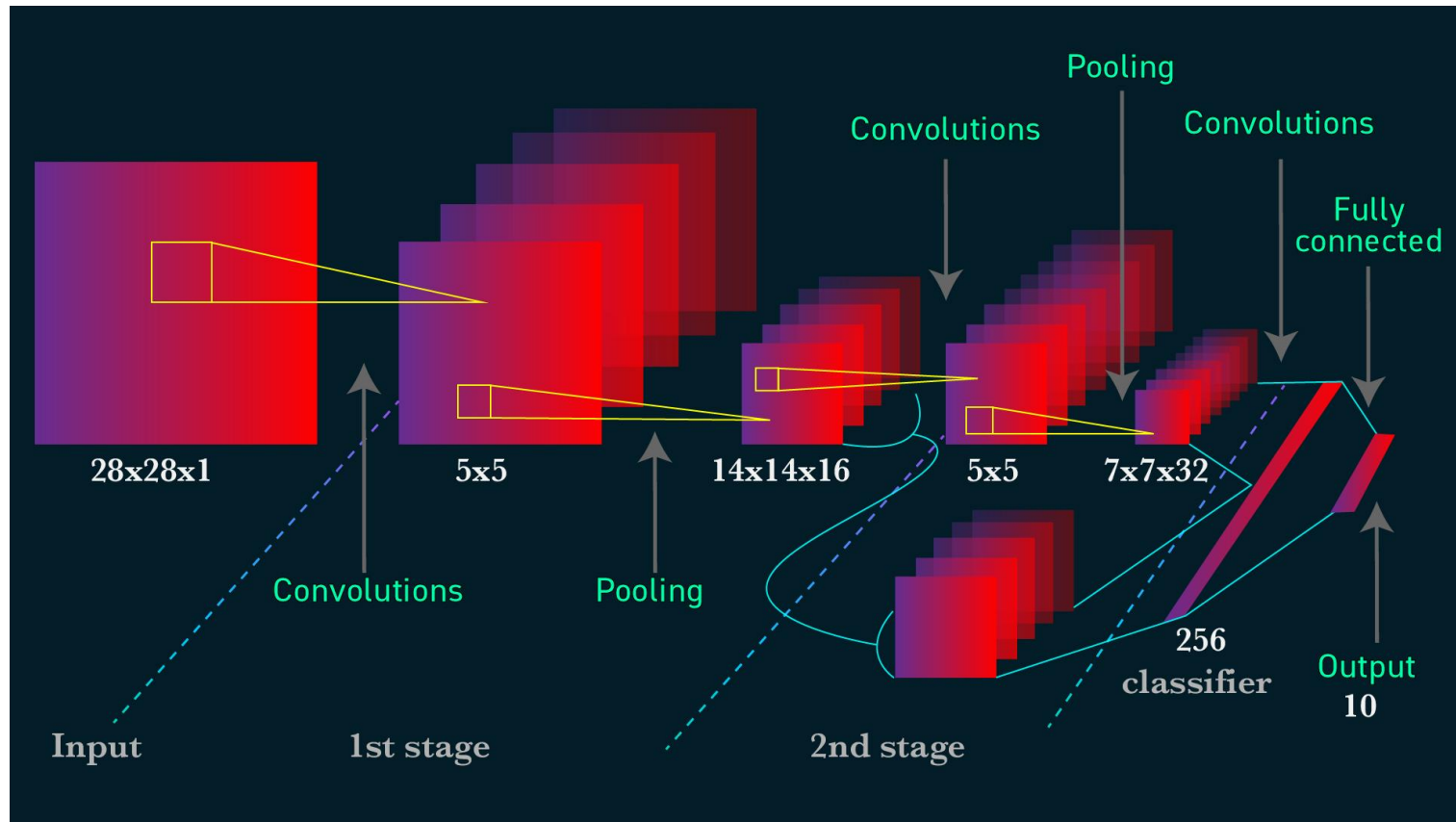
Label	Description
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot



II. Deep Learning (CNN with Fashion MNIST)

CNN : 합성곱에 의한 이미지의 특징값을 추출하는 과정이 추가된다.

Input > Convolution > Relu > Max Pooling > Output



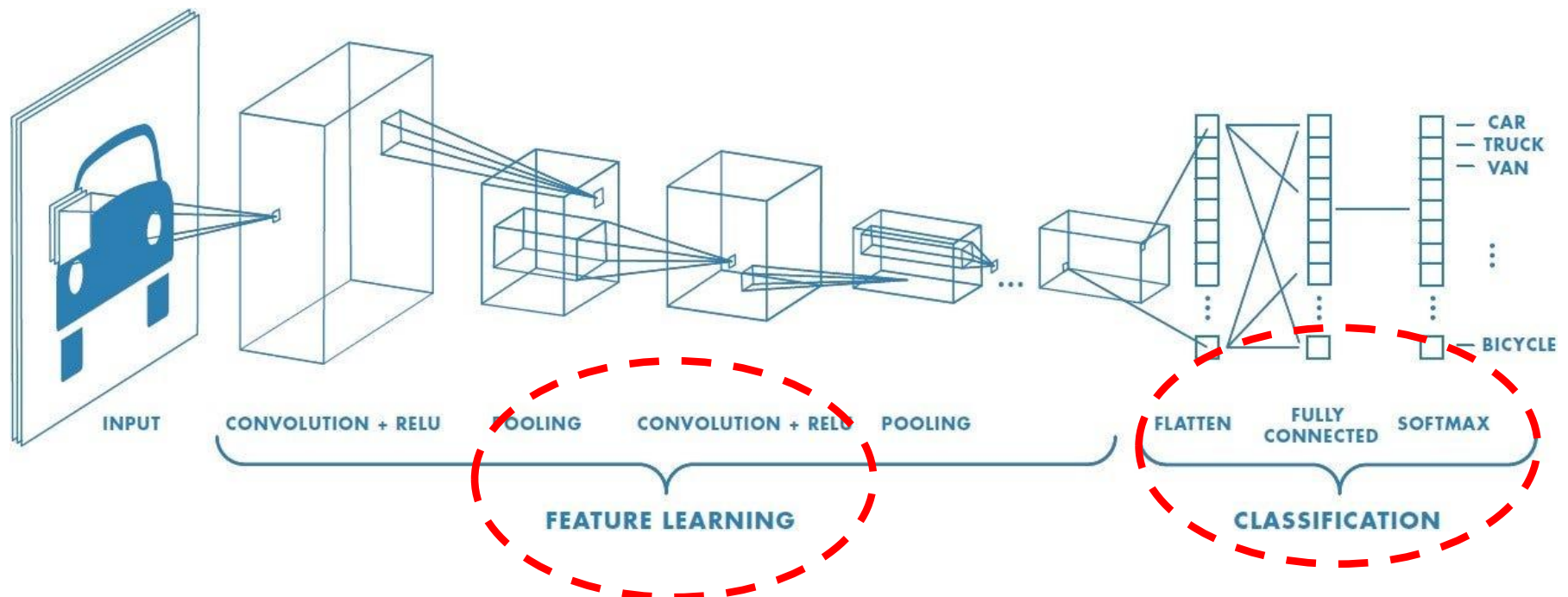
II. Deep Learning (CNN with Fashion MNIST)

CNN : 특징 추출 모듈 + 정답 처리용 분류기

Input > Convolution > Relu > Max Pooling (**Feature Extractor**)

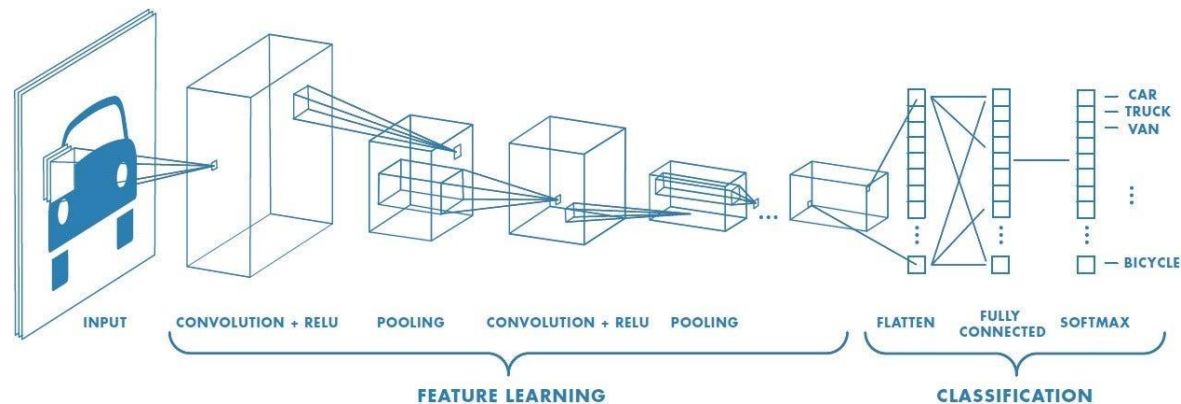
> Flatten > fully connected > Softmax (**Classification**)

> Predicted



II. Deep Learning

```
1 import numpy as np
2 import tensorflow as tf
3 from tensorflow.keras.datasets import fashion_mnist
4 from tensorflow.keras.models import Sequential
5 from tensorflow.keras.layers import Conv2D, MaxPool2D, Flatten, Dense, Dropout
```



II. Deep Learning

```
1 (x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
2
3 x_train = x_train.reshape(-1, 28, 28, 1)
4 x_test = x_test.reshape(-1, 28, 28, 1)
5
6 print(x_train.shape, x_test.shape)
7 print(y_train.shape, y_test.shape)
8
9 x_train = x_train.astype(np.float32) / 255.0
10 x_test = x_test.astype(np.float32) / 255.0
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 [=====] - 0s 1us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 [=====] - 3s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 [=====] - 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 [=====] - 1s 0us/step
(60000, 28, 28, 1) (10000, 28, 28, 1)
(60000,) (10000,)
```


II. Deep Learning

Fashion MNIST 데이터셋을 로드하고 전처리.

```
x_train=x_train.reshape(-1, 28, 28, 1)
x_test=x_test.reshape(-1, 28, 28, 1)
```

입력 데이터(x_train, x_test)의 형태를 (샘플 수, 28, 28, 1)로 reshape.

-1로 설정하면, 해당 차원의 크기는 자동으로 계산,

x_train 배열의 원래 크기를 유지하면서 3차원의 배열을 **4차원 배열로 생성**
(이미지 개수, 28, 28, 1)

```
x_train = x_train.astype(np.float32) / 255.0
x_test = x_test.astype(np.float32) / 255.0
```

np.float32로 형변환한 뒤 255로 나누어 픽셀 값의 범위를 **0과 1 사이로 정규화**,
모델이 입력 데이터를 더 잘 학습할 수 있도록 .

II. Deep Learning

```
1 import matplotlib.pyplot as plt
2
3 # 25개의 이미지 출력
4 plt.figure(figsize=(6, 6))
5
6 for index in range(25):    # 25 개 이미지 출력
7
8     plt.subplot(5, 5, index + 1) # 5행 5열
9     plt.imshow(x_train[index], cmap='gray')
10    plt.axis('off')
11    # plt.title(str(t_train[index]))
12
13 plt.show()
```



II. Deep Learning

이미지 확인용으로 총 25개의 이미지를 출력

plt.figure(figsize=(6, 6)) 출력할 그림의 크기를 설정
figsize는 가로와 세로 크기를 인치 단위로 지정

for index in range(25):

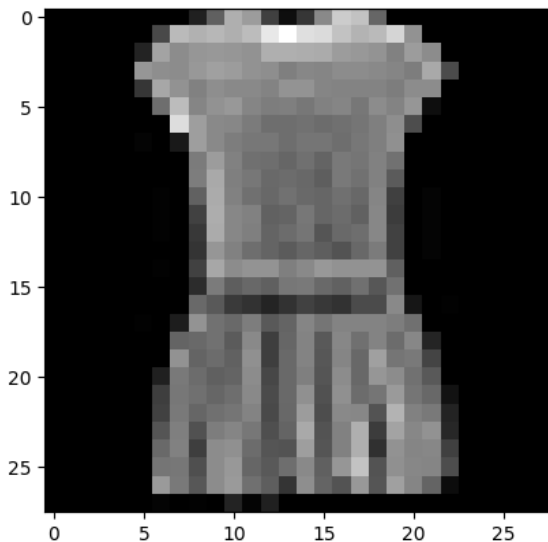
plt.subplot(5, 5, index + 1) 출력할 그림의 위치를 지정.
5행 5열의 그리드에서 현재 이미지의 위치를 지정.
index는 0부터 시작하므로 1을 더해준다.

plt.imshow(x_train[index], cmap='gray')
현재 인덱스의 이미지를 그레이스케일로 출력.
x_train[index]는 해당 인덱스의 이미지 데이터를 나타낸다.

plt.axis('off') 축을 표시하지 않도록 설정하는 부분

II. Deep Learning

```
1 # 네 번째 이미지 출력
2 image = x_train[3]
3 plt.imshow(image, cmap='gray')
4 plt.show()
5
6 # 행렬별 값 출력
7 for row in image:
8     for pixel in row:
9         print(f'{pixel:3d}', end=' ')
10    print()
```



```

TypeError                                Traceback (most recent call last)
/tmp/ipython-input-788219557.py in <cell line: 0>()
      5 for row in image:
      6     for pixel in row:
--> 7         print(f'{pixel:3d}', end = ' ')
      8     print()

```

```
TypeError: unsupported format string passed to numpy.ndarray.__format__
```

0	0	0	0	0	0	0	0	33	96	175	156	64	14	54	137	204	194	102	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	73	186	177	183	175	188	232	255	223	219	194	179	186	213	146	0	0	0	0	0	0	0	0
0	0	0	0	0	35	163	140	150	152	150	146	175	175	173	171	156	152	148	129	156	140	0	0	0	0	0	0	0
0	0	0	0	0	150	142	140	152	160	156	146	142	127	135	133	140	140	137	133	125	169	75	0	0	0	0	0	0
0	0	0	0	0	54	167	146	129	142	137	137	131	148	148	133	131	131	131	125	140	140	0	0	0	0	0	0	0
0	0	0	0	0	0	110	188	133	146	152	133	125	127	119	129	133	119	140	131	150	14	0	0	0	0	0	0	0
0	0	0	0	0	0	0	221	158	137	135	123	110	110	114	108	112	117	127	142	77	0	0	0	0	0	0	0	0
0	0	0	0	0	4	0	25	158	137	125	119	119	110	117	117	110	119	127	144	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	123	156	129	112	110	102	112	100	121	117	129	114	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	125	169	127	119	106	108	104	94	121	114	129	91	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	2	0	98	171	129	112	104	114	106	102	112	104	133	64	0	4	0	0	0	0	0	0	0
0	0	0	0	0	0	2	0	66	173	135	129	98	100	119	102	108	98	135	60	0	4	0	0	0	0	0	0	0
0	0	0	0	0	0	2	0	56	171	135	127	100	108	117	85	106	110	135	66	0	4	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	52	150	129	110	100	91	102	94	83	104	123	66	0	4	0	0	0	0	0	0	0
0	0	0	0	0	0	2	0	66	167	140	148	148	127	137	152	146	146	148	96	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	45	123	94	104	96	119	121	106	98	112	87	114	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	106	89	58	50	37	50	66	56	50	75	75	137	22	0	2	0	0	0	0	0	0
0	0	0	0	0	2	0	29	148	114	106	125	89	100	133	117	131	131	131	125	112	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	100	106	114	91	137	62	102	131	89	135	112	131	108	135	37	0	0	0	0	0	0	0
0	0	0	0	0	0	0	146	100	108	98	144	62	106	131	87	133	104	160	117	121	68	0	0	0	0	0	0	0
0	0	0	0	0	0	33																						

II. Deep Learning

```
1 cnn = Sequential()  
2  
3 cnn.add(Conv2D(input_shape=(28,28,1), kernel_size=(3,3),  
4               filters=32, activation='relu'))  
5 cnn.add(Conv2D(kernel_size=(3,3), filters=64, activation='relu'))  
6 cnn.add(MaxPool2D(pool_size=(2,2)))  
7 cnn.add(Dropout(0.25))  
8  
9 cnn.add(Flatten())  
10  
11 cnn.add(Dense(128, activation='relu'))  
12 cnn.add(Dropout(0.5))  
13 cnn.add(Dense(10, activation='softmax'))
```

입력과 첫 번째 합성곱 뒤에는 보통 풀링을 넣지 않는다.
'초기 정보 손실을 피하기 위해서'이다.

입력으로부터 특성 추출을 위한 Convolutional 레이어와 pooling 레이어를 거친 후,
특성 맵을 1차원으로 펼친 뒤 Fully Connected 레이어를 통해 분류 작업을 수행.
Softmax 레이어를 통해 클래스별 확률 값을 출력.

II. Deep Learning

```
cnn.add(Conv2D(input_shape=(28,28,1), kernel_size=(3,3),  
               filters=32, activation='relu'))
```

첫 번째 Convolutional 레이어를 추가.

입력 형태는 (28, 28, 1)이고, 커널 크기는 (3, 3)이며, 필터 개수는 32개. 활성화 함수로는 ReLU를 사용.

```
cnn.add(Conv2D(kernel_size=(3,3), filters=64, activation='relu'))
```

두 번째 Convolutional 레이어를 추가.

커널 크기는 (3, 3)이며, 필터 개수는 64개. 활성화 함수로는 ReLU를 사용.

```
cnn.add(MaxPool2D(pool_size=(2,2)))
```

Max Pooling 레이어를 추가. 풀 크기는 (2, 2).

II. Deep Learning

`cnn.add(Dropout(0.25))`

Dropout 레이어를 추가. 드롭아웃 비율은 0.25.
학습 중에 일부 뉴런을 무작위로 비활성화하여 과적합을 방지.

`cnn.add(Flatten())`

다차원의 특성 맵을 1차원으로 펼치는 Flatten 레이어.

`cnn.add(Dense(128, activation='relu'))`

Fully Connected 레이어를 추가. 뉴런 개수는 128개, 활성화 함수로는 ReLU.

`cnn.add(Dropout(0.5))`

Dropout 레이어는 학습 중에 각 뉴런을 유지할 확률(keep probability)을 지정.
비활성화된 뉴런은 역전파 단계에서 그래디언트(gradient)를 전파시키지 않으며, 이를 통해 학습 중에 다양한 뉴런들이 서로 독립적으로 학습될 수 있다.
노드의 수는 Dropout 레이어를 통과한 후에도 그대로 유지.
단지 학습에 활용되는 뉴런의 일부가 랜덤하게 비활성화.

`cnn.add(Dense(10, activation='softmax'))`

Fully Connected 레이어를 추가. 출력 뉴런 개수는 10개, 활성화 함수로는 Softmax.
Softmax 함수는 다중 클래스 분류를 위해 확률 값을 출력.

II. Deep Learning

```
1 cnn.compile(loss='sparse_categorical_crossentropy',
2             optimizer=tf.keras.optimizers.Adam(), metrics=['accuracy'])
3
4 cnn.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
conv2d_1 (Conv2D)	(None, 24, 24, 64)	18496
max_pooling2d (MaxPooling2D)	(None, 12, 12, 64)	0
dropout (Dropout)	(None, 12, 12, 64)	0
flatten (Flatten)	(None, 9216)	0
dense (Dense)	(None, 128)	1179776
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1290

```
=====
Total params: 1,199,882
Trainable params: 1,199,882
Non-trainable params: 0
=====
```

II. Deep Learning

파라미터 수 = (커널 크기 * 커널 크기 * 입력 채널 수 + 1) * 필터 수

•첫 번째 Conv2D 레이어:

- 커널 크기: 3x3
- 입력 채널 수: 1 (흑백 이미지이므로)
- 커널 별 적용되는 **bias : 1**
- 필터 수: 32
- **파라미터 수 = (3 * 3 * 1 + 1) * 32 = 320**

•두 번째 Conv2D 레이어:

- 커널 크기: 3x3
- 입력 채널 수: 32 (이전 레이어의 출력 채널 수와 동일)
- 커널 별 적용되는 **bias : 1**
- 필터 수: 64
- **파라미터 수 = (3 * 3 * 32 + 1) * 64 = 18496**

•MaxPooling2D /Dropout/Flatten 레이어는 파라미터를 가지지 않으므로 파라미터 수는 0.

첫 번째 Dense 레이어: 입력 뉴런 수: 9216 (전체 입력의 크기)/출력 뉴런 수: 128

•파라미터 수 = (입력 뉴런 수 * 출력 뉴런 수) + 출력 뉴런 수 = (9216 * 128) + 128 = 1179776

II. Deep Learning

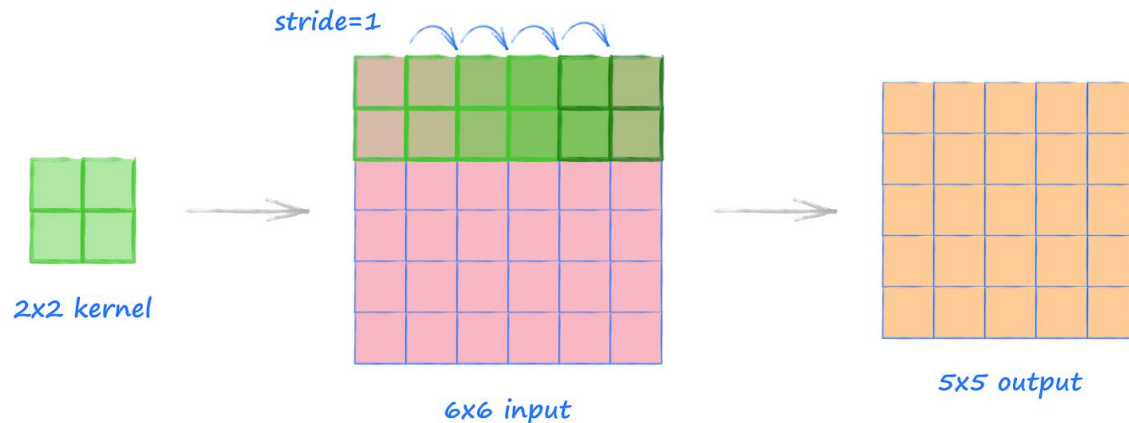
Output Size of Convolution

Convolution 연산을 수행한 후의 결과물 크기는

입력 데이터의 크기, 필터 크기, 패딩의 유무, 스트라이드(stride)의 값에 의해 결정.

1. 패딩을 적용하지 않은 경우: 출력 크기 = (입력 크기 - 필터 크기) / 스트라이드 + 1
 $(6-2)/1 + 1 = 5$ (5 * 5)

2. 패딩을 적용한 경우: 출력 크기 = (입력 크기 - 필터 크기 + 2 * 패딩) / 스트라이드 + 1
 $(6-2+2*1)/1 + 1 = 7$ (7*7)



Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
conv2d_1 (Conv2D)	(None, 24, 24, 64)	18496
max_pooling2d (MaxPooling2D)	(None, 12, 12, 64)	0

II. Deep Learning

첫 번째 Conv2D 레이어는 입력 이미지의 크기가 28x28x1, 필터의 크기는 3x3이고 총 32개의 필터를 사용. 출력 크기는 (None, 26, 26, 32).

$$\text{출력 높이} = (\text{입력 높이} - \text{필터 높이} + 2 * \text{패딩}) / \text{스트라이드} + 1$$

$$\text{출력 너비} = (\text{입력 너비} - \text{필터 너비} + 2 * \text{패딩}) / \text{스트라이드} + 1$$

패딩 값은 0이 되며, 스트라이드 값은 디폴트 값인 1이 적용.

$$\text{출력 높이} = (28 - 3 + 2 * 0) / 1 + 1 = 26$$

$$\text{출력 너비} = (28 - 3 + 2 * 0) / 1 + 1 = 26$$

두 번째 Conv2D 레이어에서는 입력 이미지의 크기가 (None, 26, 26, 32), 필터의 크기는 3x3이며 총 64개의 필터를 사용. 출력 크기는 (None, 24, 24, 64).

$$\text{출력 높이} = (26 - 3 + 2 * 0) / 1 + 1 = 24$$

$$\text{출력 너비} = (26 - 3 + 2 * 0) / 1 + 1 = 24$$

MaxPooling2D 레이어가 적용되어 출력 크기는 (None, 12, 12, 64).

Conv2D 레이어의 출력 크기가 (None, 24, 24, 64)이고, MaxPooling2D의 기본 설정을 사용한다면, 2x2 필터와 스트라이드 값 2를 적용합니다. 이에 따라 출력 크기는 다음과 같이 계산:

$$\text{출력 높이} = \text{입력 높이} / \text{풀링 크기} = 24 / 2 = 12$$

$$\text{출력 너비} = \text{입력 너비} / \text{풀링 크기} = 24 / 2 = 12$$

II. Deep Learning

Flatten 레이어가 적용되어 (None, 9216)의 크기로 변환. ($12 * 12 * 64 = 9,216$)
Dense 레이어의 입력 크기가 된다.

따라서, 주어진 예제에서 9216은 Flatten 레이어 이전의 레이어에서의 출력 크기로서, Dense 레이어의 입력 크기가 됩니다.

Dropout 레이어는 학습 중에 각 뉴런을 유지할 확률(keep probability)을 지정.

비활성화된 뉴런은 역전파 단계에서 그래디언트(gradient)를 전파시키지 않으며, 이를 통해 학습 중에 다양한 뉴런들이 서로 독립적으로 학습될 수 있다.

노드의 수는 Dropout 레이어를 통과한 후에도 그대로 유지.
단지 학습에 활용되는 뉴런의 일부가 랜덤하게 비활성화.

```
max_pooling2d (MaxPooling2D (None, 12, 12, 64)) 0
dropout (Dropout) (None, 12, 12, 64) 0
flatten (Flatten) (None, 9216) 0
```

II. Deep Learning

•MaxPooling2D /Dropout/Flatten 레이어는 파라미터를 가지지 않으므로 파라미터 수는 0.

•첫 번째 Dense 레이어:

- 입력 크기: 9216 (Flatten 이전의 출력 크기)
12 * 12 * 64
- 출력 크기: 128
- **파라미터 수 = $(9216 + 1) * 128 = 1179776$**

•두 번째 Dropout 레이어는 파라미터를 가지지 않으므로 파라미터 수는 0.

•두 번째 Dense 레이어 (출력 레이어):

- 입력 크기: 128
- 출력 크기: 10 (클래스의 수)
- **파라미터 수 = $(128 + 1) * 10 = 1290$**

flatten (Flatten)	(None, 9216)	0
dense (Dense)	(None, 128)	1179776
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1290

II. Deep Learning

cnn.compile() : 주어진 조건으로 모델을 구성하는 모델링 역할

컴파일 단계에서 모델의 손실 함수(loss function), 옵티마이저(optimizer), 그리고 평가 지표(metrics)를 설정하는 메서드.

위의 코드에서는 손실 함수로 'sparse_categorical_crossentropy'를 사용하고(정수로 라벨값이 인코딩), 옵티마이저로는 'Adam'을 선택.

또한 평가 지표로는 'accuracy'를 설정.

cnn.summary() : 모델의 구조를 요약하여 출력하는 메서드.

실행하면 모델의 레이어(layer) 정보와 각 레이어의 출력 형태(output shape) 및 파라미터 개수 등이 표시. 모델의 구조를 시각적으로 파악할 수 있다.

II. Deep Learning

```
1 hist = cnn.fit(x_train, y_train, batch_size=128,  
2               epochs=30, validation_data=(x_test, y_test))
```

```
Epoch 2/30  
469/469 [=====] - 4s 9ms/step - loss: 0.3326 - accuracy: 0.8824 - val_loss: 0.2848 - val_accuracy: 0.8965  
Epoch 3/30  
469/469 [=====] - 4s 8ms/step - loss: 0.2843 - accuracy: 0.8971 - val_loss: 0.2605 - val_accuracy: 0.9032  
Epoch 4/30  
469/469 [=====] - 8s 16ms/step - loss: 0.2547 - accuracy: 0.9079 - val_loss: 0.2483 - val_accuracy: 0.9102  
Epoch 5/30  
469/469 [=====] - 7s 15ms/step - loss: 0.2343 - accuracy: 0.9135 - val_loss: 0.2330 - val_accuracy: 0.9141  
Epoch 6/30  
469/469 [=====] - 5s 11ms/step - loss: 0.2161 - accuracy: 0.9208 - val_loss: 0.2204 - val_accuracy: 0.9215  
Epoch 7/30  
469/469 [=====] - 4s 8ms/step - loss: 0.2012 - accuracy: 0.9251 - val_loss: 0.2175 - val_accuracy: 0.9225  
Epoch 8/30  
469/469 [=====] - 4s 8ms/step - loss: 0.1830 - accuracy: 0.9327 - val_loss: 0.2080 - val_accuracy: 0.9250  
Epoch 9/30  
469/469 [=====] - 4s 9ms/step - loss: 0.1712 - accuracy: 0.9363 - val_loss: 0.2250 - val_accuracy: 0.9206
```

```
Epoch 28/30  
469/469 [=====] - 4s 9ms/step - loss: 0.0717 - accuracy: 0.9721 - val_loss: 0.2624 - val_accuracy: 0.9350  
Epoch 29/30  
469/469 [=====] - 4s 9ms/step - loss: 0.0747 - accuracy: 0.9715 - val_loss: 0.2608 - val_accuracy: 0.9318  
Epoch 30/30  
469/469 [=====] - 4s 9ms/step - loss: 0.0719 - accuracy: 0.9722 - val_loss: 0.2812 - val_accuracy: 0.9279
```


II. Deep Learning

```
hist = cnn.fit(x_train, y_train, batch_size=128,  
              epochs=30, validation_data=(x_test, y_test))
```

CNN 모델을 주어진 데이터로 학습하는 과정을 수행.

`x_train`은 학습 데이터의 입력, `y_train`은 학습 데이터의 출력(레이블).

`batch_size=128`는 한 번의 학습 단계에서 사용되는 샘플의 개수.

`epochs=30`은 전체 데이터셋을 몇 번 반복하여 학습할지.

`validation_data=(x_test, y_test)`는 학습 도중에 검증을 위해 사용되는 데이터.

학습이 진행되면서 손실 함수 값과 정확도 등의 지표가 `hist` 변수에 저장.

`hist` 변수는 학습 과정에서 기록된 지표들을 나타내는 정보를 담고 있다.

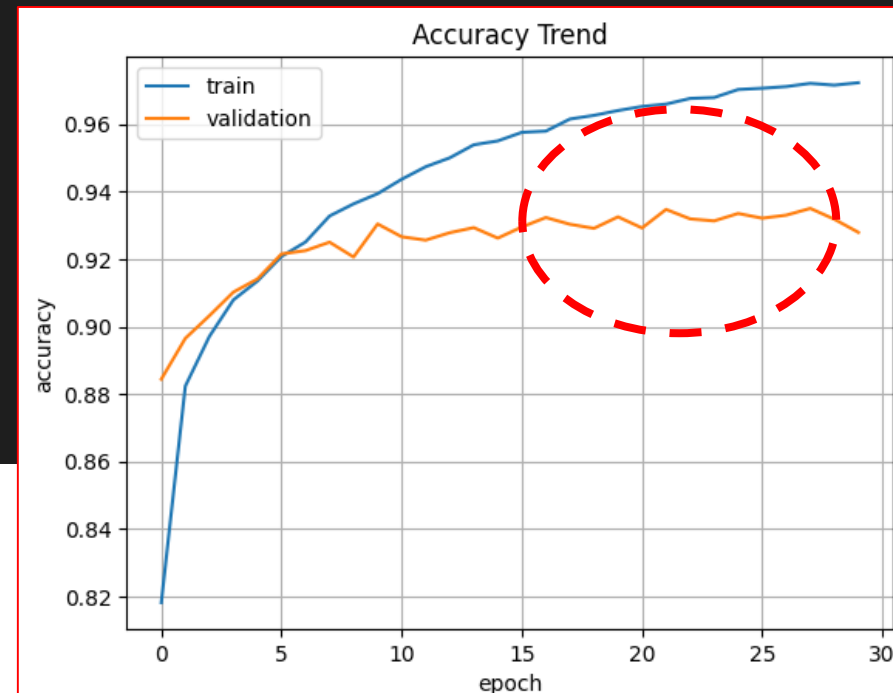
II. Deep Learning

```
1 cnn.evaluate(x_test, y_test)
```

```
313/313 [=====] - 1s 3ms/step - loss: 0.2812 - accuracy: 0.9279  
[0.2811889350414276, 0.9279000163078308]
```

```
1  
2 import matplotlib.pyplot as plt  
3  
4 plt.plot(hist.history['accuracy'])  
5 plt.plot(hist.history['val_accuracy'])  
6 plt.title('Accuracy Trend')  
7 plt.ylabel('accuracy')  
8 plt.xlabel('epoch')  
9 plt.legend(['train', 'validation'], loc='best')  
10 plt.grid()  
11 plt.show()
```

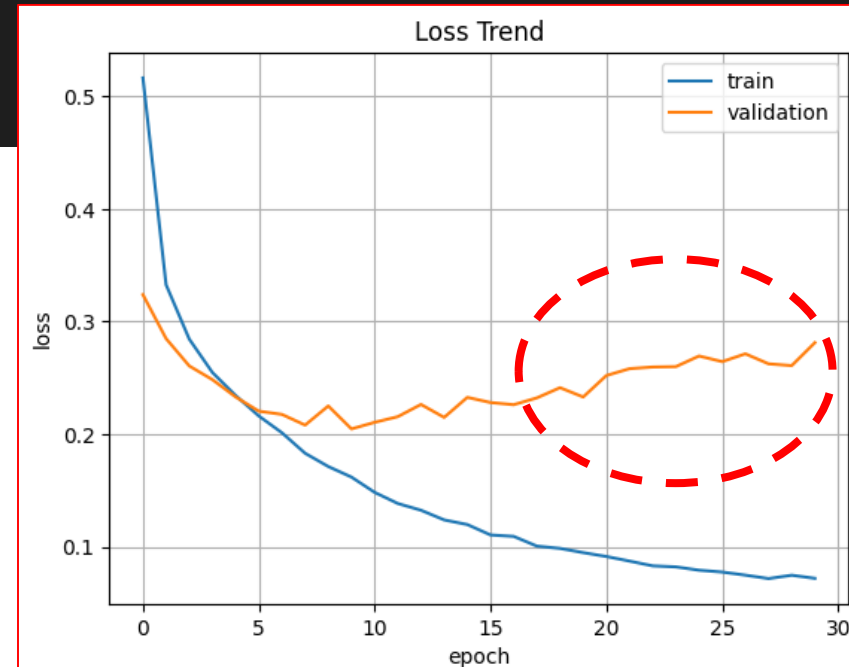
Loss 28.12%, Accuracy 92.79 %,
학습 데이터의 과적합으로 검증 자료의 정확도 감소



II. Deep Learning

```
1 plt.plot(hist.history['loss'])
2 plt.plot(hist.history['val_loss'])
3 plt.title('Loss Trend')
4 plt.ylabel('loss')
5 plt.xlabel('epoch')
6 plt.legend(['train', 'validation'], loc='best')
7 plt.grid()
8 plt.show()
```

Loss 28.12%, Accuracy 92.79 %,
학습 데이터의 과적합으로 검증 자료의 정확도 감소



1. Deep Learning **CNN** with CIFAR10

CNN with CIFAR10

비행기

자동차

새

고양이

사슴

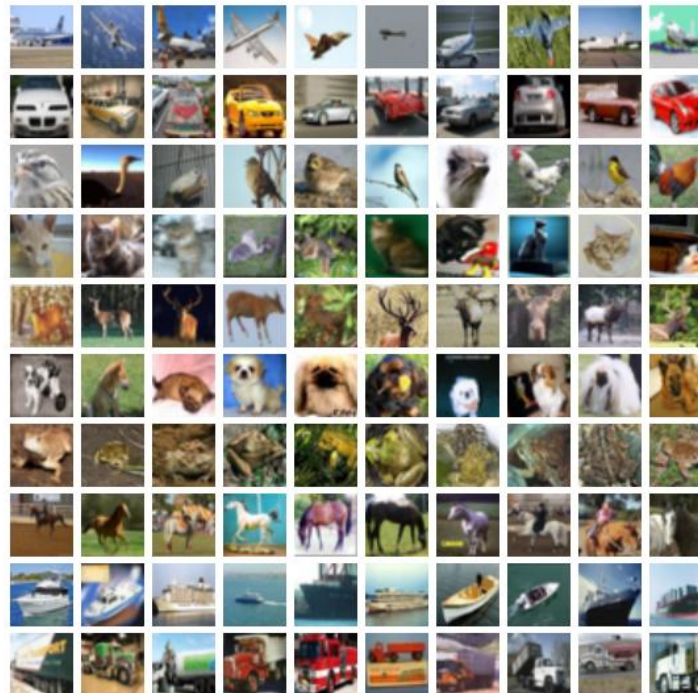
개

개구리

말

배

트럭



II. Deep Learning

CIFAR-10

10개의 다른 클래스로 구성된 60,000개의 32x32 컬러 이미지로 구성.

Airplane(비행기) Automobile(자동차) Bird(새) Cat(고양이)
Deer(사슴) Dog(개) Frog(개구리) Horse(말) Ship(배) Truck(트럭)

훈련 세트에는 클래스마다 5,000개의 이미지가 있고(50,000),
테스트 세트에는 클래스마다 1,000개의 이미지(10,000)

이미지는 다양한 크기, 각도, 조명 조건 및 배경으로 구성, 실제 세계에서의 다양한 상황을 모델에 적용/일반화를 향상.



II. Deep Learning

```
1 import numpy as np
2 import tensorflow as tf
3 from tensorflow.keras.datasets import cifar10
4 from tensorflow.keras.models import Sequential
5 from tensorflow.keras.layers import Conv2D, MaxPool2D
6 from tensorflow.keras.layers import Flatten, Dense, Dropout
```

```
1 (x_train, y_train), (x_test, y_test) = cifar10.load_data()
2
3 x_train = x_train.reshape(-1, 32, 32, 3)
4 x_test = x_test.reshape(-1, 32, 32, 3)
5
6 print(x_train.shape, x_test.shape)
7 print(y_train.shape, y_test.shape)
8
9 x_train = x_train.astype(np.float32) / 255.0
10 x_test = x_test.astype(np.float32) / 255.0
```

```
(50000, 32, 32, 3) (10000, 32, 32, 3)
(50000, 1) (10000, 1)
```

II. Deep Learning

cifar10.load_data():

CIFAR-10 데이터셋 업로드.

x_train과 x_test의 reshape()형상 변환:

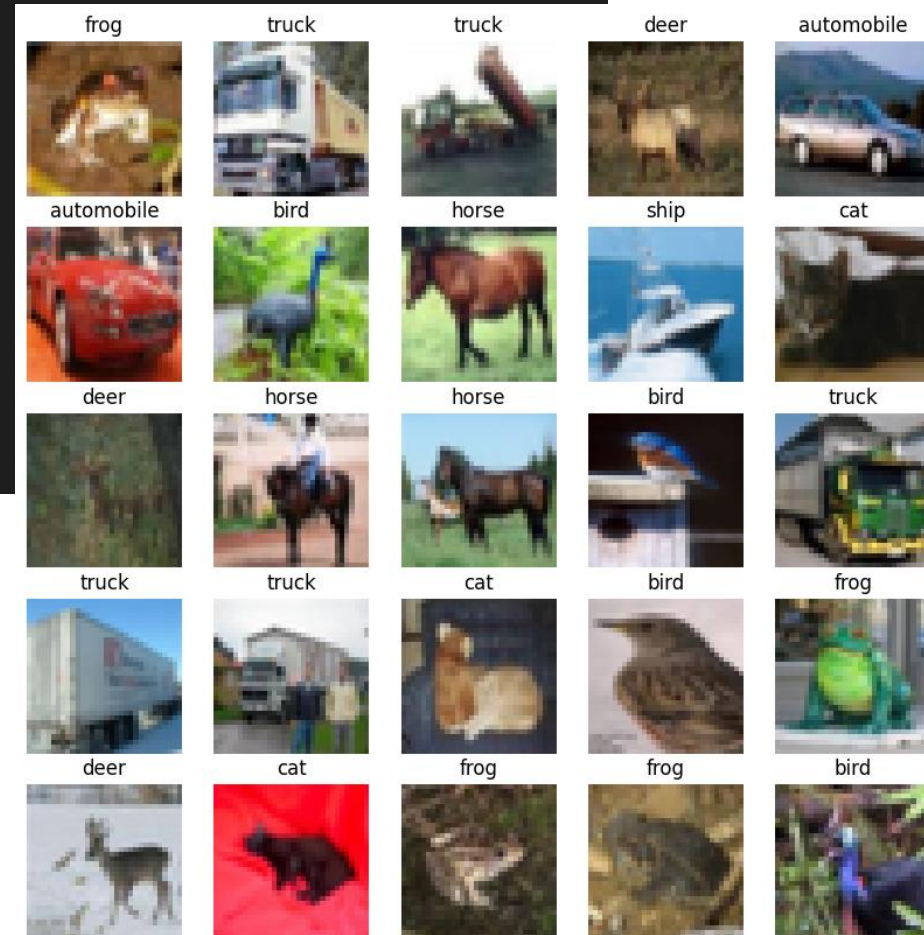
CIFAR-10 데이터셋의 이미지는 32x32 크기이며, 채널 수는 RGB 이미지 3. (훈련 데이터 수, 32, 32, 3)과 (테스트 데이터 수, 32, 32, 3)의 형태로 출력. -1은 해당위치에서 자동으로 4차원 데이터 생성 옵션.

데이터셋 정규화:

x_train과 x_test의 값을 0과 1 사이의 범위로 정규화

II. Deep Learning

```
1 import matplotlib.pyplot as plt
2 class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
3               'dog', 'frog', 'horse', 'ship', 'truck']
4
5 # 샘플 데이터 출력
6 plt.figure(figsize=(10, 10))
7 for i in range(25):
8     plt.subplot(5, 5, i+1)
9     plt.imshow(x_train[i])
10    plt.title(class_names[y_train[i][0]])
11    plt.axis('off')
12 plt.show()
```



1 y_train
array([[6], [9], [9], ..., [9], [1], [1]], dtype=uint8)

1 y_test
array([[3], [8], [8], ..., [5], [1], [7]], dtype=uint8)

II. Deep Learning

```
1 c_model = Sequential()
2
3 c_model.add(Conv2D(input_shape=(32,32,3), kernel_size=(3,3),
4                     filters=32, activation='relu'))
5 c_model.add(Conv2D(kernel_size=(3,3), filters=64, activation='relu'))
6 c_model.add(MaxPool2D(pool_size=(2,2)))
7 c_model.add(Dropout(0.25))
8
9 c_model.add(Flatten())
10
11 c_model.add(Dense(128, activation='relu'))
12 c_model.add(Dropout(0.5))
13 c_model.add(Dense(10, activation='softmax'))
14
```

II. Deep Learning

Sequential 모델 생성: `c_model = Sequential()`

Conv2D 레이어 추가: `c_model.add(Conv2D(input_shape=(32,32,3), kernel_size=(3,3), filters=32, activation='relu'))`

input_shape는 입력 데이터의 형태, (32, 32, 3)

kernel_size는 필터(커널)의 크기, (3, 3)

filters는 필터의 개수, 32개

activation은 활성화 함수, 'relu'

Conv2D 레이어 추가: `c_model.add(Conv2D(kernel_size=(3,3), filters=64, activation='relu'))`

이전 레이어와 동일한 커널 크기와 활성화 함수, 필터의 개수는 64개.

MaxPooling2D 레이어 추가: `c_model.add(MaxPool2D(pool_size=(2,2)))`

pool_size는 풀링의 크기로, (2, 2)로 설정.

II. Deep Learning

Dropout 레이어 : `c_model.add(Dropout(0.25))`, 과적합을 방지하기 위해 일부 뉴런을 랜덤하게 비활성화.

Flatten 레이어 추가: `c_model.add(Flatten())`, 다차원의 데이터를 1차원으로 변환

Dense 레이어 추가: `c_model.add(Dense(128, activation='relu'))` 뉴런 128개
활성화 함수로는 'relu'

Dropout 레이어 추가: `c_model.add(Dropout(0.5))`, 50%의 뉴런을 비활성화

마지막 Dense 레이어 추가: `c_model.add(Dense(10, activation='softmax'))`
뉴런의 개수는 10개로 설정, 활성화 함수로는 'softmax'

II. Deep Learning

```
1 c_model.compile(loss='sparse_categorical_crossentropy',  
2                 optimizer=tf.keras.optimizers.Adam(), metrics=['accuracy'])  
3  
4 c_model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 30, 30, 32)	896
conv2d_3 (Conv2D)	(None, 28, 28, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 14, 14, 64)	0
dropout_2 (Dropout)	(None, 14, 14, 64)	0
flatten_1 (Flatten)	(None, 12544)	0
dense_2 (Dense)	(None, 128)	1605760
dropout_3 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 10)	1290

II. Deep Learning

1. 출력 형태 계산:

1) Convolutional 레이어:

1) 출력 높이 = $(\text{입력 높이} - \text{커널 높이} + 2 * \text{패딩}) / \text{스트라이드} + 1$

2) 출력 너비 = $(\text{입력 너비} - \text{커널 너비} + 2 * \text{패딩}) / \text{스트라이드} + 1$

2) MaxPooling 레이어:

1) 출력 높이 = $\text{입력 높이} / \text{풀링 윈도우 높이}$

2) 출력 너비 = $\text{입력 너비} / \text{풀링 윈도우 너비}$

3) Flatten 레이어: 출력 형태는 (None, n). 여기서 n은 이전 레이어의 출력의 요소 수.

4) Dense 레이어: 입력 형태에 따라 출력 형태가 결정.

2. 파라미터 수 계산:

1) Convolutional 레이어: $(\text{커널 높이} * \text{커널 너비} * \text{입력 채널 수} + 1) * \text{필터 수}$

2) Dense 레이어: $(\text{이전 레이어의 출력 크기} + 1) * \text{현재 레이어의 뉴런 수}$

II. Deep Learning

conv2d (Conv2D) 레이어: (input_shape=(32,32,3), kernel_size=(3,3), filters=32, activation='relu')

입력 형태: (None, 32, 32, 3)

출력 형태: (None, 30, 30, 32), $30 = (32 - 3 + 2 * 0) / 1 + 1$

파라미터 수: $(3 * 3 * 3 + 1) * 32 = 896$

conv2d_1 (Conv2D) 레이어: (kernel_size=(3,3), filters=64, activation='relu')

입력 형태: (None, 30, 30, 32)

출력 형태: (None, 28, 28, 64), $28 = (30 - 3 + 2 * 0) / 1 + 1$

파라미터 수: $(3 * 3 * 32 + 1) * 64 = 18,496$

max_pooling2d (MaxPooling2D) 레이어:

입력 형태: (None, 28, 28, 64)

출력 형태: (None, 14, 14, 64), $14 = 28 / 2$

파라미터 수: 0 (풀링 레이어는 학습 가능한 파라미터가 없음)

dropout (Dropout) 레이어:

입력 형태: (None, 14, 14, 64)

출력 형태: (None, 14, 14, 64)

파라미터 수: 0 (드롭아웃 레이어는 학습 가능한 파라미터가 없음)

II. Deep Learning

flatten (Flatten) 레이어:

입력 형태: (None, 14, 14, 64)

출력 형태: (None, 12,544), $12,544 = 14 * 14 * 64$

파라미터 수: 0 (플래튼 레이어는 학습 가능한 파라미터가 없음)

dense (Dense) 레이어:

입력 형태: (None, 12,544)

출력 형태: (None, 128)

파라미터 수: $(12,544 + 1) * 128 = 1,605,760$

dropout_1 (Dropout) 레이어:

입력 형태: (None, 128)

출력 형태: (None, 128)

파라미터 수: 0 (드롭아웃 레이어는 학습 가능한 파라미터가 없음)

dense_1 (Dense) 레이어:

입력 형태: (None, 128)

출력 형태: (None, 10)

파라미터 수: $(128 + 1) * 10 = 1,290$

II. Deep Learning

```
1 c_history = c_model.fit(x_train, y_train, batch_size=128,  
2                       epochs=30, validation_data=(x_test, y_test))
```

```
Epoch 1/30  
391/391 [=====] - 7s 12ms/step - loss: 1.6448 - accuracy: 0.4046 - val_loss: 1.2606 - val_accuracy: 0.5633  
Epoch 2/30  
391/391 [=====] - 4s 11ms/step - loss: 1.2953 - accuracy: 0.5391 - val_loss: 1.0846 - val_accuracy: 0.6235  
Epoch 3/30  
391/391 [=====] - 5s 12ms/step - loss: 1.1552 - accuracy: 0.5937 - val_loss: 0.9954 - val_accuracy: 0.6521  
Epoch 4/30  
391/391 [=====] - 4s 11ms/step - loss: 1.0701 - accuracy: 0.6220 - val_loss: 0.9487 - val_accuracy: 0.6656  
Epoch 5/30  
391/391 [=====] - 5s 12ms/step - loss: 1.0075 - accuracy: 0.6414 - val_loss: 0.9209 - val_accuracy: 0.6791  
Epoch 6/30  
391/391 [=====] - 4s 11ms/step - loss: 0.9497 - accuracy: 0.6641 - val_loss: 0.8922 - val_accuracy: 0.6853  
Epoch 7/30  
391/391 [=====] - 4s 11ms/step - loss: 0.8997 - accuracy: 0.6816 - val_loss: 0.8837 - val_accuracy: 0.6923  
Epoch 8/30  
391/391 [=====] - 4s 11ms/step - loss: 0.8597 - accuracy: 0.6991 - val_loss: 0.8752 - val_accuracy: 0.7000  
Epoch 9/30  
391/391 [=====] - 4s 11ms/step - loss: 0.8200 - accuracy: 0.7166 - val_loss: 0.8667 - val_accuracy: 0.7077  
Epoch 10/30  
391/391 [=====] - 4s 11ms/step - loss: 0.7806 - accuracy: 0.7341 - val_loss: 0.8582 - val_accuracy: 0.7154  
Epoch 11/30  
391/391 [=====] - 4s 11ms/step - loss: 0.7414 - accuracy: 0.7516 - val_loss: 0.8497 - val_accuracy: 0.7231  
Epoch 12/30  
391/391 [=====] - 4s 11ms/step - loss: 0.7023 - accuracy: 0.7691 - val_loss: 0.8412 - val_accuracy: 0.7308  
Epoch 13/30  
391/391 [=====] - 4s 11ms/step - loss: 0.6633 - accuracy: 0.7866 - val_loss: 0.8327 - val_accuracy: 0.7385  
Epoch 14/30  
391/391 [=====] - 4s 11ms/step - loss: 0.6244 - accuracy: 0.8041 - val_loss: 0.8242 - val_accuracy: 0.7462  
Epoch 15/30  
391/391 [=====] - 4s 11ms/step - loss: 0.5856 - accuracy: 0.8216 - val_loss: 0.8157 - val_accuracy: 0.7539  
Epoch 16/30  
391/391 [=====] - 4s 11ms/step - loss: 0.5469 - accuracy: 0.8391 - val_loss: 0.8072 - val_accuracy: 0.7616  
Epoch 17/30  
391/391 [=====] - 4s 11ms/step - loss: 0.5083 - accuracy: 0.8566 - val_loss: 0.7987 - val_accuracy: 0.7693  
Epoch 18/30  
391/391 [=====] - 4s 11ms/step - loss: 0.4698 - accuracy: 0.8741 - val_loss: 0.7902 - val_accuracy: 0.7770  
Epoch 19/30  
391/391 [=====] - 4s 11ms/step - loss: 0.4314 - accuracy: 0.8916 - val_loss: 0.7817 - val_accuracy: 0.7847  
Epoch 20/30  
391/391 [=====] - 4s 11ms/step - loss: 0.3931 - accuracy: 0.9091 - val_loss: 0.7732 - val_accuracy: 0.7924  
Epoch 21/30  
391/391 [=====] - 4s 11ms/step - loss: 0.3548 - accuracy: 0.9266 - val_loss: 0.7647 - val_accuracy: 0.8001  
Epoch 22/30  
391/391 [=====] - 4s 11ms/step - loss: 0.3166 - accuracy: 0.9441 - val_loss: 0.7562 - val_accuracy: 0.8078  
Epoch 23/30  
391/391 [=====] - 4s 11ms/step - loss: 0.2784 - accuracy: 0.9616 - val_loss: 0.7477 - val_accuracy: 0.8155  
Epoch 24/30  
391/391 [=====] - 4s 11ms/step - loss: 0.2403 - accuracy: 0.9791 - val_loss: 0.7392 - val_accuracy: 0.8232  
Epoch 25/30  
391/391 [=====] - 4s 11ms/step - loss: 0.2023 - accuracy: 0.9966 - val_loss: 0.7307 - val_accuracy: 0.8309  
Epoch 26/30  
391/391 [=====] - 4s 11ms/step - loss: 0.1644 - accuracy: 1.0141 - val_loss: 0.7222 - val_accuracy: 0.8386  
Epoch 27/30  
391/391 [=====] - 4s 11ms/step - loss: 0.1266 - accuracy: 1.0316 - val_loss: 0.7137 - val_accuracy: 0.8463  
Epoch 28/30  
391/391 [=====] - 4s 11ms/step - loss: 0.0888 - accuracy: 1.0491 - val_loss: 0.7052 - val_accuracy: 0.8540  
Epoch 29/30  
391/391 [=====] - 4s 11ms/step - loss: 0.0511 - accuracy: 1.0666 - val_loss: 0.6967 - val_accuracy: 0.8617  
Epoch 30/30  
391/391 [=====] - 5s 12ms/step - loss: 0.0134 - accuracy: 1.0841 - val_loss: 0.6882 - val_accuracy: 0.8694
```

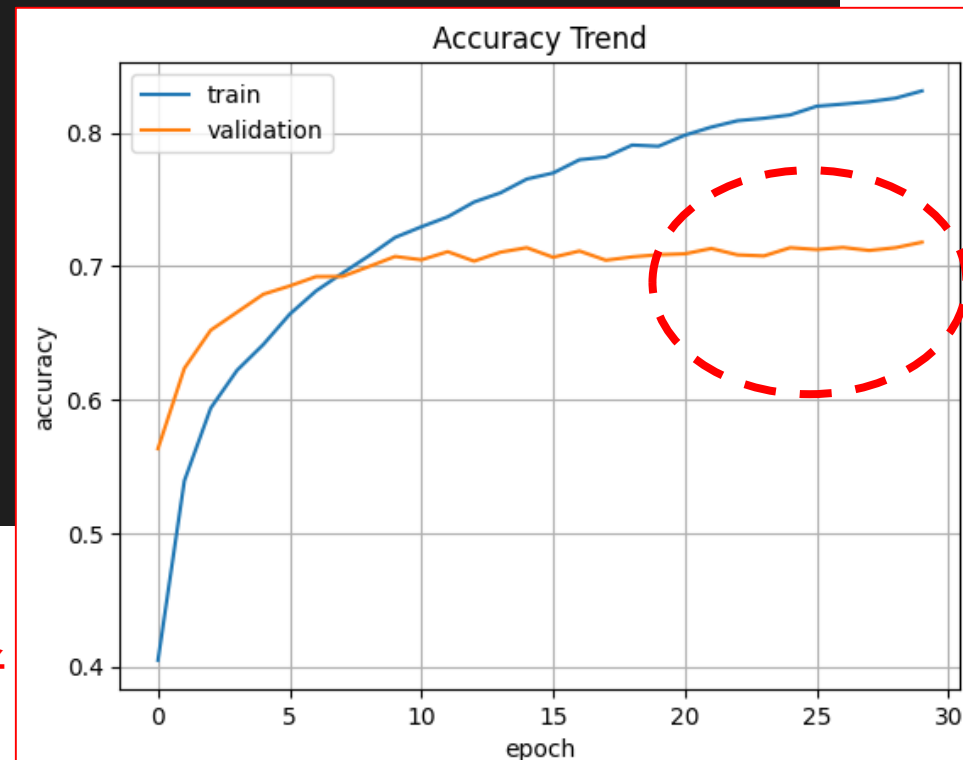

II. Deep Learning

```
1 c_model.evaluate(x_test, y_test)
```

```
313/313 [=====] - 1s 3ms/step - loss: 0.9695 - accuracy: 0.7181  
[0.9694998264312744, 0.7181000113487244]
```

```
1 import matplotlib.pyplot as plt  
2  
3 plt.plot(c_history.history['accuracy'])  
4 plt.plot(c_history.history['val_accuracy'])  
5 plt.title('Accuracy Trend')  
6 plt.ylabel('accuracy')  
7 plt.xlabel('epoch')  
8 plt.legend(['train', 'validation'], loc='best')  
9 plt.grid()  
10 plt.show()
```

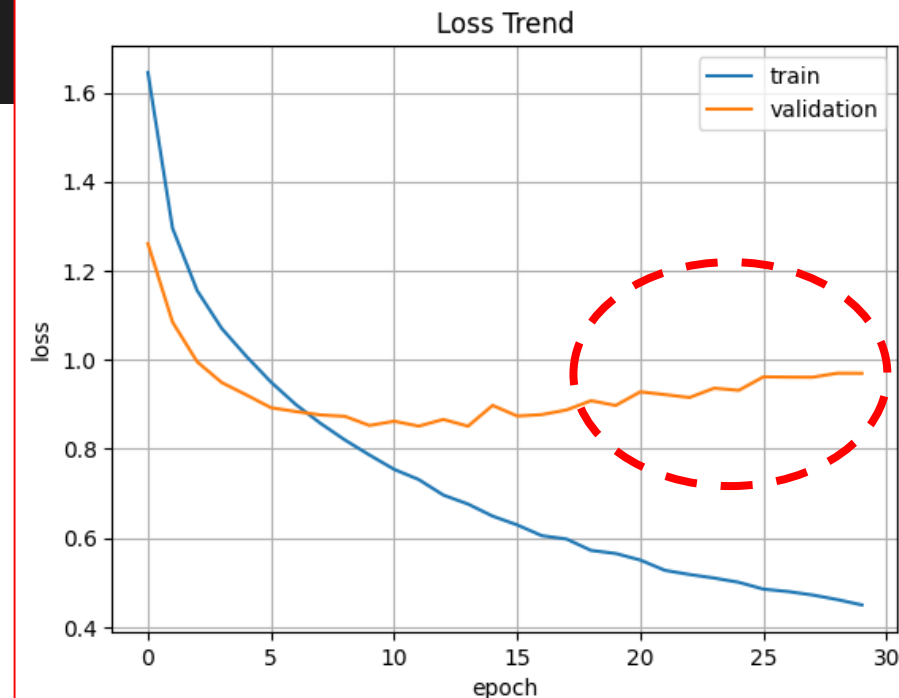
Loss 96.94%, Accuracy 71.81 %,
학습 데이터의 과적합으로 검증 자료의 정확도 감소



II. Deep Learning

```
1 plt.plot(c_history.history['loss'])
2 plt.plot(c_history.history['val_loss'])
3 plt.title('Loss Trend')
4 plt.ylabel('loss')
5 plt.xlabel('epoch')
6 plt.legend(['train', 'validation'], loc='best')
7 plt.grid()
8 plt.show()
```

Loss 96.94%, Accuracy 71.81 %,
학습 데이터의 과적합으로 검증 자료의 정확도 감소



II. Deep Learning

같은 test 데이터를 썼는데
`fit(..., validation_data=...)`의 `val_accuracy`와
`model.evaluate()` 결과가 완전히 같지 않은 이유

```
Epoch 29/30
391/391 ————— 10s 12ms/step - accuracy: 0.8466 - loss: 0.4103 - val_accuracy: 0.7094 - val_loss: 0.9876
Epoch 30/30
391/391 ————— 4s 9ms/step - accuracy: 0.8497 - loss: 0.4027 - val_accuracy: 0.7137 - val_loss: 0.9712

1 c_model.evaluate(x_test, y_test)

313/313 ————— 2s 3ms/step - accuracy: 0.7168 - loss: 0.9466
[0.971243679523468, 0.713699996471405]
```

이유1 : 측정 시점이 다름

- (1) `fit()` 중의 `val_accuracy` : 각 epoch가 끝날 때 가중치 상태로 validation 데이터를 한 번 평가
- (2) `model.evaluate()` : 학습 종료 후 최종 가중치 상태에서 test 데이터를 다시 전체 평가

이유2 : Dropout / BatchNorm이 있으면

- (1) `fit()` 중 validation 내부적으로 배치 단위로 계산
- (2) `evaluate()` : 완전히 inference 모드, BatchNorm 통계가 더 안정된 상태

II. Deep Learning

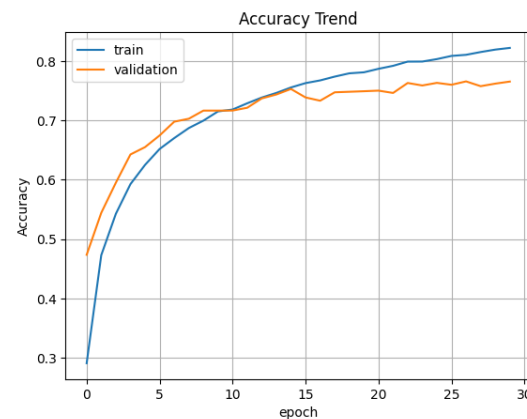
모델은 훈련 데이터에 잘 적합되었지만,
검증 데이터에 대한 일반화 성능이 한계에 다다른 것으로 보임.
Epoch 10 근처에서 Early Stopping 했으면 더 나은 일반화 성능을 얻을 수 있었을 가능성 있음

EarlyStopping 콜백 추가 → 검증 손실이 더 이상 감소하지 않을 때 학습 조기 종료

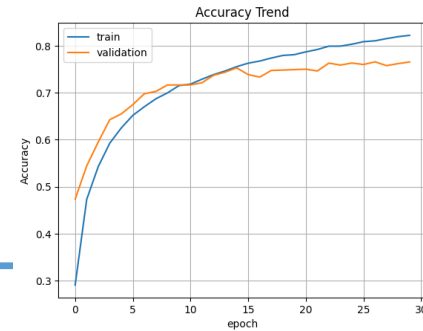
Dropout 비율 증가 또는 L2 정규화 → 모델 복잡도 제어로 과적합 방지

데이터 증강 → 더 다양한 학습 패턴 제공으로 일반화 능력 향상

에포크 수 줄이기 → 현재 30 에포크는 과적합을 심화시키고 있음



II. Deep Learning



실습) CNN with MNIST, Fashion_MNIST, CIFAR 10의 과적합 코드를 바탕으로
Batch Normalization, dropout, earlystopping 등을 사용하여
과적합이 개선된 코드의 코랩 링크를 공유하세요(LLM사용 가능합니다)

```
model = models.Sequential([
    layers.Flatten(input_shape=(28, 28)), # 28x28 이미지를
    1차원으로 평탄화

    layers.Dense(512),
    layers.BatchNormalization(), # 배치 정규화: 학습 안정화
    layers.Activation('relu'),
    layers.Dropout(0.2), # 드롭아웃: 과적합 방지

    layers.Dense(256),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dropout(0.2),

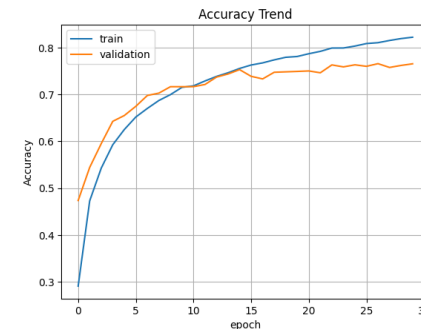
    layers.Dense(10, activation='softmax') # 출력층 (10개 클래스)
])
```

```
# Early Stopping 설정
# 검증 손실(val_loss)이 3회(patience) 이상 개선안되면 자동 종료
early_stop = callbacks.EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)

ModelFit = model.fit(
    x_train, y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2, # 학습 데이터의 20%를 검증에 사용
    callbacks=[early_stop], # 조기 종료 적용
    verbose=1
)
```

II. Deep Learning

```
from tensorflow.keras.layers import (  
    Input, Conv2D, MaxPooling2D, Flatten,  
    Dense, Dropout, BatchNormalization, Activation  
)  
  
cnn = Sequential()  
  
# Input  
cnn.add(Input(shape=(28, 28, 1)))  
  
# Conv Block 1  
cnn.add(Conv2D(32, (3, 3), padding="same"))  
cnn.add(BatchNormalization())  
cnn.add(Activation("relu"))  
  
# Conv Block 2  
cnn.add(Conv2D(64, (3, 3), padding="same"))  
cnn.add(BatchNormalization())  
cnn.add(Activation("relu"))  
  
cnn.add(MaxPooling2D((2, 2)))  
cnn.add(Dropout(0.25))  
  
# Fully Connected  
cnn.add(Flatten())  
  
cnn.add(Dense(128))  
cnn.add(BatchNormalization())  
cnn.add(Activation("relu"))  
cnn.add(Dropout(0.5))  
  
# Output  
cnn.add(Dense(10, activation="softmax"))
```



II. Deep Learning

```
from tensorflow.keras.layers import (Dropout, BatchNormalization, Activation, Input)

c_model = Sequential()

# Input
c_model.add(Input(shape=(32, 32, 3)))

# Conv Block 1
c_model.add(Conv2D(32, (3, 3), padding="same"))
c_model.add(BatchNormalization())
c_model.add(Activation("relu"))

# Conv Block 2
c_model.add(Conv2D(64, (3, 3), padding="same"))
c_model.add(BatchNormalization())
c_model.add(Activation("relu"))

c_model.add(MaxPooling2D(pool_size=(2, 2)))
c_model.add(Dropout(0.25))

# Fully Connected
c_model.add(Flatten())

c_model.add(Dense(128))
c_model.add(BatchNormalization())
c_model.add(Activation("relu"))
c_model.add(Dropout(0.5))

# Output
c_model.add(Dense(10, activation="softmax"))
```

II. Deep Learning

```
# Early Stopping 설정
# 검증 손실(val_loss)이 3회(patience) 이상 개선되지 않으면 학습 자동 종료
early_stop = callbacks.EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)
```

```
import tensorflow as tf
from tensorflow.keras import layers, models, callbacks

c_history = c_model.fit(x_train, y_train, batch_size = 128,
                        epochs=30, validation_data=(x_test, y_test),
                        callbacks=[early_stop])
```


1. Deep Learning **CNN 성능 개선**

CNN 성능 개선

1. Deep Learning CNN 성능 개선

CNN의 성능을 향상시키는 방법

- 1.데이터 증강 (Data Augmentation):** 데이터 증강은 원본 데이터셋을 변형하여 새로운 데이터를 생성하는 기법. 이미지 회전, 확대/축소, 이동, 뒤집기 등의 변형을 적용. 과적합을 방지
- 2.전이 학습 (Transfer Learning):** 대규모 데이터셋에서 사전 훈련된 모델의 특징 추출 능력 활용
- 3.하이퍼파라미터 조정 :** 그리드 탐색(Grid Search), 랜덤 탐색(Random Search).
- 4.정규화 (Regularization):** 드롭아웃(Dropout)과 같은 기법을 사용하여 일부 뉴런을 무작위로 비활성화.
- 5.앙상블 학습 (Ensemble Learning):** 여러 개의 CNN 모델을 독립적으로 훈련하고, 이들의 예측을 평균 또는 다수결 등의 방법으로 결합

II. Deep Learning

CIFAR Data **Normalization**



Data **Augmentation**



CNN Modeling



Model Compiling



Model Evaluation

II. Deep Learning

TensorFlow v2.12.0

Overview

Python

C++

Java

More

Filter

image

Overview

DirectoryIterator

ImageDataGenerator

Iterator

TensorFlow > API > TensorFlow v2.12.0 > Python

도움이 되었나요?  

tf.keras.preprocessing.image.ImageDataGenerator



```
tf.keras.preprocessing.image.ImageDataGenerator(  
    featurewise_center=False,  
    samplewise_center=False,  
    featurewise_std_normalization=False,  
    samplewise_std_normalization=False,  
    zca_whitening=False,  
    zca_epsilon=1e-06,  
    rotation_range=0,  
    width_shift_range=0.0,  
    height_shift_range=0.0,  
    brightness_range=None,  
    shear_range=0.0,  
    zoom_range=0.0,  
    channel_shift_range=0.0,  
    fill_mode='nearest',  
    cval=0.0,  
    horizontal_flip=False,  
    vertical_flip=False,  
    rescale=None,  
    preprocessing_function=None,  
    data_format=None,  
    validation_split=0.0,  
    interpolation_order=1,  
    dtype=None
```

II. Deep Learning 배치정규화

배치 정규화(Batch Normalization)

배치 정규화의 목적

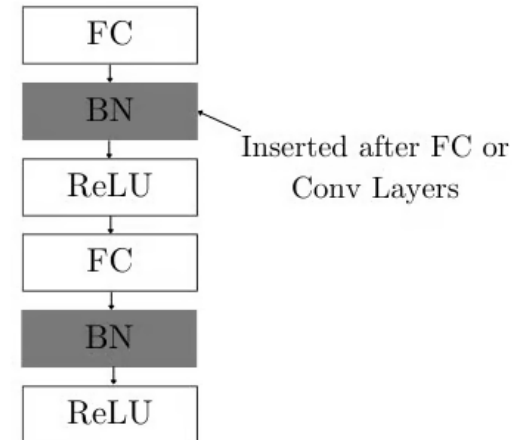
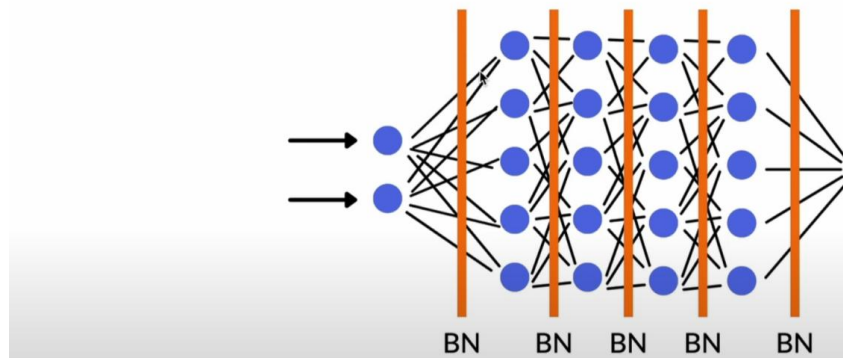
- 학습 중 은닉층 출력 분포의 불안정성을 줄이기 위함
- 가중치 업데이트로 인해 계속 변하는 분포를 매 step마다 안정화
- 입력 정규화는 출발선 정렬
- 배치 정규화는 달리는 중 균형 조절

배치 정규화의 핵심 동작

- 현재 배치 기준으로 평균·분산 계산
- 배치 내 각 샘플을 개별적으로 표준화
- 학습 가능한 파라미터(γ , β)로 다시 조정

II. Deep Learning 배치정규화

Batch Normalization



II. Deep Learning 배치정규화

구분	BatchNorm	LayerNorm
정규화 기준	배치 축	feature 축(한 샘플 내부)
배치 의존성	있음	없음
Batch size 영향	큼	거의 없음

LayerNorm : 하나의 샘플을 기준으로:

[노드1, 노드2, 노드3, ..., 노드D]

평균/분산 계산 대상: 노드 방향배치 크기와 무관샘플 하나만 있어도 정상 작동
↳ "한 데이터 내부의 표현 안정화"

Batch Normalization : BatchNorm은 하나의 노드를 기준으로:

[샘플1, 샘플2, 샘플3, ..., 샘플N]

평균/분산 계산 대상: 배치 방향배치가 작으면 불안정학습
/추론 시 동작이 다름

↳ "같은 노드가 배치마다 흔들리지 않게 안정화"

II. Deep Learning 배치정규화

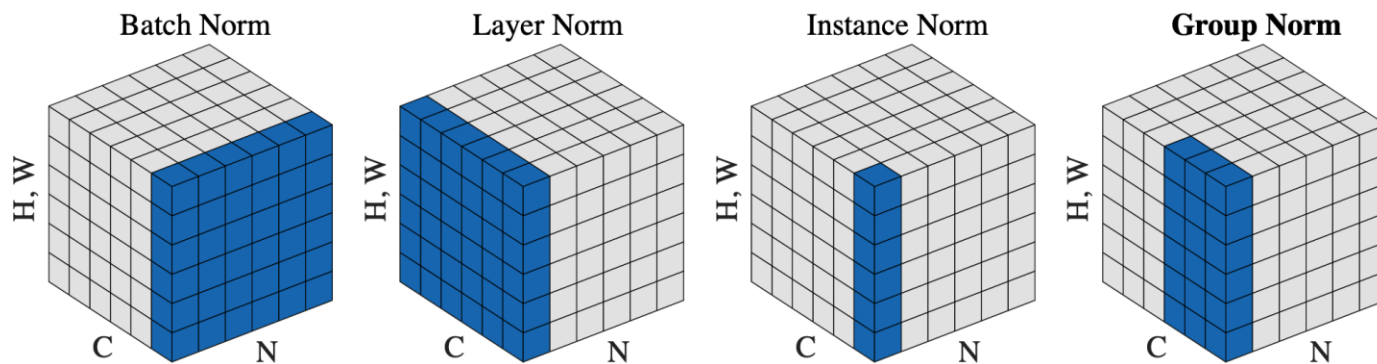


Figure 2. **Normalization methods.** Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

II. Deep Learning 배치정규화

Batch (N samples)

sample 1 → LN
sample 2 → LN
sample 3 → LN
...
sample N → LN

직관적 비유

- LayerNorm 학생 각자 자기 평균/편차로 점수 정규화
- BatchNorm 과목 하나를 기준으로 반 전체 평균/편차로 정규화

👉 학생 수만큼 시험지를 쌓는 게 아님

II. Deep Learning **PyTorch**

PyTorch

II. Deep Learning **PyTorch**

 PyTorch

Learn ▼

Ecosystem ▼

Edge ▼

Docs ▼

Blog & News ▼

About ▼

Become a Member



PyTorch

GET STARTED

Choose Your Path: Install PyTorch Locally or
Launch Instantly on Supported Cloud Platforms

Get started >

II. Deep Learning PyTorch

단계	목적	주요 메서드/API	설명
1. 데이터 로딩	학습 데이터 준비	<code>torchvision.datasets</code> , <code>torch.utils.data.DataLoader</code>	<code>Dataset</code> 과 배치 처리용 <code>DataLoader</code>
2. 전처리	Tensor 변환, 정규화	<code>transforms.ToTensor()</code> , <code>transforms.Normalize()</code>	<code>torchvision transforms</code> 모듈
3. 모델 정의	모델 아키텍처 구현	<code>nn.Module</code> 상속 클래스	<code>forward()</code> 메서드 구현 필요
4. 손실/옵티마이저	손실함수 & 업데이트 정의	<code>nn.CrossEntropyLoss()</code> , <code>optim.Adam()</code>	매 epoch마다 손실 계산 & 역전파
5. 학습 루프	<code>forward</code> → <code>loss</code> → <code>backward</code> → <code>update</code>	<code>model.train()</code> , <code>loss.backward()</code> , <code>optimizer.step()</code>	명시적 학습 루프
6. 저장	모델 or 가중치 저장	<code>torch.save()</code>	<code>state_dict</code> 또는 전체 모델 저장
7. 불러오기	저장된 모델 로드	<code>torch.load()</code> + <code>model.load_state_dict()</code>	재현 가능한 학습 이어받기
8. 평가	성능 확인	<code>model.eval()</code> + <code>with torch.no_grad()</code>	학습 꺼두고 성능 측정
9. 예측	추론 결과 반환	<code>model(input)</code>	Tensor → label 변환 필요시 <code>.argmax()</code>

II. Deep Learning **PyTorch**

항목	Keras	TensorFlow (Low-level)	PyTorch
데이터 로딩	NumPy / keras.datasets	tf.data.Dataset	torch.utils.data.Dataset
모델 정의	Sequential, Functional	Model 상속 / subclassing	nn.Module 상속 + forward()
학습	.fit() 자동화	GradientTape() 수동	수동 학습 루프 (backward())
손실 함수	문자열 지정 가능	tf.keras.losses	torch.nn.CrossEntropyLoss 등
옵티마이저	문자열 or 객체	tf.keras.optimizers	torch.optim
저장	.save()	saved_model.save()	torch.save()
추론	.predict()	model(input)	model(input)
평가	.evaluate()	test_step()	수동 구현 + .eval() + no_grad()

II. Deep Learning PyTorch

구분	코드	설명
1. 텐서 변환	<code>torch.tensor(data, dtype=...)</code>	NumPy 데이터를 PyTorch 텐서로 변환. 학습을 위해 float32나 long 타입을 명시해야 합니다.
2. 신경망 정의	<code>nn.Module</code> <code>super(...).__init__()</code>	사용자 정의 모델 클래스는 반드시 <code>nn.Module</code> 을 상속받아야 합니다. 부모 클래스 초기화로, 네트워크 계층 정의 전에 필수입니다.
3. 계층 정의	<code>nn.Linear(in, out)</code> <code>nn.Sigmoid()</code> <code>nn.Softmax(dim=1)</code>	입력 차원을 받아 선형 계층을 생성합니다. 예: <code>fc1 = nn.Linear(4, 50)</code> 시그모이드 활성화 함수 정의 다중 클래스 분류를 위한 소프트맥스 함수 정의. <code>dim=1</code> 은 각 샘플별 확률 계산
4. 손실 함수	<code>nn.CrossEntropyLoss()</code>	분류 문제용 손실 함수.
5. 최적화기	<code>optim.Adam(params, lr=...)</code>	Adam 최적화기 초기화. <code>model.parameters()</code> 로 파라미터 전달
6. 순전파	<code>model(inputs)</code>	모델의 <code>forward()</code> 함수를 호출하여 예측값을 반환
7. 손실 계산	<code>criterion(outputs, labels)</code>	예측값과 정답 사이의 손실을 계산
8. 역전파	<code>loss.backward()</code>	손실을 기준으로 모든 파라미터의 그래디언트를 계산
9. 그래디언트 초기화	<code>optimizer.zero_grad()</code>	학습 시 이전 그래디언트를 누적 방지 위해 초기화
10. 가중치 업데이트	<code>optimizer.step()</code>	계산된 그래디언트를 바탕으로 파라미터 업데이트 수행
11. 평가 모드	<code>torch.no_grad()</code>	추론(평가) 시 그래디언트 계산 비활성화로 메모리 절약
12. 예측 결과	<code>torch.max(outputs, 1)</code>	소프트맥스 결과에서 가장 높은 확률의 클래스 인덱스를 반환
13. 정확도 계산	<code>(pred == label).float().mean()</code>	예측값과 실제값이 같은 비율을 평균으로 계산

II. Deep Learning PyTorch

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from sklearn import datasets
5 from sklearn.model_selection import train_test_split
6 from torch.utils.data import DataLoader, TensorDataset
7
8 # 모델 정의
9 class IrisModel(nn.Module):
10     def __init__(self):
11         super(IrisModel, self).__init__()
12         self.fc1 = nn.Linear(4, 50) # 입력 4, 출력 50
13         self.fc2 = nn.Linear(50, 30) # 입력 50, 출력 30
14         self.fc3 = nn.Linear(30, 3) # 입력 30, 출력 3 (softmax로 변환)
15         self.sigmoid = nn.Sigmoid()
16         self.softmax = nn.Softmax(dim=1)
17
18     def forward(self, x):
19         x = self.sigmoid(self.fc1(x))
20         x = self.sigmoid(self.fc2(x))
21         return self.softmax(self.fc3(x))
```

II. Deep Learning **PyTorch**

```
class IrisModel(nn.Module):  
    def __init__(self):
```

PyTorch에서 신경망 모델을 만들기 위해 상속해야 하는 기본 클래스가 torch.nn.Module

이 클래스는 모델의 레이어 정의, 순전파(forward) 정의, 파라미터 추적, 저장/불러오기 등의 기능을 제공합니다. 즉, IrisModel은 nn.Module을 확장한 사용자 정의 모델

```
    super(IrisModel, self).__init__()
```

부모 클래스인 nn.Module의 초기화 메서드를 호출
이것을 호출해야 PyTorch가 이 클래스가 Module임을 인식하고,
내부에서 레이어와 파라미터를 관리

II. Deep Learning PyTorch

```
1 def compute_accuracy(model, X, y):
2     model.eval() # 평가 모드로 설정 (Dropout, BatchNorm 등 off)
3     with torch.no_grad(): # 그래디언트 계산 비활성화 (속도 ↑)
4         outputs = model(X)
5         predicted = torch.argmax(outputs, dim=1)
6         correct = (predicted == y).sum().item()
7         accuracy = correct / len(y)
8     model.train() # 다시 학습 모드로 전환
9     return accuracy
```

II. Deep Learning **PyTorch**

```
23 # 데이터 로드
24 iris = datasets.load_iris()
25 X, y = iris.data, iris.target
26
27 # Train/test split
28 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
29
30 # 데이터를 텐서로 변환
31 X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
32 y_train_tensor = torch.tensor(y_train, dtype=torch.long)
33 X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
34 y_test_tensor = torch.tensor(y_test, dtype=torch.long)
35
36 # DataLoader 생성
37 train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
38 train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
```

II. Deep Learning **PyTorch**

```
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)  
y_train_tensor = torch.tensor(y_train, dtype=torch.long)
```

torch.long 64비트 정수형 자료형, 즉 **int64**를 의미
PyTorch에서 텐서를 만들 때 사용할 수 있는 자료형(dtype) 중 하나

torch.int64와 동일한 의미이며, 클래스 내부적으로는 같은 타입.

PyTorch의 nn.CrossEntropyLoss() 함수는 타겟(label) 텐서가 정수형 클래스 인덱스여야 합니다.
예: **클래스가 3개일 경우 정답 라벨은 [0, 1, 2] 형식이어야** 하며,

float형 또는 one-hot 형식이면 오류 발생.

II. Deep Learning **PyTorch**

```
1 model = IrisModel()
2 criterion = nn.CrossEntropyLoss()
3 optimizer = optim.Adam(model.parameters(), lr=0.001)
4
5 num_epochs = 100
6 for epoch in range(num_epochs):
7     for inputs, labels in train_loader:
8         outputs = model(inputs)
9         loss = criterion(outputs, labels)
10
11         optimizer.zero_grad()
12         loss.backward()
13         optimizer.step()
14
15     # 학습 정확도 평가
16     train_acc = compute_accuracy(model, X_train_tensor, y_train_tensor)
17     print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}, Accuracy: {train_acc:.4f}")
18
```

```
Epoch [95/100], Loss: 0.6273, Accuracy: 0.9619
Epoch [96/100], Loss: 0.6330, Accuracy: 0.9619
Epoch [97/100], Loss: 0.6079, Accuracy: 0.9619
Epoch [98/100], Loss: 0.6151, Accuracy: 0.9619
Epoch [99/100], Loss: 0.6053, Accuracy: 0.9619
Epoch [100/100], Loss: 0.6096, Accuracy: 0.9619
```

II. Deep Learning PyTorch

nn.CrossEntropyLoss() = Sparse Categorical Cross Entropy

손실 함수 `nn.CrossEntropyLoss()` 는 정수형 라벨 인덱스 (e.g. 0, 1, 2) 를 입력으로 받습니다. 즉, 라벨을 one-hot encoding 하지 않고도 사용할 수 있습니다.

따라서:Keras의 `sparse_categorical_crossentropy`와 같은 방식입니다. PyTorch에서는 label이 정수형이면 `CrossEntropyLoss`가 적절합니다.

항목	의미
categorical crossentropy	라벨이 one-hot 벡터 (예: <code>[0, 0, 1]</code>) 형태
sparse categorical crossentropy	라벨이 정수 인덱스 (예: <code>2</code>) 형태

II. Deep Learning **PyTorch**

```
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
```

이 객체를 통해 (input, **label**) 쌍이 구성됩니다.

```
for inputs, labels in train_loader:
```

...

여기서 inputs는 배치된 입력 데이터 (x), **labels**은 배치된 정답 레이블 (y)입니다.

```
optimizer.zero_grad()
```

```
loss.backward()
```

```
optimizer.step()
```

optimizer.zero_grad()는 필수적인 그래디언트 초기화 작업

이전 미니배치에서 계산된 gradient가 여전히 남아 있음

loss.backward()가 호출되면 그 위에 새로운 gradient가 누적됨따라서,
매 학습 반복마다 수동으로 gradient를 초기화해주어야 함

II. Deep Learning

```
60 # 평가
61 with torch.no_grad():
62     outputs = model(X_test_tensor)
63     _, predicted = torch.max(outputs, 1)
64     accuracy = (predicted == y_test_tensor).float().mean()
65     print(f'Test Accuracy: {accuracy.item() * 100:.2f}%')
```

```
Epoch [98/100], Loss: 0.6200
Epoch [99/100], Loss: 0.6095
Epoch [100/100], Loss: 0.6255
Test Accuracy: 100.00%
```

II. Deep Learning (Pytorch with MNIST)

CNN with MNIST

(Pytorch version)

GPU용 PyTorch 재설치 (RTX 4060 최적화)

현재 가상환경(0209vision)에 설치된 PyTorch를 지우고,
CUDA 12.4를 지원하는 버전으로 다시 설치

```
pip uninstall torch torchvision torchaudio -y
```

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu124 --trusted-host download.pytorch.org --trusted-host pypi.org --trusted-host files.pythonhosted.org
```


II. Deep Learning (Pytorch with MNIST)

```
import torch

# 1. CUDA 사용 가능 여부 확인
print(f"CUDA Available: {torch.cuda.is_available()}")

# 2. 사용 가능한 GPU 개수
print(f"GPU Count: {torch.cuda.device_count()}")

# 3. 현재 GPU 이름 (RTX 4060이 나와야 함)
if torch.cuda.is_available():
    print(f"GPU Name: {torch.cuda.get_device_name(0)}")

# 4. 장치 설정
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

II. Deep Learning (Pytorch with MNIST)

```
[5] 1 # GPU 사용 여부 확인  
    2 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')  
    3 print(f'Using device: {device}')
```

↪ Using device: cuda

✓ 데이터 로딩 데이터 전처리

```
[1] 1 import torch  
    2 import torch.nn as nn  
    3 import torch.optim as optim  
    4 import torchvision  
    5 from torchvision import datasets, transforms  
    6 from torch.utils.data import DataLoader  
    7 import matplotlib.pyplot as plt
```

```
from PIL import Image  
import numpy as np
```

II. Deep Learning (Pytorch with MNIST)

✓ Dataset 과 DataLoader 만들기

```
[2] 1 # 1. 데이터 로딩 및 전처리
    2 transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])
    3
    4 train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
    5 test_dataset = datasets.MNIST(root='./data', train=False, download=True, transform=transform)
    6
    7 train_loader = DataLoader(dataset=train_dataset, batch_size=64, shuffle=True)
    8 test_loader = DataLoader(dataset=test_dataset, batch_size=64, shuffle=False)
```

II. Deep Learning (Pytorch with MNIST)

정규화시 데이터의 범위를 표준 정규분포로 변환하는 과정

1. 입력하는 값 평균/표준편차

학습 데이터셋의 평균(Mean): 각 채널(R, G, B)별 픽셀 값의 산술 평균.

학습 데이터셋의 표준편차(Std): 각 채널별 픽셀 값이 평균으로부터 떨어진 정도를 나타내는 표준편차

2. 수치 입력 방식 (두 가지 케이스)

① 일반적인 외부 데이터셋 (예: ImageNet)

가장 많이 쓰이는 값은 수백만 장의 이미지를 가진 ImageNet 데이터셋의 통계치
직접 계산하기 어려운 경우 이 값을 관례적으로 사용합니다.

- Mean: [0.485, 0.456, 0.406]
- Std: [0.229, 0.224, 0.225]

② 본인의 고유 데이터셋 (예: 의료 영상 PET, 특수 사물)

데이터셋이 일반적인 사진과 크게 다르다면(예: 흑백 위주의 의료 영상), 직접 계산한 값을 넣어야 성능이 올라갑니다.

계산 방법: 모든 이미지의 R, G, B 채널 값을 각각 합산한 뒤, 전체 픽셀 수로 나누어
평균과 표준편차를 구합니다.

II. Deep Learning (Pytorch with MNIST)

✓ 모델 정의



```
1 # 2. CNN 모델 정의
2 class CNN(nn.Module):
3     def __init__(self):
4         super(CNN, self).__init__()
5         self.conv1 = nn.Conv2d(1, 32, kernel_size=3)
6         self.conv2 = nn.Conv2d(32, 64, kernel_size=3)
7         self.pool = nn.MaxPool2d(2, 2)
8         self.fc1 = nn.Linear(64*5*5, 128)
9         self.fc2 = nn.Linear(128, 10)
10
11     def forward(self, x):
12         x = self.pool(torch.relu(self.conv1(x)))
13         x = self.pool(torch.relu(self.conv2(x)))
14         x = x.view(x.size(0), -1) # Flatten
15         x = torch.relu(self.fc1(x))
16         x = self.fc2(x)
17         return x
```



```
1 # 3. 모델, 손실 함수 및 옵티마이저 정의
2 model = CNN().to(device) # 모델을 gpu 실행
3 loss_fn = nn.CrossEntropyLoss()
4 optimizer = optim.Adam(model.parameters(), lr=0.001) # lr=1-e3
```

II. Deep Learning (Pytorch with MNIST)

✓ 학습과 평가에 필요한 루틴함수 만들기

```
1 # 4. 학습 함수
2 def train(model, train_loader, optimizer, loss_fn, device):
3     model.train()
4     running_loss = 0.0
5     correct = 0
6     total = 0
7     for images, labels in train_loader:
8         images, labels = images.to(device), labels.to(device)
9
10        # Forward pass
11        outputs = model(images)
12        loss = loss_fn(outputs, labels)
13
14        # Backward pass and optimization
15        optimizer.zero_grad()
16        loss.backward()
17        optimizer.step()
18
19        # Loss 및 Accuracy 계산
20        running_loss += loss.item()
21        _, predicted = torch.max(outputs, 1)
22        correct += (predicted == labels).sum().item()
23        total += labels.size(0)
24
25    avg_loss = running_loss / len(train_loader)
26    accuracy = correct / total * 100
27    return avg_loss, accuracy
```

II. Deep Learning (Pytorch with MNIST)

```
[8] 1 # 5. 평가 함수
    2 def evaluate(model, test_loader, loss_fn, device):
    3     model.eval()
    4     test_loss = 0.0
    5     correct = 0
    6     total = 0
    7     with torch.no_grad():
    8         for images, labels in test_loader:
    9             images, labels = images.to(device), labels.to(device)
   10
   11             outputs = model(images)
   12             loss = loss_fn(outputs, labels)
   13             test_loss += loss.item()
   14
   15             _, predicted = torch.max(outputs, 1)
   16             correct += (predicted == labels).sum().item()
   17             total += labels.size(0)
   18
   19     avg_loss = test_loss / len(test_loader)
   20     accuracy = correct / total * 100
   21     return avg_loss, accuracy
   22
```

II. Deep Learning (Pytorch with MNIST)

✓ 학습 및 평가 수행

가장 좋은 테스트 정확도를 기록할 때 모델을 저장

1회 이후 이전 결과와 비교하여 성능 개선시 해당 모델을 저장한다

```
[9] 1 # 6. 학습 및 평가
    2 num_epochs = 10
    3 train_losses, train_accuracies = [], []
    4 test_losses, test_accuracies = [], []
    5
    6 best_test_acc = 0.0 # 가장 좋은 테스트 정확도를 기록하기 위한 변수 (이 부분을 학습 루프 외부에 정의)
    7
    8 for epoch in range(num_epochs):
    9     train_loss, train_acc = train(model, train_loader, optimizer, loss_fn, device)
   10     test_loss, test_acc = evaluate(model, test_loader, loss_fn, device)
   11
   12     train_losses.append(train_loss)
   13     train_accuracies.append(train_acc)
   14     test_losses.append(test_loss)
   15     test_accuracies.append(test_acc)
   16
   17     print(f'Epoch [{epoch+1}/{num_epochs}], Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}%, '
   18           f'Test Loss: {test_loss:.4f}, Test Acc: {test_acc:.2f}%')
   19
   20     # 가장 좋은 테스트 정확도를 기록할 때 모델을 저장()
   21     if test_acc > best_test_acc:
   22         best_test_acc = test_acc
   23         torch.save(model.state_dict(), f'best_model_epoch_{epoch+1}.pth') # 모델 저장
   24         print(f"Best model saved with Test Accuracy: {test_acc:.2f}% at Epoch {epoch+1}")
```

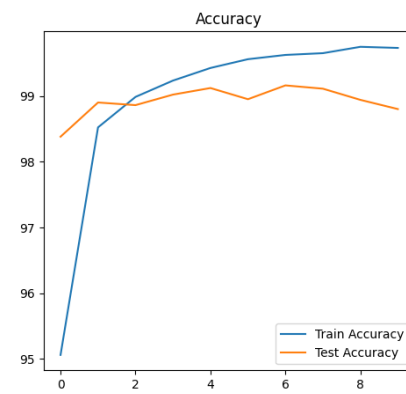
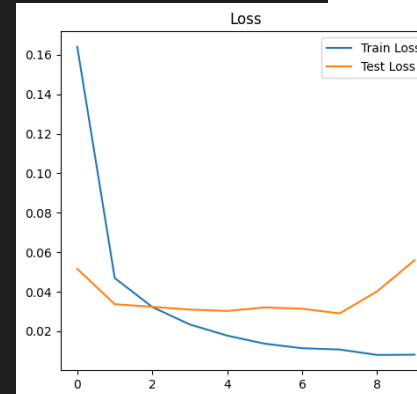

II. Deep Learning (Pytorch with MNIST)

 best_model_epoch_2.pth	 Epoch [1/10], Train Loss: 0.1641, Train Acc: 95.06%, Test Loss: 0.0516, Test Acc: 98.38% Best model saved with Test Accuracy: 98.38% at Epoch 1
 best_model_epoch_3.pth	Epoch [2/10], Train Loss: 0.0469, Train Acc: 98.52%, Test Loss: 0.0337, Test Acc: 98.90% Best model saved with Test Accuracy: 98.90% at Epoch 2
 best_model_epoch_4.pth	Epoch [3/10], Train Loss: 0.0323, Train Acc: 98.98%, Test Loss: 0.0324, Test Acc: 98.86%
 best_model_epoch_5.pth	Epoch [4/10], Train Loss: 0.0234, Train Acc: 99.23%, Test Loss: 0.0310, Test Acc: 99.02% Best model saved with Test Accuracy: 99.02% at Epoch 4
 best_model_epoch_6.pth	Epoch [5/10], Train Loss: 0.0178, Train Acc: 99.42%, Test Loss: 0.0302, Test Acc: 99.12% Best model saved with Test Accuracy: 99.12% at Epoch 5
 best_model_epoch_7.pth	Epoch [6/10], Train Loss: 0.0137, Train Acc: 99.56%, Test Loss: 0.0321, Test Acc: 98.95%
 best_model_epoch_8.pth	Epoch [7/10], Train Loss: 0.0114, Train Acc: 99.62%, Test Loss: 0.0314, Test Acc: 99.16% Best model saved with Test Accuracy: 99.16% at Epoch 7
 best_model_epoch_9.pth	Epoch [8/10], Train Loss: 0.0107, Train Acc: 99.65%, Test Loss: 0.0290, Test Acc: 99.11%
	Epoch [9/10], Train Loss: 0.0080, Train Acc: 99.75%, Test Loss: 0.0402, Test Acc: 98.94%
	Epoch [10/10], Train Loss: 0.0081, Train Acc: 99.73%, Test Loss: 0.0560, Test Acc: 98.80%

II. Deep Learning (Pytorch with MNIST)

✓ 모델 성능 평가 시각화

```
1 # 7. 성능 평가 결과 시각화
2 plt.figure(figsize=(12, 5))
3 plt.subplot(1, 2, 1)
4 plt.plot(train_losses, label='Train Loss')
5 plt.plot(test_losses, label='Test Loss')
6 plt.title('Loss')
7 plt.legend()
8
9 plt.subplot(1, 2, 2)
10 plt.plot(train_accuracies, label='Train Accuracy')
11 plt.plot(test_accuracies, label='Test Accuracy')
12 plt.title('Accuracy')
13 plt.legend()
14
15 plt.show()
```



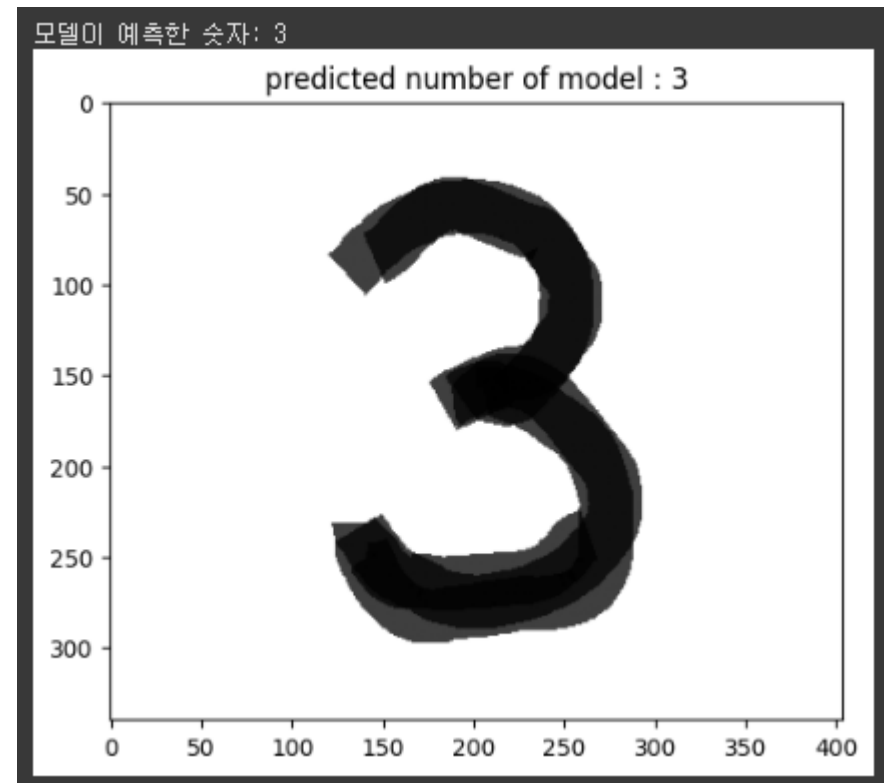
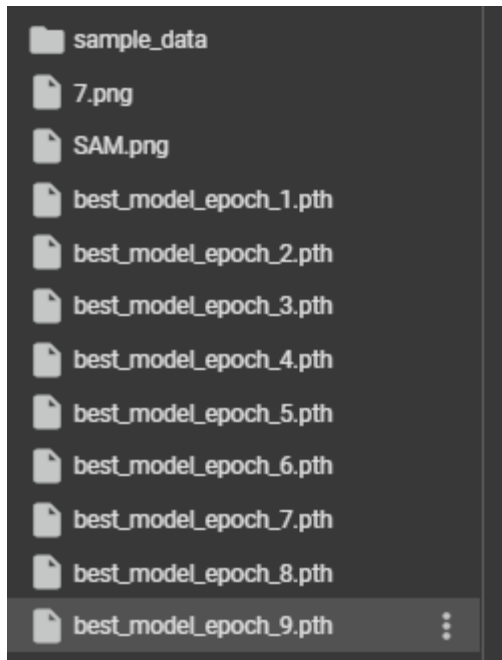
II. Deep Learning (Pytorch with MNIST)

```
1 # 새로운 모델 인스턴스 생성
2 model = CNN()
3
4 # 저장된 모델 가중치만 로딩 (weights_only=True 추가)
5 # 저장된 모델 파일 경로를 적절히 수정
6 model.load_state_dict(torch.load('best_model_epoch_9.pth', weights_only=True))
7 model.eval()
8
9 # 이미지 전처리 함수 정의 (배경이 흰색이면 반전 필요)
10 def preprocess_image(image_path):
11     # 이미지 로드 및 흑백으로 변환
12     image = Image.open(image_path).convert('L') # 'L'은 그레이스케일 모드
13
14     # 이미지 전처리 함수 정의 (배경이 흰색이면, 검은색으로 반전, 숫자는 흰색)
15     image = np.array(image)
16     image = Image.fromarray(255 - image)
17
18     # MNIST 데이터와 같은 크기로 조정 (28x28)
19     transform = transforms.Compose([
20         transforms.Resize((28, 28)),
21         transforms.ToTensor(),
22         transforms.Normalize((0.5,), (0.5,)) # MNIST와 비슷하게 정규화
23     ])
24
25     # 이미지 텐서 변환 후 배치 차원 추가 (1, 28, 28)
26     image = transform(image).unsqueeze(0)
27     return image
```

II. Deep Learning (Pytorch with MNIST)

```
29 📁 손글씨 이미지 경로
30 image_path = '/content/SAM.png' # 손글씨 이미지 파일 경로로 수정
31
32 # 이미지 전처리 및 예측
33 image = preprocess_image(image_path)
34 with torch.no_grad():
35     output = model(image)
36     _, predicted = torch.max(output, 1)
37     print(f"모델이 예측한 숫자: {predicted.item()}")
38
39 # 이미지 시각화
40 plt.imshow(Image.open(image_path).convert('L'), cmap='gray')
41 plt.title(f"predicted number of model : {predicted.item()}")
42 plt.show()
```

II. Deep Learning (Pytorch with MNIST)



II. Deep Learning **TF vs Torch vs Keras**

목적	TensorFlow (Low-level)	PyTorch	Keras (High-level)
라이브러리 임포트	<code>import tensorflow as tf</code>	<code>import torch</code> <code>import torch.nn as nn</code>	<code>from tensorflow.keras import Sequential</code> <code>from tensorflow.keras.layers import Dense</code>
가중치 초기화	<code>tf.Variable(tf.random.normal(...))</code>	<code>nn.Linear(input_dim, output_dim)</code>	<code>Dense(units, activation='...')</code>
모델 구조 정의	<code>class Model:</code> <code>def __call__()</code>	<code>class Model(nn.Module):</code> <code>def forward()</code>	<code>Sequential([...])</code>
활성화 함수	<code>tf.nn.sigmoid,</code> <code>tf.nn.softmax</code>	<code>nn.Sigmoid(),</code> <code>nn.Softmax(dim=1)</code>	<code>activation='sigmoid', 'softmax'</code>
순전파 호출	<code>model(inputs) → __call__()</code>	<code>model(inputs) → forward()</code>	내부 자동 처리
손실 함수	<code>tf.losses.categorical_crossentropy + tf.one_hot</code>	<code>nn.CrossEntropyLoss()</code> (정수형 label 사용)	<code>loss='sparse_categorical_crossentropy'</code>
옵티마이저	<code>tf.optimizers.Adam(learning_rate=0.001)</code>	<code>torch.optim.Adam(model.parameters(), lr=0.001)</code>	<code>optimizer='adam'</code>
그래디언트 계산	<code>with tf.GradientTape() as tape:tape.gradient(...)</code>	<code>loss.backward(),</code> <code>optimizer.step()</code>	자동 처리
그래디언트 초기화	자동 (GradientTape 관리)	<code>optimizer.zero_grad()</code>	자동 처리
데이터 배치 처리	수동 슬라이싱 (<code>X[i:i+batch]</code>)	<code>DataLoader(dataset, batch_size, shuffle=True)</code>	내부에서 자동 배치 (fit 사용)
정확도 계산	<code>tf.argmax,</code> <code>tf.equal,</code> <code>tf.reduce_mean</code>	<code>torch.max,</code> <code>==,</code> <code>.mean()</code>	<code>metrics=['accuracy']</code>
학습 루프	<code>for epoch, for batch</code> (수동 반복)	<code>for epoch, for batch in DataLoader</code>	<code>model.fit(...)</code> 한 줄
평가	정확도 함수 직접 구현	<code>with torch.no_grad()</code> 후 수동 계산	<code>model.evaluate(...)</code>

II. Deep Learning **TF vs Torch vs Keras**

실습) Pytorch의 주요 메서드를 정리하고 Keras, Tensorflow와 비교하세요

II. Deep Learning **LeNet**

LeNet-5
(1998)

II. Deep Learning **LeNet**

등장 배경:

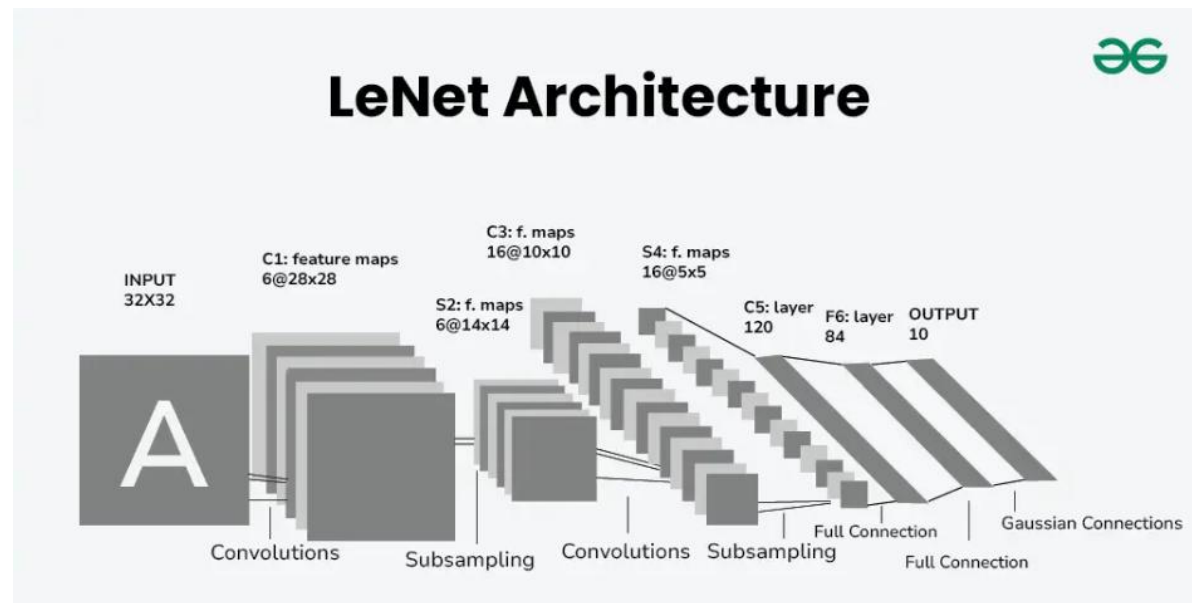
손글씨 숫자 인식 (MNIST) 문제 해결을 위해 개발됨. 당시로서는 혁신적인 구조지만, 컴퓨팅 파워 부족과 대량 데이터 부족으로 제한적 사용.

기술적 특이점:

초기 **CNN 구조의 대표: Conv → Pool → FC 구조 확립**

Activation 함수로 tanh 사용

이미지에서 특징을 추출하고 분류하는 계층 구조 개념 정립



II. Deep Learning **AlexNet**

AlexNet

(2012, ImageNet 대회 우승)

II. Deep Learning **AlexNet**

등장 배경:

GPU의 발전, 대용량 데이터셋(ImageNet)의 등장

기존 전통적인 머신러닝 기법(SVM + hand-crafted features)을 압도

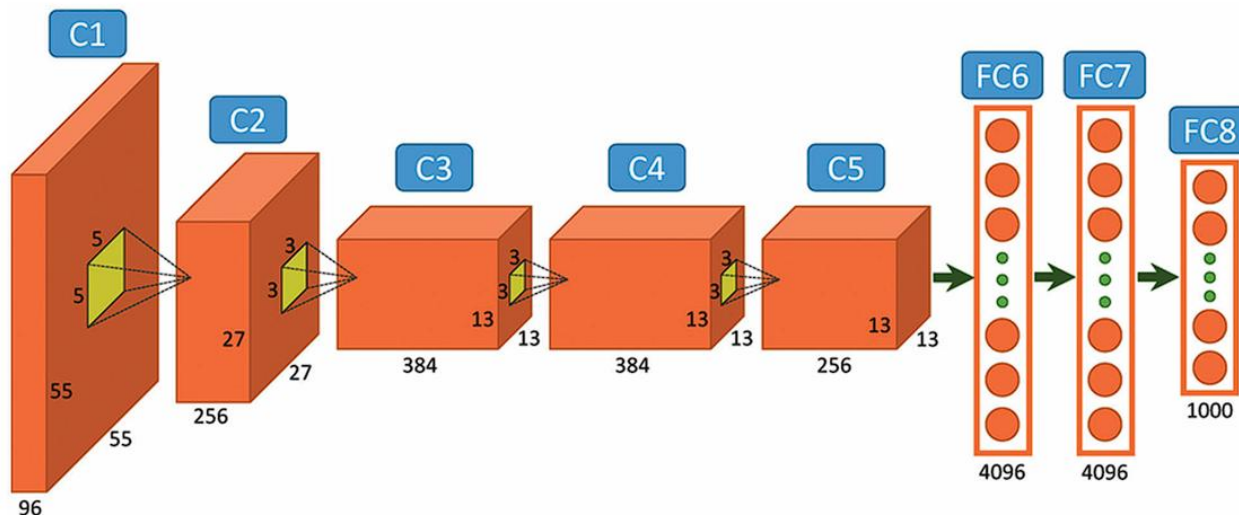
기술적 특이점:

ReLU 도입으로 학습 속도 대폭 향상

GPU 병렬 연산 (당시 GTX 580 두 개 사용)

Dropout 도입 → 과적합 방지

CNN이 비전 분야를 지배하는 시작점



II. Deep Learning **VGGNet**

VGGNet
(2014)

II. Deep Learning **VGGNet**

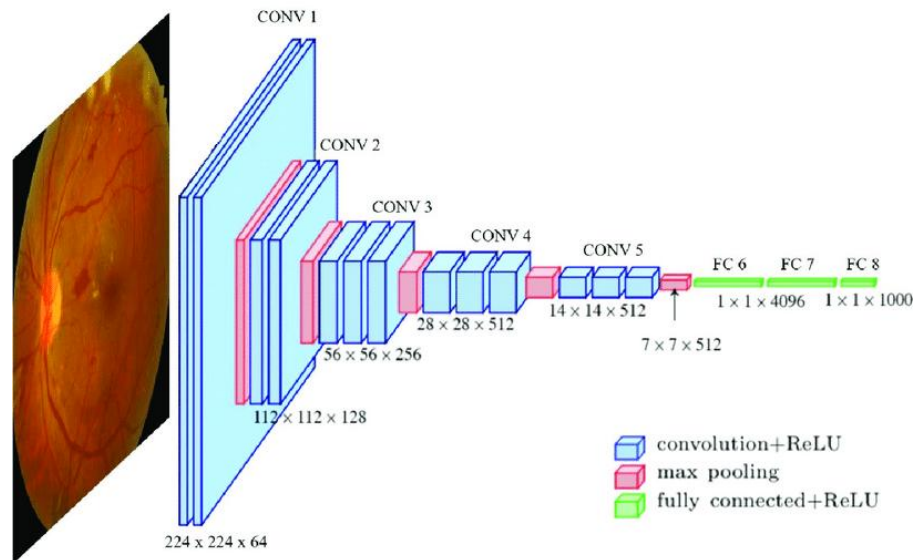
등장 배경:

AlexNet 이후, 단순하지만 **깊은 구조가 성능 향상에 중요하다**는 가설을 실험

기술적 특이점:

3x3 필터만 사용, **깊이 증가(16~19 layers)**

단순하고 반복적인 구조 → 하드웨어 최적화 용이
파라미터 수는 많지만, 구조가 정형화되어 모듈화에 용이



II. Deep Learning **Inception**

Inception
(GoogLeNet, 2014)

II. Deep Learning Inception

등장 배경:

VGG는 깊이 증가로 인한 연산량 증가 문제가 있었음

다양한 필터 크기를 병렬로 사용하는 아이디어로 정보 손실 방지 + 계산 효율화

기술적 특이점:

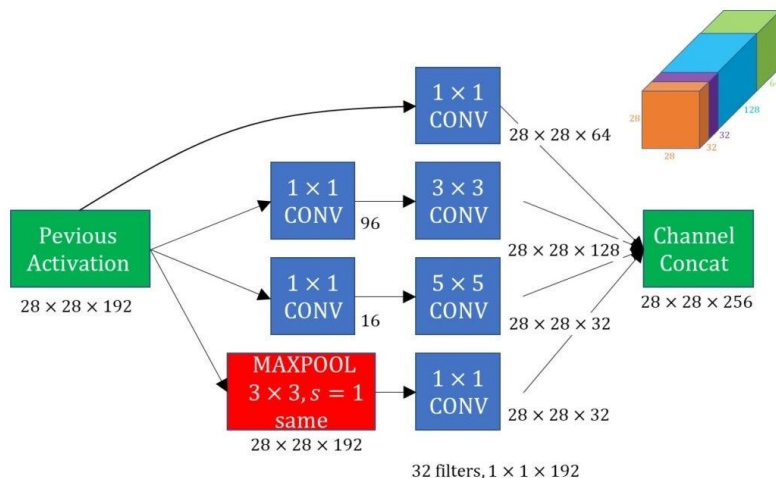
Inception 모듈: 1x1, 3x3, 5x5 컨볼루션을 병렬로 수행

연산량 줄이기 위해 1x1 conv로 차원 축소

Network-in-Network 개념 적용

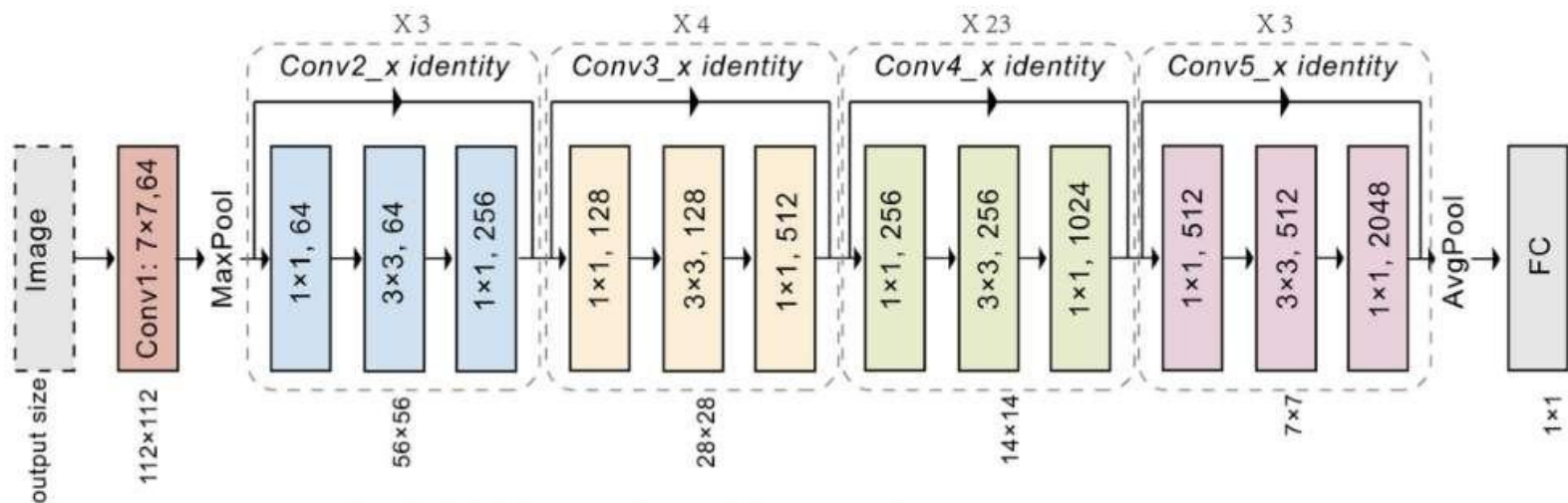
깊고 넓은 네트워크 구조 설계 가능

An example of an Inception module



II. Deep Learning **ResNet**

ResNet (Residual Network)



II. Deep Learning ResNet

2. ResNet 학습 (Residual Network)

2.1 ResNet의 등장 배경

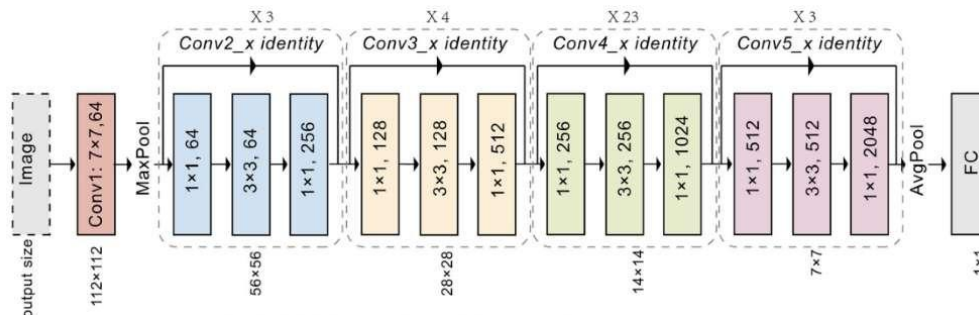
딥러닝의 깊이가 증가할수록 기울기 소실(Vanishing Gradient) 문제 발생
깊은 네트워크가 오히려 성능 저하 (degradation) 현상 발생
“이전보다 더 깊은 네트워크가 왜 더 못하냐?”는 문제의식

기술적 특이점:

Residual Learning 도입: $y=F(x)+x$

학습할 함수를 직접 예측하는 게 아니라 잔차(residual)를 학습

매우 깊은 구조(152 layers 이상)도 안정적으로 학습 가능
이후 모든 모델의 표준 설계 요소가 됨

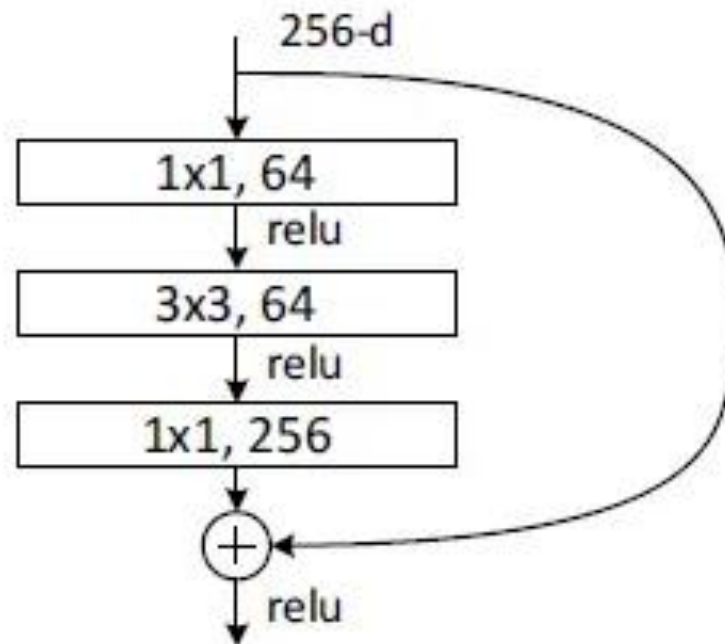


II. Deep Learning ResNet

2.2 ResNet 내부 구조 분석

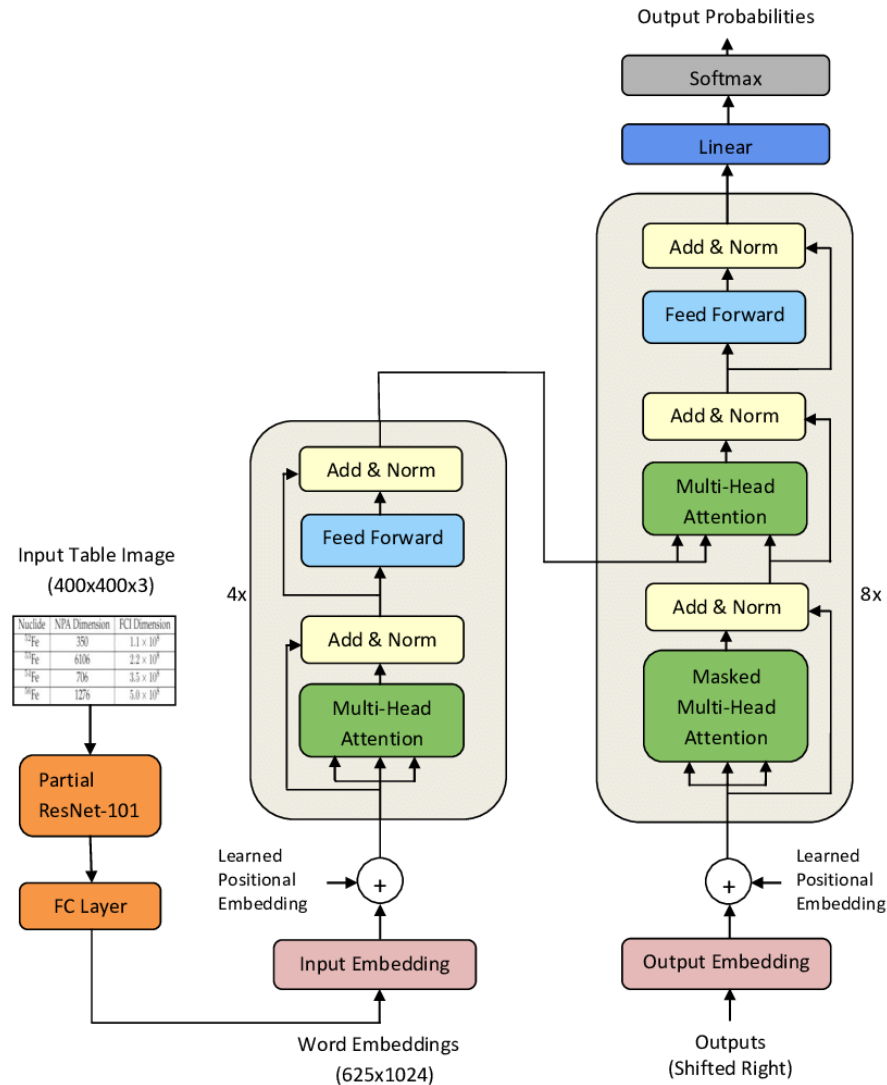
Residual Block의 동작 방식

$F(x) + x$ 형태의 스킵 연결 (Skip Connection)



II. Deep Learning ResNet

2.3 ResNet 응용(Transformer)



II. Deep Learning (ResNet)

[GET STARTED](#)[GUIDES](#)[API](#)[EXAMPLES](#)[KERAS TUNER](#)[KERAS RS](#)[KERAS HUB](#)

Keras 3 API documentation

[Models API](#)[Layers API](#)[Callbacks API](#)[Ops API](#)[Optimizers](#)[Metrics](#)[Losses](#)[Data loading](#)[Built-in small datasets](#)

ResNet and ResNetV2

ResNet50 function

[\[source\]](#)

```
keras.applications.ResNet50(  
    include_top=True,  
    weights="imagenet",  
    input_tensor=None,  
    input_shape=None,  
    pooling=None,  
    classes=1000,  
    classifier_activation="softmax",  
    name="resnet50",  
)
```

Instantiates the ResNet50 architecture.

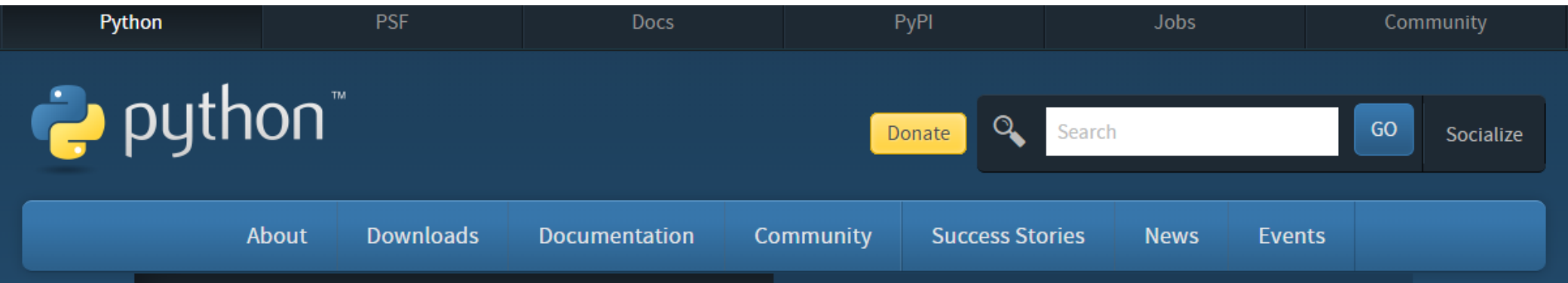
ResNet and ResNetV2

- [ResNet50 function](#)
- [ResNet101 function](#)
- [ResNet152 function](#)
- [ResNet50V2 function](#)
- [ResNet101V2 function](#)
- [ResNet152V2 function](#)

II. Deep Learning 가상환경&Visual Studio

가상환경 & Visual Studio

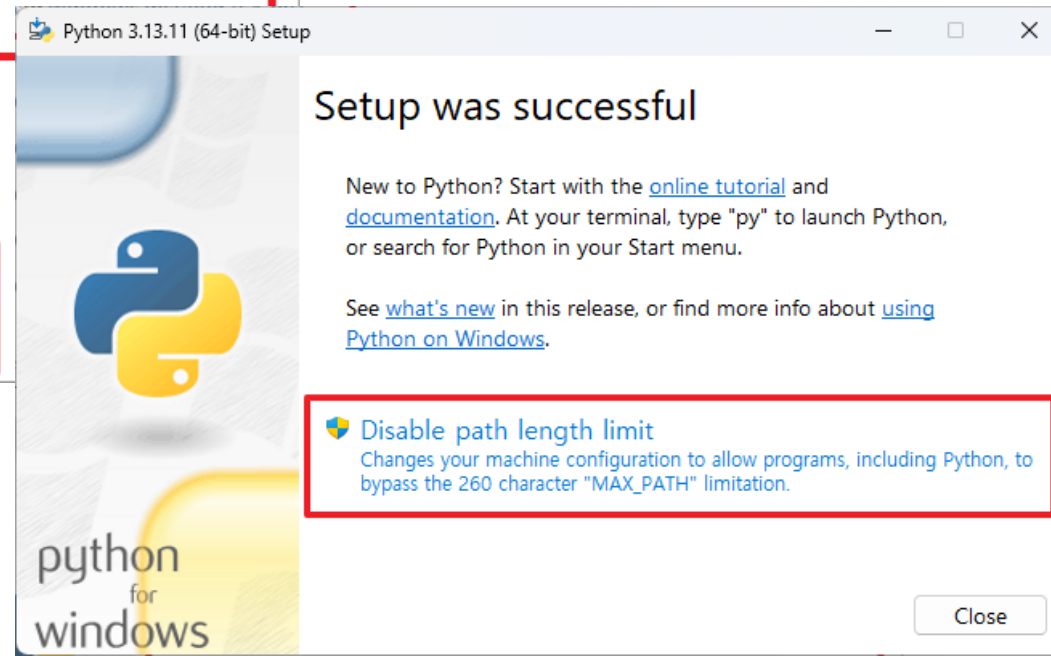
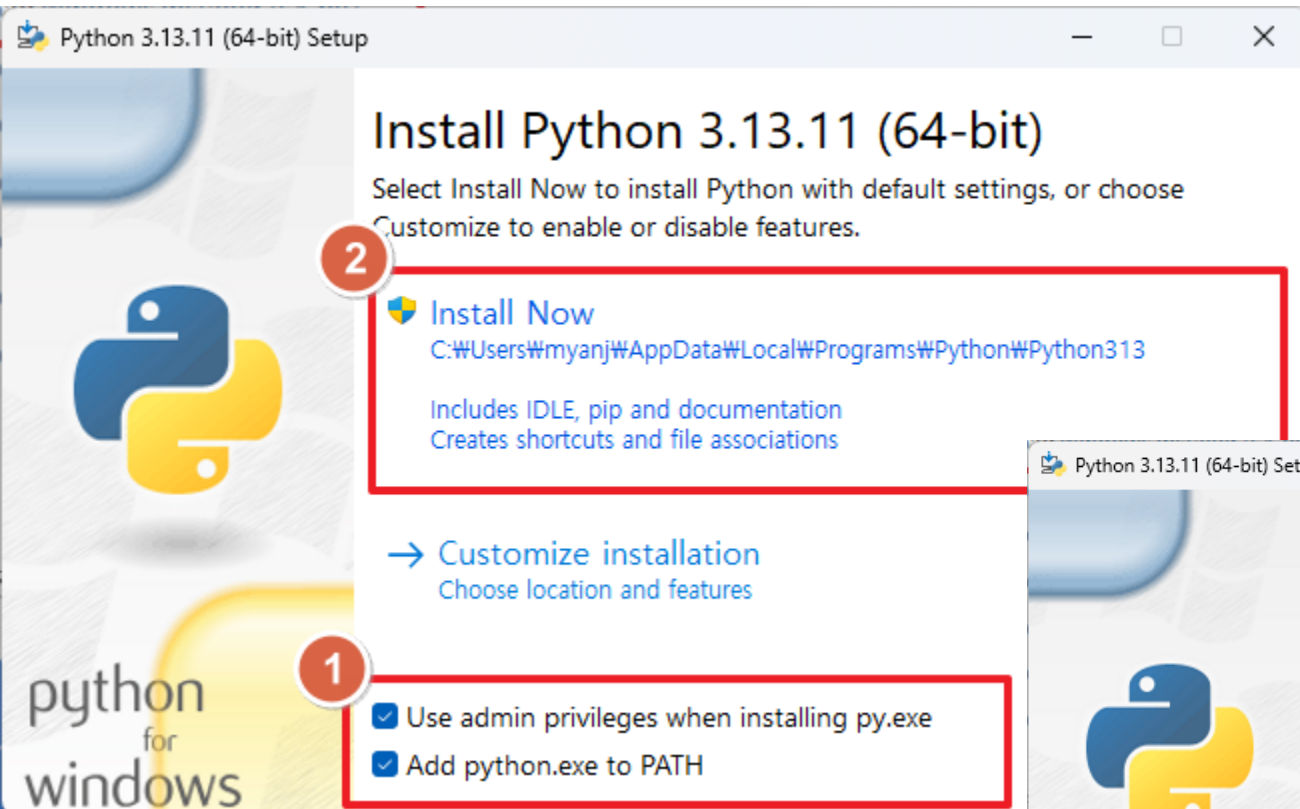
II. Deep Learning 가상환경&Visual Studio



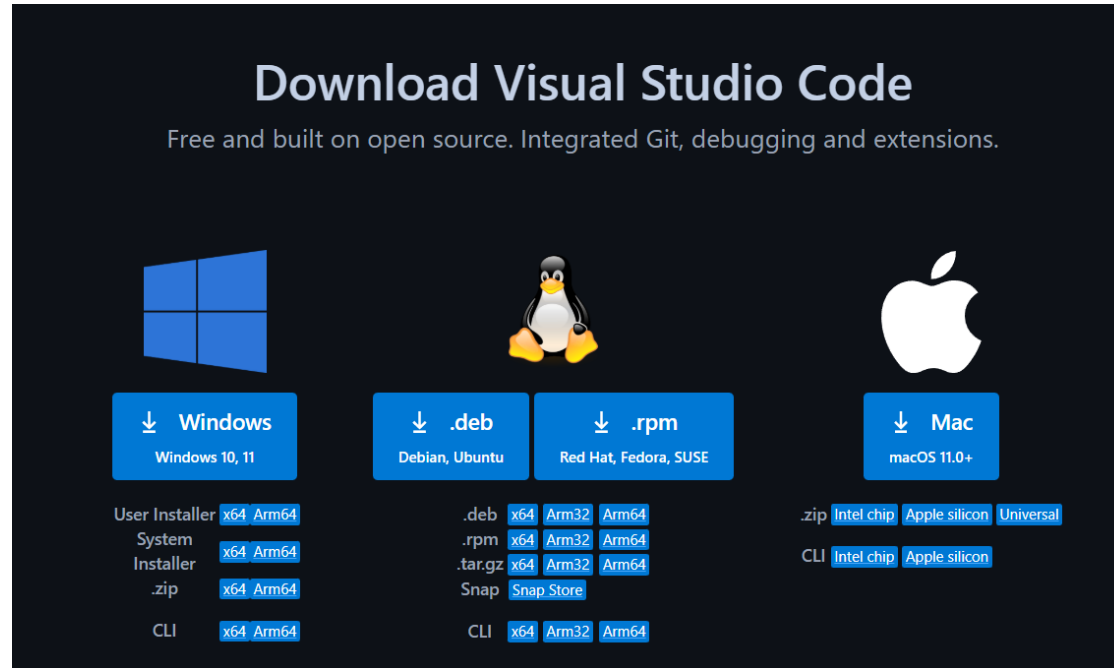
Note that Python 3.13.11 *cannot* be used on Windows 7 or earlier.

- [Download Windows installer \(64-bit\)](#)
- [Download Windows installer \(32-bit\)](#)
- [Download Windows installer \(ARM64\)](#)
- [Download Windows embeddable package \(64-bit\)](#)
- [Download Windows embeddable package \(32-bit\)](#)
- [Download Windows embeddable package \(ARM64\)](#)
- [Download Windows release manifest](#)

II. Deep Learning 가상환경&Visual Studio

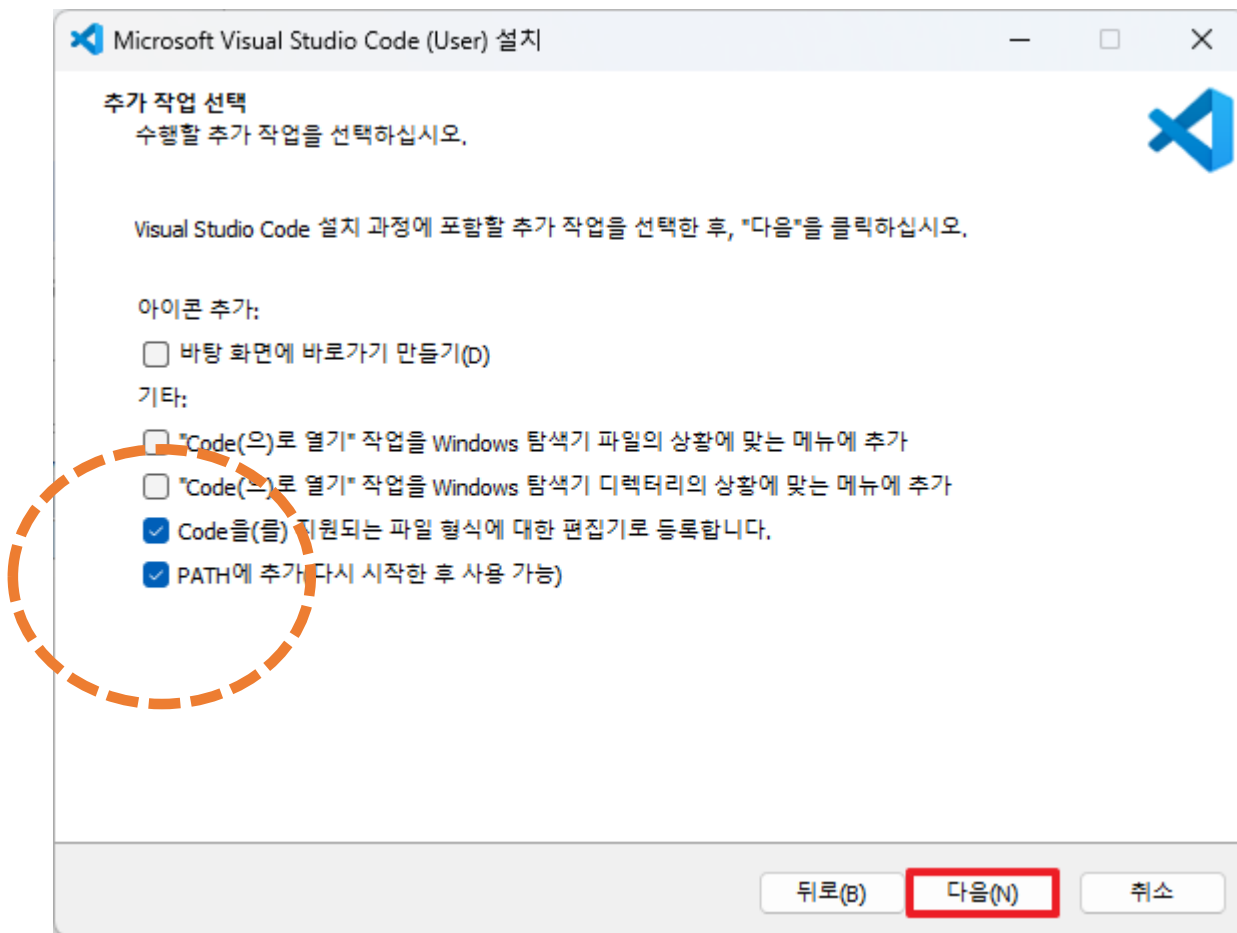


II. Deep Learning 가상환경&Visual Studio

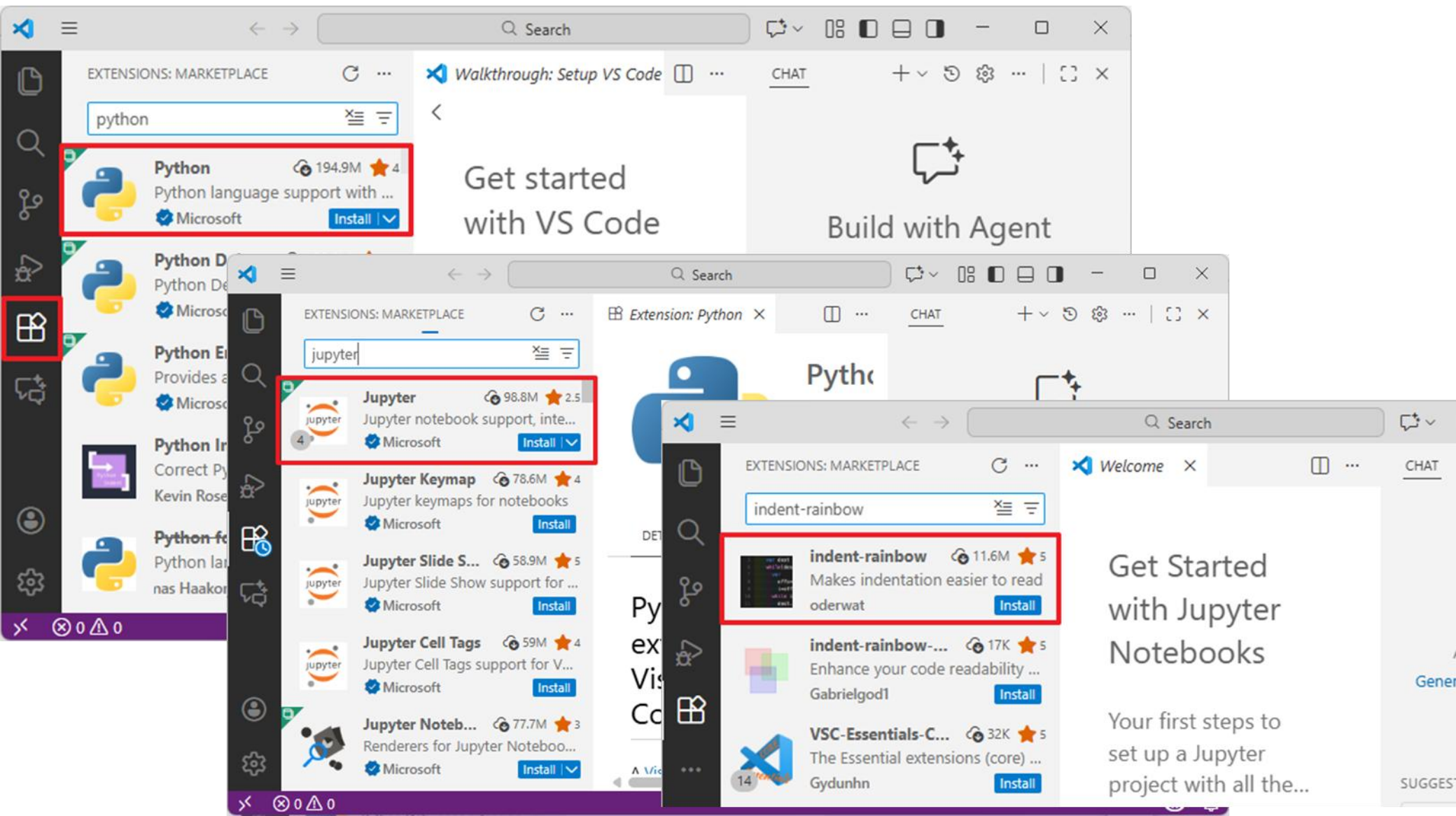


기본 설정 상태로 설치

II. Deep Learning 가상환경&Visual Studio



II. Deep Learning 가상환경&Visual Studio



II. Deep Learning 가상환경&Visual Studio

명령 프롬프트(CMD)

```
C:\Users\myanj> python --version  
Python 3.13.11
```

작업 디렉터리 생성 및 이동

```
C:\Users\myanj> mkdir c:\lab && cd c:\lab
```

```
c:\lab>
```

가상환경 생성 및 활성화

```
c:\lab> python -m venv venv  
~~~~~
```

모듈 가상환경 이름

```
c:\lab> venv\Scripts\activate
```

```
(venv) c:\lab>
```

~~~~~

활성화된 가상환경의 이름

## II. Deep Learning 가상환경&Visual Studio

---

### 명령 프롬프트(CMD)

```
C:\Users\myanj> python --version  
Python 3.13.11
```

작업 디렉터리 생성 및 이동

```
C:\Users\myanj> mkdir c:\lab && cd c:\lab
```

```
c:\lab>
```

가상환경 생성 및 활성화

```
c:\lab> python -m venv venv  
~~~~~
```

모듈 가상환경 이름

```
c:\lab> venv\Scripts\activate
```

```
(venv) c:\lab>
```

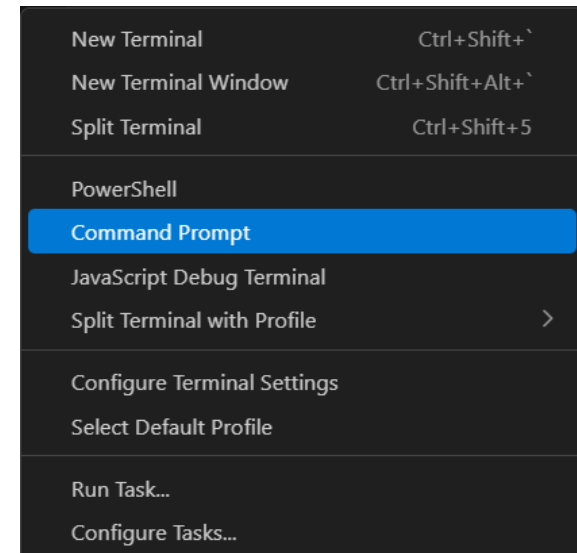
~~~~~

활성화된 가상환경의 이름

## II. Deep Learning 가상환경&Visual Studio

Cntrl + “ ` ” 로 프롬프트 탭을 활성화합니다.

가상환경을 확인합니다.



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Microsoft Windows [Version 10.0.19045.6466]  
(c) Microsoft Corporation. All rights reserved.

(venv) C:\lab>

+ v ... | [ ] x

powershell

cmd

## II. Deep Learning GPU/파이토치 환경설정

| 구분             | RTX 4060    | RTX 4070    | RTX 4080    | RTX 4090    |
|----------------|-------------|-------------|-------------|-------------|
| CUDA 코어 수      | 3,072       | 5,888       | 9,728       | 16,384      |
| VRAM (비디오 메모리) | 8GB GDDR6   | 12GB GDDR6X | 16GB GDDR6X | 24GB GDDR6X |
| 메모리 인터페이스      | 128-bit     | 192-bit     | 256-bit     | 384-bit     |
| 소비 전력 (TGP)    | 115W        | 200W        | 320W        | 450W        |
| 권장 파워(PSU)     | 550W        | 650W        | 750W        | 850W+       |
| 주요 타겟 해상도      | FHD (1080p) | QHD (1440p) | 4K 입문       | 4K 정복 / 작업용 |

## II. Deep Learning GPU/파이토치 환경설정

---

**GPU/파이토치 환경설정**

## II. Deep Learning GPU/파이토치 환경설정

### 1단계: 하드웨어 및 드라이버 인식 확인

윈도우가 내 GPU를 제대로 인식하고 드라이버가 잡혀있는지 확인

방법: Win + X 키를 누른 후 [장치 관리자] 선택 → [디스플레이 어댑터] 확인.

터미널 확인: VS Code 터미널에서 아래 명령어를 입력.

**nvidia-smi**

```
C:\Users\USER>nvidia-smi
Tue Jan 27 08:48:35 2026

+-----+
| NVIDIA-SMI 561.09 Driver Version: 561.09 CUDA Version: 12.6 |
+-----+-----+
| GPU Name Driver-Model Bus-Id Disp.A | Volatile Uncorr. ECC |
| Fan Temp Perf Pwr:Usage/Cap Memory-Usage | GPU-Util Compute M. |
|=====+=====+
| 0 NVIDIA GeForce RTX 4060 WDDM 00000000:01:00.0 On | N/A |
| 0% 50C P3 N/A / 115W 1212MiB / 8188MiB | 1% Default |
| MIG M. |
| N/A |
+-----+-----+

+-----+
| Processes: |
| GPU GI CI PID Type Process name GPU Memory |
| ID ID ID | Usage |
|=====+=====+
| 0 N/A N/A 2740 C+G ...2txyewy\StartMenuExperienceHost.exe N/A |
+-----+
```



## II. Deep Learning GPU/파이토치 환경설정

### 2단계: CUDA 지원 여부 확인 (소프트웨어 층)

딥러닝 프레임워크가 GPU를 사용하려면 CUDA라는 통로가 필요합니다.

확인 사항: 위에서 실행한 `nvidia-smi` 결과 우측 상단의 **CUDA Version**을 확인  
내 그래픽 카드가 **지원할 수 있는 최대 버전**입니다.

설치: 최근 PyTorch나 TensorFlow는 라이브러리 설치 시 호환되는 CUDA 파일을 함께 설치해주므로, 별도의 **CUDA Toolkit** 설치 없이도 3단계로 넘어가도 무방한 경우가 많습니다.

| NVIDIA-SMI 561.09 |                         |      | Driver Version: 561.09 |                   |              | CUDA Version: 12.6 |         |     |
|-------------------|-------------------------|------|------------------------|-------------------|--------------|--------------------|---------|-----|
| GPU               | Name                    | Perf | Driver-Model           | Bus-Id            | Disp.A       | Volatile           | Uncorr. | ECC |
| Fan               | Temp                    |      | Pwr:Usage/Cap          |                   | Memory-Usage | GPU-Util           | Compute | M.  |
|                   |                         |      |                        |                   |              |                    | MIG     | M.  |
| 0                 | NVIDIA GeForce RTX 4060 | WDDM | 00000000:01:00.0       | On                |              |                    |         | N/A |
| 0%                | 50C                     | P3   | N/A / 115W             | 1212MiB / 8188MiB | 1%           | Default            |         | N/A |

## II. Deep Learning GPU/파이토치 환경설정

### 3단계: 가상환경 생성 및 패키지 설치

시스템 환경과 엉키지 않도록 가상환경을 먼저 만듭니다.

가상환경 생성 (VS Code 터미널):

```
python -m venv venv
```

가상환경 활성화 PyTorch (GPU 버전) 설치:

PyTorch 공식 사이트에서 본인 환경에 맞는 명령어를 생성해 설치하세요. (가장 중요)

# 예시 (CUDA 12.1 대응)

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
```

**그래픽카드(RTX 4060)와 드라이버 버전(561.09)은 CUDA 12.4~12.6 계열과 호환**

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu124
```

Conda를 사용하는 경우

```
conda install pytorch torchvision torchaudio pytorch-cuda=12.4 -c pytorch -c nvidia
```

## II. Deep Learning GPU/파이토치 환경설정

---

### 2. 설치 전 사전 환경 설정 (체크리스트)

설치 중 오류를 방지하기 위해 다음 사항을 먼저 확인하세요.

**기존 버전 삭제:** 만약 이미 CPU 버전이나 잘못된 버전의 PyTorch가 설치되어 있다면 충돌이 발생할 수 있습니다. 먼저 삭제해 주세요.

명령어: `pip uninstall torch torchvision torchaudio`

**버전 확인:** PyTorch는 보통 Python 3.9 ~ 3.12 환경에서 가장 안정적입니다.

확인: `python --version`

**가상환경 활성화:**

반드시 해당 가상환경이 활성화된 상태에서 설치 명령어를 입력해야 합니다.

## II. Deep Learning GPU/파이토치 환경설정

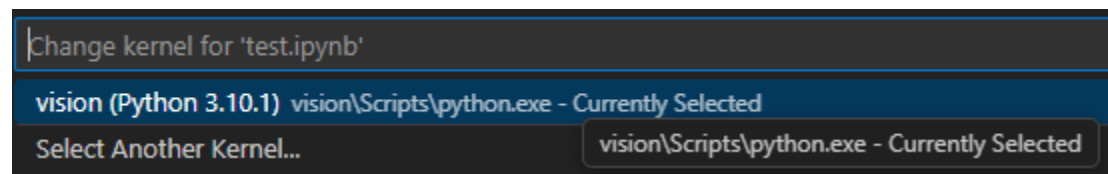
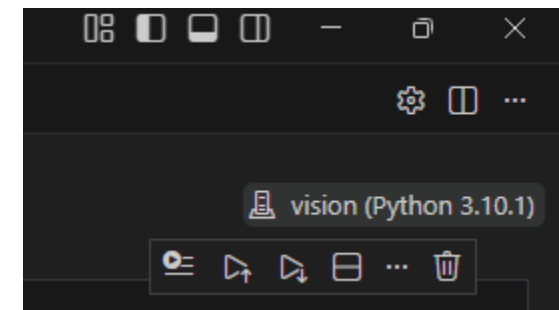
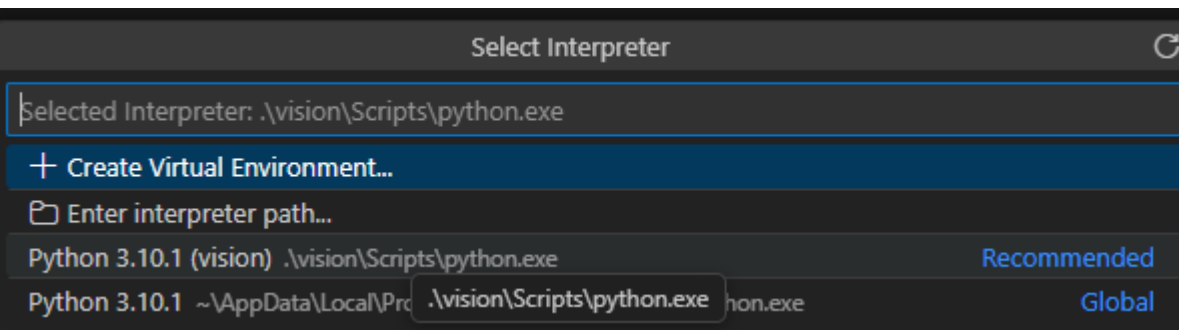
4단계: VS Code 인터프리터 및 커널 설정 (중요)

인터프리터 선택:

Ctrl + Shift + P → Python: Select Interpreter → 생성한 venv(가상환경) 선택.

Jupyter 사용 시:

우측 상단의 [Select Kernel] 버튼 클릭 → 목록에서 venv 파이썬(가상환경 파이썬) 선택.



## II. Deep Learning GPU/파이토치 환경설정

### 5단계: 최종 테스트 코드 실행

모든 설정이 끝났다면 아래 코드를 실행하여 확인합니다.

```
import torch

1. GPU 장치 개수 확인
gpu_count = torch.cuda.device_count()
print(f"사용 가능한 GPU 개수: {gpu_count}")

2. 현재 활성화된 GPU 이름 확인
if torch.cuda.is_available():
 print(f"현재 GPU 이름: {torch.cuda.get_device_name(0)}")

3. 실제 연산 테스트 (GPU로 텐서 이동)
a = torch.tensor([1.0, 2.0]).to("cuda")
print("GPU 연산 테스트 결과:", a * 2)
else:
 print("GPU를 사용할 수 없습니다. 설치 버전을 다시 확인하세요.")
```

```
사용 가능한 GPU 개수: 1
현재 GPU 이름: NVIDIA GeForce RTX 4060
GPU 연산 테스트 결과: tensor([2., 4.], device='cuda:0')
```

## II. Deep Learning GPU/파이토치 환경설정

---

### GPU 환경 설정시 개인컴퓨터의 인증서가 깨진 경우의 대처방법

#### 1. 인증서 업그레이드

Pip install -upgrade certify

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu124 --trusted-host download.pytorch.org
```

#### 2. 인증서 없이 강제로 설치 요청

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu124 --trusted-host download.pytorch.org
--trusted-host pypi.org --trusted-host files.pythonhosted.org --trusted-host pypi.python.org
```

#### 3. pip설정에 사이트 등록 후 요청

```
pip config set global.trusted-host "pypi.org files.pythonhosted.org pypi.python.org download.pytorch.org"
```

등록 확인 (설정 값이 출력되어야 합니다) : pip config list

등록 확인 후 다시 설치

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu124
```

## II. Deep Learning GPU/파이토치 환경설정

---

### 4. VS code에서 테스트 코드 실행시 ipykernel 설치오류가 발생하면

```
python -m pip install ipykernel -U --force-reinstall --trusted-host pypi.org --trusted-host files.pythonhosted.org --trusted-host pypi.python.org
```

## II. Deep Learning GPU/파이토치 환경설정

---

**Conda 기준**

**가상환경/GPU/파이토치 환경설정**



## II. Deep Learning GPU/파이토치 환경설정

---

### Conda 가상환경 생성 및 활성화

# 1. 'torch\_env'라는 이름의 가상환경 생성 (이름은 자유롭게 변경 가능)

```
conda create -n torch_env python=3.10 -y
```

# 2. 가상환경 활성화

```
conda activate torch_env
```

### PyTorch GPU 버전 설치

그래픽카드(4060), 최신 드라이버(561.09)를 사용을 기준으로,  
CUDA 12.4 빌드를 설치하는 것이 가장 안정적

```
conda install pytorch torchvision torchaudio pytorch-cuda=12.4 -c pytorch -c nvidia
```

### 설치 확인 및 GPU 연동 테스트

설치가 끝나면 아래 명령어를 한 줄씩 입력하여 GPU가 정상적으로 인식되는지 확인

```
python -c "import torch; print('PyTorch 버전:', torch.__version__); print('GPU 사용
가능 여부:', torch.cuda.is_available()); print('사용 중인 GPU:',
torch.cuda.get_device_name(0))"
```

## II. Deep Learning GPU/파이토치 환경설정

### Conda 가상환경 생성 및 활성화

## Miniconda Installer

Miniconda is a free minimal installer for conda. It is a small bootstrap version of Anaconda that includes only conda, Python, the packages they both depend on, and a small number of other useful packages (like pip, zlib, and a few others).

Download Miniconda Installer >

## Choose Your Download

Windows

Mac

Linux

### Anaconda Distribution

Complete package with 8,000+ libraries, Jupyter, JupyterLab, and Spyder IDE. Everything you need for data science.

[Windows 64-Bit Graphical Installer](#)

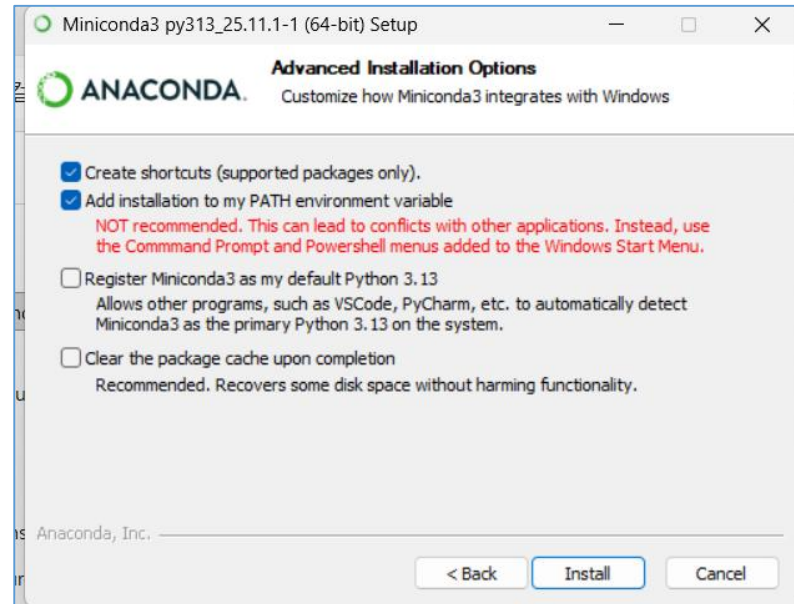
### Miniconda

Minimal installer with just Python, Conda, and essential dependencies. Install only what you need.

[Windows 64-Bit Graphical Installer](#)

## II. Deep Learning GPU/파이토치 환경설정

### Conda 가상환경 생성 및 활성화



`conda --version`

## II. Deep Learning GPU/파이토치 환경설정

### Conda 가상환경 생성 및 활성화

| 단계   | 명령어                          | 비고          |
|------|------------------------------|-------------|
| 생성   | conda create -n 이름 python=버전 | 특정 버전 지정 가능 |
| 활성화  | conda activate 이름            | 환경 진입       |
| 비활성화 | conda deactivate             | 현재 환경 탈출    |
| 조회   | conda env list               | 생성된 목록 확인   |
| 삭제   | conda env remove -n 이름       | 환경 완전 제거    |

## II. Deep Learning GPU/파이토치 환경설정

---

### Conda 가상환경 생성 및 활성화

- 새로운 환경을 만들 때는 이름과 파이썬 버전을 지정

명령어: **conda create -n [환경이름] python=[버전]**

예시: `conda create -n my_env python=3.10`

- 가상환경 실행 및 비활성화 (Activate / Deactivate)

생성한 환경 안으로 들어가거나, 다시 기본 환경으로 나올 때 사용합니다.

실행: **conda activate my\_env**

비활성화: **conda deactivate**

(어느 환경에 있든 이 명령어를 치면 기본(base) 환경으로 돌아옵니다.)

## II. Deep Learning GPU/파이토치 환경설정

### Conda 가상환경 생성 및 활성화

```
C:\Users\USER>conda --version
conda 25.11.1
```

```
C:\Users\USER>conda create -n my_env python=3.10
```

```
Do you accept the Terms of Service (ToS) for https://repo.anaconda.com/pkgs/main? [(a)ccept/(r)eject/(v)iew]: a
```

```
Do you accept the Terms of Service (ToS) for https://repo.anaconda.com/pkgs/r? [(a)ccept/(r)eject/(v)iew]: a
```

```
Do you accept the Terms of Service (ToS) for https://repo.anaconda.com/pkgs/msys2? [(a)ccept/(r)eject/(v)iew]: a
```

```
3 channel Terms of Service accepted
```

```
Retrieving notices: done
```

```
To deactivate an active environment, use
#
$ conda deactivate
```

```
C:\Users\USER>conda activate my_env
```

```
(my_env) C:\Users\USER>conda deactivate
```

```
C:\Users\USER>|
```

## II. Deep Learning GPU/파이토치 환경설정

---

### Conda 가상환경 생성 및 활성화

```
C:\lab\vision>conda activate my_env
'conda'은(는) 내부 또는 외부 명령, 실행할 수 있는 프로그램, 또는
배치 파일이 아닙니다.

C:\lab\vision>conda activate my_env

(my_env) C:\lab\vision>code .

(my_env) C:\lab\vision>
```

## II. Deep Learning GPU/파이토치 환경설정

---

### Conda 가상환경 생성(보안 오류)

Conda 가상환경 생성 및 활성화 시 오류 내용입니다.(하기 참조)

Retrieving notices: - Retrying (Retry(total=2, connect=None, read=None, redirect=None, status=None)) after connection broken by 'SSLError(SSLCertVerificationError(1, '[SSL: CERTIFICATE\_VERIFY\_FAILED] certificate verify failed: self-signed certificate in certificate chain (\_ssl.c:1032)'))': /pkgs/msys2/notices.json

Retrying (Retry(total=2, connect=None, read=None, redirect=None, status=None)) after connection broken by 'SSLError(SSLCertVerificationError(1, '[SSL: CERTIFICATE\_VERIFY\_FAILED] certificate verify failed: self-signed certificate in certificate chain (\_ssl.c:1032)'))': /pkgs/r/notices.json



## II. Deep Learning GPU/파이토치 환경설정

### Conda 가상환경 생성(보안 오류)

사내 보안망, 학교 실습실, 또는 강력한 백신이 사용되는 환경에서 발생.  
네트워크 중간에서 보안 장비가 패킷을 검사할 때 사용하는 '자체 서명 인증서'를  
Conda가 신뢰할 수 없는 외부 인증서로 판단하여 연결을 차단

1. SSL 검증 일시 비활성화 : `conda config --set ssl_verify false`

```
SSL 검증 끄기
conda config --set ssl_verify false

환경 생성 재시도 (예시)
conda create -n my_env python=3.10
```

2. Conda 설정 파일(.condarc) 직접 수정

1. 사용자 폴더(예: `C:\Users\사용자이름`)에서 `.condarc` 파일을 찾습니다. (안 보인다면 메모장을 열어 아래 내용을 작성 후 해당 경로에 저장하세요.)

2. 파일 내용을 다음과 같이 수정합니다.

```
channels:
 - defaults
ssl_verify: false
show_channel_urls: true
```

## II. Deep Learning GPU/파이토치 환경설정

### Requirements.txt

가상환경을 사용할 때 프로젝트에 필요한 모든 라이브러리의 목록과 버전을 기록해두는 명세서 역할

#### 1. 현재 가상환경의 패키지 목록 추출 (Freeze)

가상환경에서 개발을 마친 후, 설치된 라이브러리 목록을 파일로 저장할 때 사용  
가상환경 활성화 상태에서 터미널에 입력:

**pip freeze > requirements.txt**

이 명령을 실행하면 현재 폴더에 requirements.txt 파일이 생성되며, 설치된 패키지 이름과 버전(예: pandas==2.1.0)이 나열됩니다.

#### 2. requirements.txt를 사용하여 환경 복구 (Install)

새로운 컴퓨터에서 가상환경을 만들었거나, 다른 사람이 만든 프로젝트를 가져왔을 때 라이브러리를 한꺼번에 설치하는 방법

새 가상환경 활성화 상태에서 입력:

**pip install -r requirements.txt**

-r 옵션은 "해당 파일 안에 적힌 목록을 읽어서(read) 설치하라"

## II. Deep Learning GPU/파이토치 환경설정

---

### Requirements.txt

Conda 주로 사용하신다면, pip 명령어를 섞어서 쓸 때 주의할 점

Conda 환경에서도 pip 사용 가능: 콘다 가상환경을 활성화한 상태에서 위 pip 명령어들을 그대로 사용해도 해당 콘다 환경 안에만 설치됩니다.

**SSL 오류 발생 시:** `pip install --trusted-host pypi.python.org --trusted-host pypi.org --trusted-host files.pythonhosted.org -r requirements.txt`

## II. Deep Learning GPU/파이토치 환경설정

---

프로젝트에 필요한 모든 라이브러리의 목록과 버전을 기록해두는 명세서 역할

### Conda 전용 방식:

콘다 패키지(conda install) 위주로 사용하면 txt 대신 yaml 파일을 사용 권장

- 내보내기: `conda env export > environment.yaml`
- 가져오기: `conda env create -f environment.yaml`

## II. Deep Learning GPU/파이토치 환경설정

---

VScod에서 쥬피터노트북의 실행이 느려지는 경우 시도방법

- Step 1: 커널 강제 재시작 (Restart)
- Step 2: 인터프리터 다시 선택
- Step 3: ipykernel 재설치 (가장 확실한 방법)  
`pip install --upgrade --force-reinstall ipykernel --trusted-host pypi.org --trusted-host files.pythonhosted.org`
- Step 4: VS Code를 끄고 다시 실행시 관리자 권한으로 실행

## II. Deep Learning **H100 서버 사용**

---

**H100 서버 사용**

## II. Deep Learning **H100 서버 사용**

---

1. SSH 접속
2. 서버 상태 확인
3. 가상환경 구성
4. 코드/데이터 준비
5. GPU 선택
6. 학습 실행 (tmux)
7. 학습 진행 확인
8. Best 모델 확인
9. 로컬로 다운로드

## II. Deep Learning **H100 서버 사용**

---

### 1) SSH 접속명령어

```
ssh user@SERVER_IP
```

원격 H100 서버 접속

포트가 다르면:

```
ssh -p 2222 user@SERVER_IP
```



## II. Deep Learning H100 서버 사용

---

### 2) 서버 및 GPU 상태 확인

GPU 확인

nvidia-smi

H100 정상 인식 여부

다른 사용자 사용 중인지 확인

- Memory 사용량
- GPU Utilization

## II. Deep Learning H100 서버 사용

---

### 3) 작업 디렉토리 준비

```
cd ~
mkdir -p project
cd project
```

프로젝트 작업 공간 생성

## II. Deep Learning **H100 서버 사용**

---

### 4) 가상환경 구성 (Conda)

#### 4.1) 환경 생성

```
conda create -n torch_h100 python=3.10 -y
```

#### 4.2) 환경 활성화

```
conda activate torch_h100
```

프로젝트별 라이브러리 분리

## II. Deep Learning **H100 서버 사용**

---

### 5) PyTorch (H100용) 설치

pip install torch torchvision torchaudio --index-url <https://download.pytorch.org/whl/cu121>

확인

```
python -c "import torch; print(torch.cuda.is_available()); print(torch.cuda.get_device_name(0))"
```

체크포인트

True 출력

GPU 이름이 H100

## II. Deep Learning **H100 서버 사용**

---

### 6) 코드 및 데이터 준비

#### 방법 A (Git)

```
git clone <repo>
cd repo
```

#### 방법 B (SCP로 업로드)

로컬에서:

```
scp -r project user@SERVER_IP:/home/user/
```

## II. Deep Learning **H100 서버 사용**

---

### 7) 의존성 설치

```
pip install -r requirements.txt
```

## II. Deep Learning H100 서버 사용

---

### 8) GPU 선택 (다중 사용자 환경 핵심)

현재 상태 확인

`nvidia-smi`

GPU 지정 실행

`CUDA_VISIBLE_DEVICES=1 python train.py`

목적

다른 사용자와 충돌 방지

## II. Deep Learning H100 서버 사용

---

### 9) SSH 끊김 방지 (필수)

tmux 실행

```
tmux new -s train
```

학습 실행

```
CUDA_VISIBLE_DEVICES=1 python train.py > train.log 2>&1
```

세션 분리

```
Ctrl + b → d
```

재접속

```
tmux attach -t train
```



## II. Deep Learning **H100 서버 사용**

---

### 10) 학습 진행 확인

로그 확인

```
tail -f train.log
```

GPU 상태 확인

```
watch -n 1 nvidia-smi
```

## II. Deep Learning H100 서버 사용

---

### 11) Best 모델 생성 여부 확인

디렉토리 이동

```
cd ~/project/checkpoints
```

```
ls -lh
```

예

```
best.pt
```

```
last.pt
```

최신 파일 확인

```
ls -lt
```

## II. Deep Learning **H100 서버 사용**

---

### 12) 모델 정상 여부 확인 (선택)

Python

```
import torch
torch.load('checkpoints/best.pt', map_location='cpu')
```

### 13) 로컬로 모델 다운로드

로컬 PC (PowerShell)

```
scp user@SERVER_IP:/home/user/project/checkpoints/best.pt C:\models\
```

포트 지정 시

```
scp -P 2222 user@SERVER_IP:/home/user/project/checkpoints/best.pt C:\models\
```

### 14) 다운로드 검증로컬에서

```
import torch
torch.load("C:/models/best.pt", map_location="cpu")
```

## II. Deep Learning H100 서버 사용

---

## II. Deep Learning H100 서버 사용

---

## II. Deep Learning H100 서버 사용

---